

Bork in Action

Język, kompilator i mapa repozytorium

Na podstawie kodu Bork

2026

Spis treści

Przedmowa	7
O tej książce	8
Dla kogo jest ta książka	8
Jak czytać	8
Umowy typograficzne	9
Czego książka nie obiecuje	9
O źródłach tej książki	9
Rozdział 1. Pierwszy program i sposób jego uruchomienia	10
Ten rozdział obejmuje	10
Program, który wypisuje jeden wiersz	10
Sprawdzenie i zbudowanie	11
Dwa błędy, które pojawiają się na początku	12
Co język przyjmuje, a czego nie przetłumaczy na kod maszynowy	12
Drzewo regionów, zanim wyjaśnimy regiony	13
Podsumowanie	13
Rozdział 2. Po co Borkowi regiony pamięci	15
Ten rozdział obejmuje	15
Dwa rodzaje danych	15
Bufor regionu, nie pojedynczy obiekt	16
Trzy sposoby przejścia granicy regionu	16
Pętla nie może przenieść nazwy z zewnątrz	17
Czego ten model świadomie nie robi	18
Co już jest zrobione, a co jest odłożone	18
Kompromis, który warto nazwać wprost	18
Podsumowanie	19
Rozdział 3. Z czego składa się plik źródłowy	20
Ten rozdział obejmuje	20
Plik jest listą funkcji	20
Nowa linia kończy instrukcję	21
Nazwy stałe i nazwy zmienne	21
Blok jest instrukcją i otwiera region	21
Wyrażenie zapisane jako instrukcja	22
Łamanie wiersza wewnątrz nawiasów	22
Czego w składni nie ma	23

Podsumowanie	23
Rozdział 4. Typy danych i ich znaczenie	24
Ten rozdział obejmuje	24
Nazwy typów	24
Które wartości są kopiowane	25
Literały	25
Działania na liczbach i porównania	26
Brak wartości	27
Typ funkcji i nawiasy	28
Typ nieznany	28
Podsumowanie	28
Rozdział 5. Funkcje, parametry i wywołania	29
Ten rozdział obejmuje	29
Kształt funkcji	29
Własność na granicy wywołania	30
Funkcje wbudowane	31
Funkcja dopisana na końcu wywołania	31
Trzy formy przeniesienia przy bloku	32
Co z tego wynika dla programów, które mają się zbudować	32
Podsumowanie	33
Rozdział 6. Warunki, pętle i powrót z funkcji	34
Ten rozdział obejmuje	34
Warunek	34
Pętla po zakresie	35
Pętla dopóki i sterowanie obiegiem	36
Koniunkcja i alternatywa	36
Wczesny powrót i regiony	37
Podsumowanie	38
Rozdział 7. Tablice o znanej z góry długości	39
Ten rozdział obejmuje	39
Dlaczego długość należy do typu tablicy	39
Indeks	40
Wycinek o stałych granicach	40
Napis jako element	41
Jak tablica wygląda w gotowym programie	41
Podsumowanie	41
Rozdział 8. Kopiowanie, współdzielenie i przeniesienie wartości	43
Ten rozdział obejmuje	43
Własność nazw i czas życia bajtów mają jedną fazę w komunikacji	43
Znaczniki w drzewie	43
Przeniesienie zapisane przy nazwie	44
Słowo promote	44
Przypisanie do zmiennej z zewnątrz	45
Blok, który przenosi kilka nazw naraz	45

Pętla jeszcze raz	46
Stała i zmienna	46
Podsumowanie	47
Rozdział 9. Jak Bork przechowuje napisy w pamięci	48
Ten rozdział obejmuje	48
Adres i długość, a osobno znaki	48
Znaki powstają tam, gdzie stoisz	48
Zapis do zmiennej wybiera jej arenę	48
Dwie sąsiednie instrukcje mogą uniknąć drugiej kopii	49
Promote, gdy znaki już powstały za głęboko	49
Czego analiza ucieczki nie przepuści	49
Przykład z dokumentacji, który się nie kompiluje	50
Czego jeszcze nie ma	51
Podsumowanie	51
Rozdział 10. Komunikaty kompilatora i błędy podczas działania	52
Ten rozdział obejmuje	52
Język nie ma instrukcji obsługi błędu	52
Jak wygląda linia błędu	53
Błędy składni	53
Błędy typów	53
Błędy własności	54
Błędy generowania kodu	54
Awaria kompilatora	54
Gdy błędów jest kilka	55
Podsumowanie	55
Rozdział 11. Funkcje wbudowane i sposób pisania programów	56
Ten rozdział obejmuje	56
Modułów nie ma	56
Cała biblioteka, która istnieje	56
Jak pisać programy, które kompilator potrafi zbudować	57
Programy tylko do sprawdzenia	57
Zapisy, które wyglądają znajomo i są błędami	58
Styl, który pasuje do regionów	58
Podsumowanie	58
Rozdział 12. Od pliku źródłowego do gotowego programu	59
Ten rozdział obejmuje	59
Dwa wejścia, jedna wspólna kontrola	59
Co zwraca sprawdzenie	59
Dlaczego typy są przed własnością	61
Co która struktura pamięta	61
Co dzieje się przy budowaniu	61
Jak podzielony jest projekt	62
Czego w tej kolejności pracy nie ma	62
Podsumowanie	63

Rozdział 13. Jak kompilator czyta tekst programu	64
Ten rozdział obejmuje	64
Jedna gramatyka czyta i składa	64
Co lekser pomija, a co zamienia na token	64
Kolejność operatorów	65
Drzewo składni	65
Po co przerabiać nowe linie	66
Od błędu parsera do komunikatu	66
Podsumowanie	66
Rozdział 14. Jak kompilator sprawdza własność nazw	67
Ten rozdział obejmuje	67
Analiza własności opisuje nazwy, a nie bajty	67
Co pamięta jedno wiązanie	67
Jak otwiera się region	68
Reguła jednego odczytu	68
Pętla zabrania przenosić to, co było widać na wejściu	69
Warunek scala przeniesienia z obu gałęzi	69
Czego ta analiza nie sprawdza	69
Podsumowanie	70
Rozdział 15. Jak kompilator sprawdza typy	71
Ten rozdział obejmuje	71
Drugie drzewo opisuje program po typach	71
Jakie typy pojawiają się dopiero teraz	72
Jak sprawdzanie typów schodzi po programie	72
Oczekiwany typ steruje literałem i pustą wartością	72
Funkcje wbudowane są specjalnym przypadkiem	73
Dlaczego generator kodu nie może iść tylko po tym drzewie	73
Podsumowanie	73
Rozdział 16. Jak kompilator wybiera miejsce na napis	74
Ten rozdział obejmuje	74
Te trzy przejścia biegną tylko na programie bez wcześniejszych błędów	74
Wyniesienie alokacji rozpoznaje dokładnie dwie sąsiednie instrukcje	74
Analiza ucieczki liczy głębokość bajtów	75
Jeden spacer trzyma oznaczenie regionów i generator kodu w zgodzie	76
Który region pobiera bufor	76
Podsumowanie	77
Rozdział 17. Jak powstaje kod maszynowy	78
Ten rozdział obejmuje	78
Kontrola przed generowaniem kodu odrzuca konstrukcje, których emisja nie zna	78
Moduł wymaga funkcji main i deklaruje pozostałe funkcje wcześniej	78
Liczby, napisy i miejsce przeznaczenia	79
Regiony w wygenerowanym kodzie są stosem uchwytów	79
Co emisja naprawdę tłumaczy	80
Biblioteka wykonawcza trzyma pulę buforów	80
Konsolidacja używa clang i statycznego archiwum	81

Co zostało uruchomione przy pisaniu tej książki	81
Podsumowanie	81
Rozdział 18. Komunikaty błędów, polecenia i edytor	82
Ten rozdział obejmuje	82
Jeden komunikat ma fazę, treść i opcjonalny zakres	82
Dwa polecenia i kody zakończenia	82
Serwer liczy sprawdzenie od zera przy każdej zmianie	83
Podpowiedź pokazuje region i własność, a nie typ	83
Rozszerzenie uruchamia serwer z katalogu projektu	84
Podsumowanie	84
Rozdział 19. Jak czytać repozytorium	85
Ten rozdział obejmuje	85
Katalogi, które mają znaczenie	85
Za co odpowiada każdy moduł	86
Jak rosły warstwy widoczne w kodzie	86
Od czego zacząć czytanie	86
Pułapki przy nawigacji	87
Podsumowanie	89
Rozdział 20. Jeden program od źródła do uruchomienia	90
Ten rozdział obejmuje	90
Program, który zwraca trzy i wypisuje sumę	90
Czytanie składni widzi dwie funkcje	91
Sprawdzanie typów widzi liczby i napis	91
Analiza własności widzi kopię w pętli i współdzielenie w bloku	91
Oznaczenie buforów, wyniesienie i ucieczka nie mają tu pracy poza korzeniem funkcji	92
Kontrola, moduł i wygenerowane wywołania	93
Plik obiektowy, konsolidacja i proces	93
Czego ten ślad nie pokrywa	93
Podsumowanie	94
Rozdział 21. Testy, szukanie przyczyny błędu i nowa konstrukcja języka	95
Ten rozdział obejmuje	95
Dwa polecenia testów, bo generowanie kodu jest osobną opcją	95
Gdzie szukać, gdy wynik nie zgadza się z oczekiwaniem	96
Jak do języka doszła pętla while	97
Braki, które są już zapisane	98
Czego nie robić w pierwszej poprawce	98
Podsumowanie	98
Rozdział 22. Ćwiczenia	99
Ten rozdział obejmuje	99
Część A. Samo sprawdzenie	99
Część B. Zbudowany program	100
Część C. Kod kompilatora	100
Odpowiedzi do części A	101

Odpowiedzi do części B	101
Odpowiedzi do części C	101
Podsumowanie	102
Dodatek A. Zestawienie składni	103
Program i parametry	103
Typy	103
Instrukcje	103
Wyrażenia	104
Własność nazw w skrócie	105
Funkcje wbudowane i polecenia	105
Dodatek B. Słownik pojęć	106
Dodatek C. Czego kompilator jeszcze nie potrafi	108
Rzeczy świadomie niezaimplementowane	108
Gdzie dokument mówi co innego niż kod	108
Awarie i ostre krawędzie, które zostały uruchomione	109
Co jest pokryte testami	110
Kolejność zaufania	110

Przedmowa

Bork powstał z praktycznego zderzenia trzech przyzwyczajęń. W Kotlinie pisze się krótko. Blok kodu, funkcja przekazana na końcu wywołania i warunek zapisany jako wyrażenie nie wymagają od programisty osobnego rytuału. W Rustcie program natywny jest szybki i nie ma garbage collector, ale czas życia każdej referencji trzeba opisać i obronić przed sprawdzaniem pożyczek. W C pamięć zwalnia się ręcznie, więc każdy free jest osobną decyzją, którą łatwo pomylić.

Bork wybiera inną zasadę. Nawiasy klamrowe nie są tylko sposobem grupowania instrukcji. Oznaczają region pamięci. Napis albo tablica utworzona w takim bloku żyje tak długo, jak długo wykonanie pozostaje w tym bloku. Wyjście z bloku zwalnia cały bufor regionu naraz. Nie ma odśmiecania w tle i nie ma ogólnych referencji, które można by schować w polu struktury. Struktury zresztą w obecnej wersji języka nie ma.

Ta książka opisuje język taki, jaki przyjmuje kompilator w tym repozytorium, oraz sam kompilator, plik po pliku. To rozróżnienie jest ważne. Część kompilatora, która sprawdza program, rozumie wartości puste, operator `?:`, wykrzykniki `!!` oraz funkcję dopisaną na końcu wywołania. Generator kodu maszynowego tłumaczy węższy zestaw konstrukcji. Niektóre programy, które przechodzą sprawdzenie, dostają przy budowaniu zwykły komunikat błędu. Inne, i to trzeba powiedzieć wprost, kończą się awarią samego kompilatora. Dodatek C wymienia te miejsca.

Nazwa diagnostyki jest częścią projektu. Gdy program jest niepoprawny, komunikat mówi, w której fazie kompilator przerwał pracę. Język nie ma wyjątków, typu `Result` ani instrukcji `try`. Błąd programisty jest albo błędem kompilacji, albo przerwaniem programu w czasie działania, na przykład przy dzieleniu przez zero albo przy indeksie spoza tablicy.

Książka jest po polsku. Terminy, które w kodzie i w komunikatach kompilatora pozostają angielskie, podaję przy pierwszym użyciu i potem trzymam się jednego brzmienia. Słowo `move` w pliku źródłowym zostaje słowem `move`. W zdaniu objaśniającym mówię o przeniesieniu własności.

Jeśli chcesz najpierw napisać program, czytaj część pierwszą i trzymaj się przykładów, które dają się zbudować. Jeśli chcesz czytać kod kompilatora albo dopisać nową konstrukcję, część trzecia prowadzi przez repozytorium w kolejności, w której kolejne fazy naprawdę na sobie polegają.

O tej książce

Ta książka uczy języka Bork i jednocześnie objaśnia kod kompilatora napisanego w Rustcie. Nie jest listą życzeń. Jeśli jakiejś funkcji nie ma w parserze, w sprawdzaniu typów, w analizie własności albo w generatorze kodu, nie opisuję jej jako części języka. Jeśli występuje tylko w notatkach projektowych z katalogu docs/superpowers albo w pliku `TOD0.md`, zaznaczam, że jest to plan albo zaległość, a nie gotowa możliwość.

Dla kogo jest ta książka

Zakładam, że piszesz już w jakimś języku programowania. Najwygodniej będzie, jeśli znasz choć jeden z trzech punktów odniesienia, do których Bork sam się porównuje.

Kotlin albo inny język z blokami, z warunkiem użytym jako wyrażenie i z funkcją dopisywaną na końcu wywołania da ci znajomą składnię. Rust albo C++ da ci słownictwo własności i przeniesienia, nawet jeśli w Borku działa ono inaczej niż sprawdzanie pożyczek. Odrobina wiedzy o kompilatorach wystarczy w części drugiej. Wystarczy wiedzieć, że program najpierw jest czytany jako tekst, potem dostaje strukturę, potem jest sprawdzany, a na końcu zamieniany na kod maszynowy. Nie musisz znać biblioteki Inkwell ani protokołu serwera językowego.

Część pierwszą da się czytać bez zaglądania do plików Rusta. Część druga podaje nazwy funkcji i plików. Część trzecia zakłada, że masz repozytorium otwarte obok książki.

Jak czytać

Są trzy sensowne przejścia.

Ścieżka języka prowadzi przez przedmowę, rozdziały od 1 do 11 oraz dodatki A i C. Przy każdym listingu, o którym piszę, że został uruchomiony, warto uruchomić go samodzielnie. Listing, o którym piszę, że przechodzi tylko sprawdzenie, kompilator akceptuje poleceniem `bork plik.bork`, ale polecenie `bork build` albo go odrzuca, albo kończy się awarią kompilatora.

Ścieżka kompilatora zaczyna się od rozdziału 2, bo bez powodu istnienia regionów dalsze fazy wyglądają jak zbiór niepowiązanych przejść. Potem idą rozdziały od 12 do 18 i rozdział 20. Rozdział 20 przeprowadza jeden krótki program przez kolejne funkcje kompilatora.

Ścieżka osoby, która chce zmieniać kompilator, to rozdziały od 19 do 22 i dodatek C. Rozdział 21 pokazuje, które testy łapią którą fazę, i jak nowe słowo kluczowe, na

przykład `while`, musiało przejść przez wszystkie warstwy.

Komunikaty w książce pochodzą z uruchomienia programu `bork` złożonego w tym środowisku z LLVM 23.1.2. Linia błędu ma postać `plik:linia:kolumna: error: faza: treść`. Kolumna liczy znaki od początku wiersza. Gdy błąd nie ma pozycji w pliku, numer linii znika. Tak jest na przykład przy próbie zwrócenia wyniku funkcji `concat`.

Umowy typograficzne

Listingi są numerowane w obrębie rozdziału. Pod listingiem zdania omawiają, co program robi i jaki był wynik uruchomienia, jeśli uruchomienie było możliwe. Numer w nawiasie, na przykład (1), odsyła do komentarza w kodzie tylko wtedy, gdy kod naprawdę ma taki znacznik.

Ramki mają trzy role. Nota dopowiada fakt, który łatwo przeoczyć. Wskazówka mówi, co zrobić w praktyce. Ostrzeżenie dotyczy programu, który sprawdzenie akceptuje, a budowanie odrzuca albo na którym kompilator się wywraca. Osobno ostrzeżenie dotyczy zachowania, które przerywa już uruchomiony program.

Kod Borka jest w blokach oznaczonych `bork`. Kod Rusta, który jest częścią kompilatora, jest w blokach `rust` i ma w podpisie ścieżkę pliku. Diagramy są zapisane w Mermaid. W pliku PDF są obrazkami.

Czego książka nie obiecuje

W tej wersji Bork nie ma modułów, polecenia `import`, pakietów ani przestrzeni nazw. Nie ma struktur, wyliczeń definiowanych przez programistę, cech ani typów ogólnych. Nie ma wyjątków. Nie ma jednoargumentowego minusa, więc nie zapiszesz literału ujemnego. Instrukcji nie rozdziela się średnikiem. Nie ma garbage collector. Nie ma referencji w stylu `& i &mut` z Rusta. Nie ma interpretera, trybu `bork run` ani debuggera. Generator kodu nie tłumaczy funkcji dopisanej na końcu wywołania, wartości `Some` i `None`, operatora `?:` ani asercji `!!`.

Napis przekazany do funkcji napisanej przez programistę jest akceptowany przy sprawdzaniu. Przy budowaniu kompilator przerywa pracę awaryjnie. To błąd kompilatora, a nie reguła języka. Rozdział 17 pokazuje miejsce w kodzie.

O źródłach tej książki

Rozdziały leżą w katalogu `docs/book`. Przykłady, które uruchamiałem, są w `docs/book/przykłady` razem z plikiem `WYNIKI.md`. Historia commitów projektu jest tłem rozdziału 19. Nie streszczam każdego pull requestu.

Język i kompilator są projektem Andrzeja Witkowskiego. Ta książka jest opisem tego kodu dla innych programistów i nie zmienia semantyki. Gdy dokument `docs/language.md` rozmija się z kompilatorem, opisuję to, co robi kompilator, i mówię, na czym polega różnica. Najważniejszy przykład dotyczy konkatenacji dwóch zmiennych napisowych wewnątrz bloku. Dokument pokazuje wywołanie bez `move`. Kompilator takiego programu nie przyjmuje.

Rozdział 1. Pierwszy program i sposób jego uruchomienia

Ten rozdział obejmuje

- czym jest program napisany w Borku
- jak sprawdzić plik, nie budując programu wykonywalnego
- jak zbudować program i jak odczytać jego kod wyjścia
- jak czytać pierwszy komunikat o błędzie
- które konstrukcje język już rozumie, a których generator kodu jeszcze nie tłumaczy

Program, który wypisuje jeden wiersz

Bork jest językiem kompilowanym do kodu maszynowego. Kompilator czyta plik tekstowy, sprawdza, czy program jest poprawny, i na życzenie tłumaczy go na plik, który da się uruchomić. Składnia jest zbliżona do Kotlin. Funkcję rozpoczyna słowo `fun`, nazwę niezmienną słowo `val`, nazwę zmienną słowo `var`, a grupę instrukcji obejmuje się nawiasami klamrowymi. Pamięcią nie zarządza garbage collector. Sens tej decyzji wyjaśnia następny rozdział. Tutaj celem jest zobaczyć działający program i pierwszy błąd.

Listing 1.1. Funkcja `main` wypisuje słowo i przechodzi do nowego wiersza.

```
fun main() {  
    println("hi")  
}
```

Program składa się z funkcji. Nie ma instrukcji zapisanych luzem na poziomie pliku. `main` jest zwykłą funkcją, która przy budowaniu programu wykonywalnego dostaje szczególną rolę. Punkt wejścia procesu nazywa się właśnie tak. Jeśli nie podasz typu wyniku, funkcja nie zwraca wartości. Taki `main` kończy się kodem wyjścia zero. `println` jest funkcją wbudowaną. Wypisuje argument i dodaje znak nowego wiersza. Ten program został zbudowany i uruchomiony. Na standardowym wyjściu pojawił się tekst `hi` oraz nowy wiersz, a proces zakończył się kodem zero.

Listing 1.2. Wynik funkcji `main` staje się kodem wyjścia procesu.

```
fun add(a: i32, b: i32): i32 {  
    return a + b  
}
```

```
fun main(): i32 {
    return add(40, 2)
}
```

Funkcja `add` przyjmuje dwie liczby typu `i32` i zwraca ich sumę. Parametr zapisany jako nazwa: `Typ`, bez słowa `val` albo `var`, jest nazwą niezmienną. `main` z zadeklarowanym wynikiem `i32` przekazuje tę liczbę systemowi jako kod wyjścia. Po zbudowaniu i uruchomieniu proces kończy się kodem 42. Innych typów wyniku funkcji `main` generator kodu nie obsługuje. Próba zwrócenia `i64`, `f64` albo `bool` przechodzi sprawdzenie typów, a przy budowaniu dostaje komunikat, że taki wynik nie jest jeszcze obsługiwany.

NOTA

NOTA. Sprawdzenie pliku bez funkcji `main` może się udać, jeśli reszta programu jest poprawna. Budowanie programu wykonywalnego wymaga `main`. Brak tej funkcji jest błędem fazy generowania kodu, a nie błędem typu.

Sprawdzenie i zbudowanie

Samą poprawność programu sprawdzisz bez biblioteki LLVM. LLVM to biblioteka, która później tłumaczy sprawdzony program na kod maszynowy. Na tym etapie nie jest potrzebna. Z katalogu repozytorium:

```
cargo run --bin bork -- sciezka/do/pliku.bork
cargo run --bin bork -- --dump-arenas sciezka/do/pliku.bork
```

Pierwsze polecenie uruchamia sprawdzenie. Obejmuje ono składnię, typy i własność nazw. Drugie polecenie dopisuje na standardowe wyjście drzewo regionów. Komunikaty o błędach idą na standardowe wyjście błędów. Kod zero oznacza brak błędów. Kod jeden oznacza błąd w programie. Kod dwa oznacza złe wywołanie albo problem z odczytem pliku.

Zbudowanie programu wymaga opcji kompilacji `codegen`, biblioteki LLVM 23, kompilatora `clang` i bibliotek, z którymi ta wersja LLVM jest powiązana. Szczegóły instalacji są w pliku `README` repozytorium. Ubuntu 24.04 nie ma LLVM 23 w domyślnym archiwum pakietów. Po złożeniu kompilatora budujesz program tak:

```
cargo run --features codegen -- build plik.bork
cargo run --features codegen -- build -o wyjście plik.bork
```

Jeśli nie podasz `-o`, nazwa pliku wynikowego powstaje przez odcięcie rozszerzenia `.bork`. Kompilator odmawia pracy, gdy plik wynikowy nadpisałby plik źródłowy. Komunikat brzmi wtedy `error: output path would overwrite input file`, a kod wyjścia wynosi dwa. Bez opcji `codegen` pod polecenie `build` kończy się informacją, że ta opcja jest wymagana, również kodem dwa.

Listing 1.3. Próba zbudowania pliku, w którym nie ma funkcji `main`.

Źródło to jedna funkcja pomocnicza zwracająca liczbę 7. Budowanie kończy się kodem jeden i nie zostawia pliku wykonywalnego. Komunikat, przepisany z uruchomienia, brzmi:

```
nomain.bork: error: codegen: `fun main` is required to build an executable
```

Słowo codegen w komunikacie jest nazwą fazy. Sprawdzenie tego samego pliku, bez budowania, kończy się sukcesem. Język nie wymaga, żeby każdy plik miał main. Wymaga tego dopiero tłumaczenie na program wykonywalny.

Dwa błędy, które pojawiają się na początku

Instrukcje rozdziela się nową linią, nie średnikiem. Dwie instrukcje w jednym wierszu nie są programem.

Listing 1.4. Parser odrzuca dwie deklaracje zapisane bez przejścia do nowego wiersza.

```
fun main() { val a = 1 val b = 2 }
```

Komunikat zaczyna się od fazy parse i mówi, że token val jest nieoczekiwany, a wśród rzeczy oczekiwanych jest NL, czyli znak nowej linii. Lista oczekiwanych tokenów jest surowa. Pochodzi wprost z generatora parserów i nie została przepisana na zdanie dla człowieka. Średnik jest traktowany tak samo. W wierszu val n = 1; kompilator zgłasza nieoczekiwany token średnika.

Drugi częsty błąd dotyczy zmiany nazwy, która miała pozostać stała.

Listing 1.5. Przypisanie do nazwy wprowadzonej przez val.

```
fun main() {  
    val n = 1  
    n = 2  
}
```

Komunikat, z kolumną wskazującą przypisanie, brzmi:

```
err_val.bork:3:5: error: type: cannot assign to immutable `val` binding `n`
```

Faza type pochodzi ze sprawdzania typów. Faza ownership pochodzi z analizy własności. Obie mogą pojawić się w jednym uruchomieniu. W wydruku błędy własności stoją przed błędami typów, chociaż sprawdzanie typów wykonuje się wcześniej. Powód tej kolejności jest techniczny i wraca w rozdziale 12. Tutaj wystarczy czytać fazę w komunikacie i nie zakładać, że pierwsza linia jest zawsze pierwszym błędem, który kompilator znalazł w czasie.

Co język przyjmuje, a czego nie przetłumaczy na kod maszynowy

Poniższy program przechodzi sprawdzenie. Jest to próbka zapisana w kodzie kompilatora jako MVP_SAMPLE. Budowanie kończy się kodem jeden. Test build_rejects_mvp_sample_a pilnuje, żeby w komunikacie były słowa codegen i not supported.

Listing 1.6. Funkcja przekazana na końcu wywołania. Sprawdzenie kończy się sukcesem. Budowanie jest odrzucane.

```
fun action(a: Int, b: Int, block: (Int, Int) -> Int): Int {  
    return block(a, b)
```

```

}

fun main(): i32 {
    val threshold = 5
    var accumulator = 0
    for (i in 0..10) {
        accumulator = action(accumulator, i) { acc, current ->
            if (current > threshold) {
                acc + current
            } else {
                acc
            }
        }
    }
    return accumulator
}

```

To jest prawdziwy fragment języka, a nie martwa reguła gramatyki. Sprawdzanie typów kontroluje liczbę parametrów funkcji dopisanej na końcu i typ jej wyniku. Analiza własności buduje dla niej osobny region. Generator kodu zatrzymuje się wcześniej i wypisuje trailing closures are not supported by codegen.

OSTRZEŻENIE

OSTRZEŻENIE. Nie każdy program odrzucony przy budowaniu dostaje taki komunikat. Przekazanie napisu do funkcji napisanej przez programistę przechodzi sprawdzenie, a potem proces kompilatora przerywa się awaryjnie, z kodem 101 i śladem stosu Rusta. Funkcje wbudowane `println` i `concat` tego nie robią. Rozdział 5 i rozdział 17 opisują tę różnicę na konkretnych plikach.

Drzewo regionów, zanim wyjaśnimy regiony

Dla listingu 1.2 polecenie z `--dump-arenas` wypisuje drzewo. Węzeł `fun add` ma dwie nazwy lokalne, a oraz b. Węzeł `fun main` nie ma nazw lokalnych, bo jedyną instrukcją jest `return`. Znacznik `Local` przy nazwie oznacza, że nazwa powstała w tym regionie. W rozdziale 8 pojawią się pozostałe znaczniki: kopiowanie, współdzielenie i przeniesienie. Ten sam tekst drzewa pokazuje w edytorze polecenie `Bork: Dump Arenas`.

Podsumowanie

- Program w Borku jest listą funkcji, a instrukcje rozdziela nowa linia.
- Polecenie `bork plik.bork` sprawdza program. Polecenie `bork build` tłumaczy obsługiwany podzbiór na plik wykonywalny.
- Funkcja `main` z wynikiem `i32` przekazuje ten wynik jako kod wyjścia. Funkcja `main` bez typu wyniku kończy się zerem.
- Komunikat błędu ma fazę. Na tym etapie spotkasz `parse`, `type` i `codegen`. Faza `ownership` dojdzie w rozdziałach o własności.

- Funkcja dopisana na końcu wywołania jest częścią języka sprawdzanego przez kompilator i nie jest jeszcze tłumaczona na kod maszynowy.

Rozdział 2. Po co Borkowi regiony pamięci

Ten rozdział obejmuje

- jaki problem z pamięcią Bork chce rozwiązać bez garbage collector'a i bez sprawdzania pożyczek
- czym jest region i czym jest arena
- trzy legalne sposoby użycia wartości spoza bieżącego bloku
- co już działa w kompilatorze, a co jest jeszcze świadomie odłożone

Dwa rodzaje danych

W programie systemowym są dane, które mieszczą się w rejestrze albo na stosie, i dane, które tam się nie mieszczą. Liczba, wartość logiczna i brak wartości użytkowej należą do pierwszej grupy. Napis i tablica należą do drugiej. Wartość napisu w gotowym programie jest adresem bajtów oraz długością. Same znaki leżą gdzie indziej. Trzeba wiedzieć, kto te bajty zwalnia i skąd kompilator wie, że adres jest jeszcze ważny.

Garbage collector odpowiada, że środowisko wykonawcze zwolni pamięć kiedyś, a programista nie wskazuje chwili. Sprawdzanie pożyczek w Rustcie odpowiada, że każda referencja ma czas życia dowiązany do konkretnego wypożyczenia i że to wypożyczenie nie może przeżyć właściciela. Oba rozwiązania są dobre w swoich miejscach. Bork nie chce płacić ich kosztu. Odśmiecanie odbiera programiście pewność, kiedy pamięć wróci do puli. Ogólne sprawdzanie pożyczek wymaga składni i komunikatów, których ten projekt świadomie nie buduje.

Odpowiedź Borka jest zapisana w docs/memory-model.md i zrealizowana w analizie własności oraz w bibliotece `bork_runtime`. Blok ograniczony nawiasami klamrowymi otwiera region. Region ma arenę, czyli jeden bufor, do którego dopisuje się kolejne bajty przez przesuwanie wskaźnika. Wyjście z bloku ustawia ten wskaźnik z powrotem na początek. Cały bufor wraca do puli buforów wolnych. Nie zwalnia się pojedynczego napisu w środku regionu.

Dalej używam słowa region w tym właśnie znaczeniu. Region jest jednocześnie fragmentem programu i czasem życia pamięci z nim związanej. Słowo arena oznacza bufor tego regionu. W kodzie kompilatora nazwa arena pojawia się jeszcze raz, w innym sensie. Wrócę do tego pod koniec rozdziału, bo pomieszczenie tych dwóch rzeczy utrudnia czytanie źródeł.

Bufor regionu, nie pojedynczy obiekt

Listing 2.1. Napis zapisany do zmiennej z zewnątrz bloku pozostaje dostępny po wyjściu z bloku.

```
fun main() {  
    var outer = "a"  
    {  
        outer = "b"  
    }  
    println(outer)  
}
```

Program został zbudowany i uruchomiony. Na standardowym wyjściu jest b oraz nowy wiersz, a kod wyjścia wynosi zero. Zmienna outer powstała w regionie funkcji. Przypisanie wewnątrz bloku zmienia tę zmienną. Nie jest próbą wyniesienia nazwy, która żyje tylko w bloku. Bajty nowego napisu trafiają do areny zmiennej outer, a nie do areny wewnętrznego bloku, który za chwilę zniknie. Ten mechanizm nazywa się alokacją w miejscu przeznaczenia. Rozdział 9 rozpisuje go na reguły. Tutaj widać skutek. Po zamknięciu nawiasu napis "b" nadal wolno wypisać.

Gdybyś wprowadził wewnątrz bloku nazwę inner i próbował użyć jej za zamykającym nawiasem, nazwa w ogóle nie jest widoczna. To jest zwykły zasięg leksykalny i nie trzeba do niego areny. Region nie przedłuża zasięgu nazwy. Nazwa umiera według bloków, a bajty umierają według areny, w której zostały położone. Te dwie rzeczy są związane, ale nie są tym samym.

Trzy sposoby przejścia granicy regionu

Wartość z regionu otaczającego można w regionie wewnętrznym użyć na trzy sposoby. Kompilator wybiera sposób według typu i według tego, czy nazwa jest stała, czy zmienna.

Kopiowanie dotyczy typów, które mieszczą się w wartości samej i nie są puste. Należą do nich i32, bool i pozostałe niepuste typy proste. Odczyt kopiuje bity. Nazwa w regionie zewnętrznym zostaje żywa.

Współdzielenie dotyczy stałej, której typ nie jest kopiowany, na przykład napisu wprowadzonego przez val. Region wewnętrzny patrzy na te same bajty. Nazwa zewnętrzna zostaje żywa. Jest to bezpieczne bez sprawdzania pożyczek, bo region wewnętrzny kończy się wcześniej niż region zewnętrzny, a język nie ma typu referencji, którą dałoby się zapisać w polu na dłużej.

Przeniesienie dotyczy zmiennej, której typ nie jest kopiowany, albo jawnego słowa move. Własność przechodzi na nowe użycie, a stara nazwa staje się martwa.

Listing 2.2. Stały napis jest w bloku wewnętrznym tylko oglądany.

```
fun main() {  
    val s = "x"  
    {  
        println(s)  
    }  
}
```

```

    }
}

```

Program został zbudowany. Wypisuje `x` i nowy wiersz. Drzewo regionów pokazuje w bloku wewnętrznym nazwę `s` ze znacznikiem współdzielenia z regionu `fun main`. Region wewnętrzny nie kopiuje znaków i nie odbiera własności.

Listing 2.3. Po przeniesieniu stara nazwa nie może być użyta.

```

fun main() {
    var s: String = "hi"
    val t = move s
    val u = s
}

```

Sprawdzenie kończy się błędem fazy ownership:

```
err_use_after.bork:4:13: error: ownership: use of `s` after move from fun main
```

Przeniesienie nie wycina bajtów ze starej areny. Zostają one miejscem, do którego nie ma już dojścia, aż arena zostanie wyczyszczona. Kompilator unieważnia nazwę. To wystarcza, dopóki nie ma struktur, w których dałoby się przechować adres. Struktur definiowanych przez programistę nie ma.

Słowo `move` przy wartości kopiowanej nie odbiera nazwy. Program, który robi `val n = 1`, potem `val m = move n`, a potem zwraca `n + m`, przechodzi sprawdzenie i budowanie. Kod wyjścia wynosi 2. Drzewo regionów nie oznacza `n` jako przeniesionej.

Pętla nie może przenieść nazwy z zewnątrz

Ciało pętli wykonuje się wiele razy. Własność zmiennej utworzonej przed pętlą da się oddać tylko raz. Analiza własności zapamiętuje nazwy widoczne przy wejściu do `for` albo `while` i zabrania przenosić je w ciało.

Listing 2.4. Przeniesienie napisu utworzonego przed pętlą jest błędem.

```

fun main() {
    var s = "Hello, World!"
    for (i in 1..10) {
        move {
            val t = s
        }
    }
}

```

Kompilator wypisuje dwa komunikaty na tym samym miejscu. Pierwszy mówi, że `s` nie może być przeniesione wewnątrz pętli, bo przy kolejnym obiegu byłoby już przeniesione. Drugi mówi, że `s` nie jest kopiowane i trzeba je przenieść do bloku `move`. Dwa zdania wynikają z dwóch sprawdzeń, które widzą ten sam zapis. Nazwę utworzoną wewnątrz ciała pętli wolno przenieść, bo każdy obieg dostaje świeżą zmienną. Przykład jest w rozdziale 8.

Ciało pętli czyści swoją arenę przy każdym obiegu. Pamięć zajmowana przez wartości tymczasowe pętli nie rośnie razem z liczbą obiegów, o ile wynik nie jest zapisywany do zmiennej z zewnątrz. To jest osobna sprawa od zakazu przenoszenia. Poprawna pętla, która buduje tymczasowy napis i nie wynosi go na zewnątrz, zużywa stałą ilość pamięci regionu.

Czego ten model świadomie nie robi

Dokument modelu pamięci wymienia rzeczy, których nie planuje. Nie będzie ogólnych referencji ani referencji zmiennych trzymanych w polach między regionami. Nie będzie niejawnego głębokiego kopiowania przy każdym odczycie spoza regionu. Nie będzie zapisywania wypożyczenia z regionu wewnętrznego do regionu zewnętrznego. Jedyna obserwacja krótsza niż właściciel to współdzielenie stałej z regionu otaczającego. Region wewnętrzny i tak jest w nim zagnieżdżony w tekście programu.

Co już jest zrobione, a co jest odłożone

Bufor areny ma pojemność 4096 bajtów. Stała nazywa się `ARENA_CAPACITY` i leży w `crates/bork_runtime/src/lib.rs`. Taką samą pojemność ma model opisany w `src/arena.rs`. Wejście do regionu, który naprawdę alokuje, pobiera bufor z puli. Wyjście zwraca bufor do puli. Czyszczenie wskaźnika bez zwrotu bufora jest używane w pętli, żeby nie oddawać pamięci tylko po to, by za chwilę wziąć ją z powrotem. Przepelnienie bufora przerywa program komunikatem Rusta `arena overflow`, z podaną liczbą bajtów i pojemnością 4096. Nie jest to błąd, który program w Borku mógłby obsłużyć.

Odłożone są rzeczy opisane w tym samym dokumencie i widoczne w kodzie analizy ucieczki. Analiza ucieczki, po angielsku `escape analysis`, sprawdza, czy bajty wartości nie będą użyte po zwolnieniu areny, w której powstały. Dziś nie ma osobnego bufora na wynik zwracany do funkcji wywołującej, więc nie zwrócisz świeżo sklejonego napisu. Przeniesienie alokacji do zewnętrznej zmiennej rozpoznaje tylko bardzo prosty układ dwóch sąsiednich instrukcji. Słowa `promote` nie używa się przy `return`. Generator kodu nie pomija kopiowania pamięci nawet wtedy, gdy bajty już leżą we właściwej arenie.

NOTA

NOTA. Plik `src/arena.rs` opisuje bufor 4096 bajtów i pulę takich buforów. Analiza własności go nie wywołuje. Ta analiza buduje drzewo regionów. Prawdziwe bufory powstają w `bork_runtime`, gdy generator kodu wstawi wywołanie `bork_arena_push`. Czytając źródła, nie traktuj tych dwóch plików jako jednej struktury.

Kompromis, który warto nazwać wprost

Region jest gruby. Napis i duża tablica utworzone w tym samym bloku dzielą jeden bufor 4096 bajtów i giną razem. Nie da się zwolnić napisu wcześniej. To jest cena braku `free` i braku `garbage collector`. Stały rozmiar upraszcza generator kodu, bo region ma jeden wskaźnik końca danych i jedno czyszczenie. Gdy dane mają przeżyć blok, trzeba zapisać

je do zmiennej z regionu zewnętrznego. Nie ma sterty, która żyłaby dłużej niż funkcja, poza literałami umieszczonymi w stałych programu.

Litera "hi" może w ogóle nie leżeć w arenie. Generator kodu kładzie znaki w stałej, a w wartości zostawia adres i długość. Koniec bloku nie unieważnia wtedy znaków. Mimo to nazwa po move jest martwa. Stały napis nie jest powodem, żeby używać nazwy po przeniesieniu.

Podsumowanie

- Blok w nawiasach klamrowych jest regionem. Alokacja dopisuje bajty do bufora, a wyjście z bloku zwalnia bufor w całości.
- Arena w tej książce oznacza bufor regionu. W źródłach kompilatora podobna nazwa oznacza także osobny model tego bufora, którego analiza własności nie używa.
- Kopiowanie, współdzielenie i przeniesienie są trzema legalnymi sposobami użycia wartości spoza bieżącego regionu.
- Przeniesienie unieważnia nazwę. Nie porządkuje bajtów w starym buforze.
- Pętla nie przenosi nazwy utworzonej poza jej ciałem i czyści arenę ciała przy każdym obiegu.
- Pojemność jednego bufora wynosi 4096 bajtów. Przepełnienie przerywa program.
- Ogólnych referencji nie planuje się. Zwrot świeżo zbudowanego napisu z funkcji też jeszcze nie działa, bo brakuje bufora należącego do wywołującego.

Rozdział 3. Z czego składa się plik źródłowy

Ten rozdział obejmuje

- jak wygląda plik z programem i jak zapisuje się komentarz
- dlaczego nowa linia rozdziela instrukcje
- czym różni się nazwa stała od nazwy zmiennej
- kiedy kilka par nawiasów klamrowych staje się jednym regionem
- dlaczego wywołanie funkcji wolno zapisać w wielu wierszach

Plik jest listą funkcji

Plik o rozszerzeniu `.bork` zawiera zero lub więcej funkcji. Pusty plik oraz plik złożony z samej pustej linii dają się odczytać jako program bez funkcji. Komentarz zaczyna się od `//` i trwa do końca wiersza. Nie ma komentarza blokowego, atrybutów ani polecenia `import`.

Listing 3.1. Komentarz nie zmienia wyniku funkcji.

```
fun main(): i32 {  
    // komentarz  
    val n = 1 // też komentarz  
    return n  
}
```

Program został zbudowany. Proces kończy się kodem wyjścia 1, bo funkcja zwraca liczbę 1.

Słowa rozpoznawane przez lekser jako osobne tokeny to `if`, `fun`, `val`, `var`, `for`, `in`, `return`, `Some`, `None`, `move` i `promote`. Słowa `while`, `break`, `continue`, `true` i `false` są w gramatyce zapisane wprost. Nie użyjesz ich jako nazw. Słowo `else` ma osobną regułę. Przed nim wolno złamać wiersz i wstawić komentarz, a parser i tak widzi `else`, nie osobny koniec instrukcji. Dzięki temu warunek może mieć drugą gałąź w następnym wierszu.

Nazwa składa się z liter alfabetu łacińskiego, cyfr i znaku podkreślenia, a nie może zaczynać się od cyfry. Polskie znaki w nazwach nie są przyjmowane.

Nowa linia kończy instrukcję

Listing 3.2. Średnik nie jest częścią języka.

Źródło `val n = 1;` kończy się błędem fazy `parse`. Komunikat mówi, że token średnika jest nieoczekiwany i że w tym miejscu parser spodziewa się między innymi nowej linii albo końca bloku.

Pusta linia między instrukcjami jest w porządku, bo kilka przejść do nowego wiersza skleja się w jeden separator. Dwie instrukcje bez tego separatora nie są poprawne. Gdy komunikat wymienia token `NL`, prawie zawsze dwie rzeczy stoją w jednym wierszu albo w kodzie został średnik przyniesiony z C albo z Rusta.

Nazwy stałe i nazwy zmienne

Listing 3.3. `val` wprowadza nazwę, której nie wolno przypisać ponownie. `var` wprowadza nazwę, którą wolno zmienić.

```
val n = 1
var count: i32 = 0
var s: String = "hi"
count = count + 1
```

Adnotacja typu po dwukropku jest opcjonalna, gdy wyrażenie po prawej stronie ma znany typ. Przypisanie do nazwy stałej jest błędem sprawdzania typów. Tekst tego błędu był w rozdziale 1. Nazwa wprowadzona w bloku przesłania nazwę z bloku zewnętrznego aż do końca bloku wewnętrznego. Inicjalizator nowej nazwy może jeszcze odczytać przesłanianą nazwę. Program, który na zewnątrz ma `val n = 1`, a w bloku `val n = n + 1` i zwraca to wewnętrzne `n`, został zbudowany i kończy się kodem 2. Drzewo regionów pokazuje w bloku lokalne `n` oraz osobną obserwację, że zewnętrzne `n` zostało skopiowane.

Przypisanie po lewej stronie może dotyczyć tylko nazwy albo elementu tablicy zapisanego jako `nazwa[indeks]`. Pole, wywołanie i dowolne inne wyrażenie po lewej stronie znaku równości nie przechodzą składni. Komunikat gramatyki brzmi `assignment target must be a name or name[index]`. Ten rodzaj błędu, zgłaszany przez samą gramatykę, dostaje pozycję na końcu pliku, a nie przy lewym składniku. To ograniczenie tłumaczenia błędu parsera na diagnostykę, opisane w `diag::from_parse`. Nie jest decyzją, że błąd „jest na końcu programu”.

Słowo `var` albo `val` przy lokalnej nazwie stoi na początku instrukcji i jest obowiązkowe. Nie ma zapisu `n := 1` ani samego `n = 1` w roli deklaracji. Przy parametrze funkcji te słowa są opcjonalne. Rozdział 5 wyjaśnia, co wtedy znaczą.

Blok jest instrukcją i otwiera region

Samotna para nawiasów klamrowych w ciele funkcji jest instrukcją. Otwiera region o etykiecie `Block` w drzewie regionów. Kilka warstw nawiasów, które nie zawierają nic poza jednym wewnętrznym blokiem, analiza własności skleja w jeden region.

Listing 3.4. Trzy pary nawiasów, a w drzewie regionów jeden blok.

```

fun main() {
    val n = 1
    {
        {
            {
                val m = n
            }
        }
    }
}

```

Sprawdzenie przechodzi. Wypis drzewa zawiera węzeł Block (compacted 2 braces). Zewnętrzna para nawiasów zostaje regionem. Dwie kolejne, które tylko owijają wewnętrzną treść, znikają z drzewa i zostają licznikiem. To nie jest ozdoba wypisu. Generator kodu i analiza własności muszą zdejmować takie opakowania w ten sam sposób. W przeciwnym razie drzewo regionów rozminie się z drzewem programu używanym przy tłumaczeniu na kod. Funkcja `peel_blocks` występuje w dwóch miejscach, w analizie własności i w reprezentacji pośredniej, a komentarz w kodzie każe trzymać je w zgodzie.

Warunek, pętla, funkcja i funkcja dopisana na końcu wywołania nie sklejają się z otoczeniem. Każde z nich ma w drzewie własną etykietę.

Wyrażenie zapisane jako instrukcja

Każde wyrażenie może stać samodzielnie jako instrukcja. Jego wartość jest wtedy porzucana. Warunek bez drugiej gałęzi jest typowym przykładem. Służy do sterowania, a gdy nikt nie oczekuje od niego wartości, jego typem jest typ pusty. Typ pusty nazywa się `unit`. Funkcja, która nie deklaruje typu wyniku, też ma wynik `unit`.

Listing 3.5. Warunek, który tylko zmienia zmienną.

```

fun main(): i32 {
    var n = 0
    if (true) {
        n = 1
    }
    return n
}

```

Program został zbudowany i kończy się kodem 1. Składnia warunku wymaga nawiasów okrągłych wokół wyrażenia logicznego i wymaga bloków w gałęziach. Nie napiszesz `if n > 0 n else 0`. Gdy warunek ma dać wartość, obie gałęzie są blokami i potrzebna jest gałąź `else`. Szczegóły typów są w rozdziale 6.

Łamanie wiersza wewnątrz nawiasów

Wywołanie i każda para nawiasów okrągłych może zająć wiele wierszy. Zanim parser zobaczy tekst, funkcja `normalize_parenthesized_newlines` w pliku `src/layout.rs` zamienia znak nowej linii na znak powrotu karetki wewnątrz nawiasów, na tej samej

głębokości nawiasów klamrowych, na której nawias został otwarty. Pomija przy tym napisy i komentarze. Lekser traktuje powrót karetki jak zwykły odstęp, więc nowa linia w środku listy argumentów nie kończy instrukcji.

Listing 3.6. Argumenty funkcji w osobnych wierszach.

```
fun add(a: i32, b: i32): i32 {  
    return a + b  
}  
  
fun main(): i32 {  
    return add(  
        1,  
        2  
    )  
}
```

Program został zbudowany. Wynik wynosi 3, więc kod wyjścia też wynosi 3. Zamiana znaku nowej linii na powrót karetki nie zmienia długości pliku w bajtach, bo oba znaki zajmują jeden bajt. Pozycje błędów pozostają tam, gdzie były. Testy w `tests/parser.rs` sprawdzają wielowierszowe wywołanie i stabilność miejsca błędu.

Poza nawiasami okrągłymi nowa linia jest znacząca. Nie ma reguły, która ciągnęłaby wyrażenie do następnego wiersza tylko dlatego, że wiersz skończył się operatorem.

Czego w składni nie ma

Sprawdziłem trzy zapisy, które wyglądają naturalnie, jeśli przychodzisz z innego języka. Wszystkie kończą się błędem fazy parse i kodem wyjścia jeden.

Słowo `struct` na początku pliku daje komunikat, że oczekiwano nowej linii albo słowa `fun`. Tak samo zachowuje się słowo `module`. Zapis `return -1` daje komunikat o nieoczekiwanym tokenie minusa. Minus jest tylko operatorem między dwiema wartościami. Liczbę ujemną da się uzyskać odejmowaniem, na przykład `0 - 1`. Nie ma literału ujemnego. Funkcji nie zagnieżdża się wewnątrz innych funkcji. Funkcje stoją wyłącznie na poziomie programu.

Podsumowanie

- Plik źródłowy jest listą funkcji. Komentarz to tylko `//` do końca wiersza.
- Instrukcję kończy nowa linia. Średnik jest błędem składni.
- Nazwa wprowadzona przez `val` nie przyjmuje późniejszego przypisania. Nazwa wprowadzona przez `var` przyjmuje. Nazwa w bloku może przesłonić nazwę zewnętrzną i w swoim inicjalizatorze może ją jeszcze odczytać.
- Kilka par nawiasów, które tylko owijają jeden blok, składa się na jeden region.
- Wewnątrz nawiasów okrągłych nowa linia jest odstępem. Poza nimi rozdziela instrukcje.
- Nie ma struktur, modułów ani jednoargumentowego minusa.

Rozdział 4. Typy danych i ich znaczenie

Ten rozdział obejmuje

- jakie typy wolno napisać i które nazwy są tylko skrótami
- które wartości kompilator kopiuje, a których nie
- jak literał dostaje typ
- jak zapisuje się brak wartości i typ funkcji
- jakie komunikaty pojawiają się, gdy typy do siebie nie pasują

Nazwy typów

Typ zapisuje się po dwukropku, przy parametrze, przy wyniku funkcji albo przy deklaracji nazwy. Część nazw jest skrótem. Kompilator zamienia skrót na typ właściwy już przy budowie drzewa składni, w funkcji `Type::from_ident`. W dalszych fazach nie ma osobnego typu o nazwie `Int`. Jest trzydziestodwubitowa liczba całkowita ze znakiem.

Zapis w programie	Typ po zamianie
i8, i16, i32, i64	liczba całkowita ze znakiem o podanej szerokości
u8, u16, u32, u64	liczba całkowita bez znaku o podanej szerokości
Int	i32
Long	i64
Byte	u8
Float	f32
Double	f64
f32, f64	liczba zmiennoprzecinkowa
bool	wartość logiczna, literały true i false
unit	typ pusty
String	napis
[T; N]	tablica o długości N
T?	wartość, która może być pusta
(A, B) -> R	typ funkcji

Listing 4.1. Skróty typów są przyjmowane i dają się zbudować, dopóki wartości zmiennoprzecinkowych tylko przechowujesz.

```
fun main(): i32 {  
    val n: Int = 1  
    val wide: Long = 2  
    val byte: Byte = 3  
    val f: Float = 1.5  
    val d: Double = 2.25  
    val flag: bool = true  
    return n  
}
```

Program został zbudowany i kończy się kodem 1, bo zwracane jest n. Samo zapisanie liczb zmiennoprzecinkowych w lokalnych nazwach generator kodu obsługuje. Użycie ich w dodawaniu albo w porównaniu jest osobną, niedokończoną ścieżką. Ten listing ich nie używa, dlatego plik wykonywalny powstaje. Rozdział 17 opisuje, co dzieje się przy porównaniu.

Każda inna nazwa typu, na przykład Point albo Unit zapisane wielką literą, daje błąd unknown named type. Słowo unit małymi literami jest typem pustym. Wielka litera Unit nie jest skrótem.

Które wartości są kopiowane

Wartość jest kopiowana tylko wtedy, gdy jest typem prostym i nie jest pusta. Poza kopiowaniem zostają napis, każda tablica, każdy typ funkcji oraz cokolwiek z pytańnikiem, także i32?. Ta sama odpowiedź obowiązuje przy sprawdzaniu typów i przy analizie własności. Typ, którego nie udało się ustalić, też nie jest traktowany jako kopiowany. Dlatego błąd typu potrafi pociągnąć dodatkowy błąd własności. Drugiego komunikatu nie należy ignorować tylko dlatego, że pierwszy już coś zgłosił.

Literały

Literał całkowity jest w drzewie składni liczbą ze znakiem o szerokości 64 bitów. Typ, który dostanie w programie, zależy od kontekstu. Jeśli kontekst jest typem całkowitym, literał przyjmuje ten typ. Bez kontekstu staje się i32.

Listing 4.2. Literał dopasowany do i64. Sprawdzenie przechodzi. Budowanie odrzuca wynik funkcji main.

```
fun main(): i64 {  
    val n: i64 = 1  
    return n  
}
```

Komunikat budowania brzmi main returning i64 is not supported by codegen yet. Sprawdzenie typów jest zadowolone. Ograniczenie dotyczy tylko tego, co wolno zwrócić z main, a nie samego typu i64 w funkcji pomocniczej. Funkcja, która odejmuje

dwie wartości i64 i porównuje wynik, została zbudowana. Proces kończy się kodem 7. Przykład jest w zestawie programów do rozdziału 17.

Litera 1.5 bez adnotacji staje się f64. Z adnotacją Float albo f32 zostaje liczbą o pojedynczej precyzji. Nie ma przyrostka w rodzaju 1.5f32.

Litera napisu używa cudzysłowu. Sekwencje `\n`, `\r`, `\t`, `\\` i `\"` są zamieniane na znak nowej linii, powrót karetki, tabulację, ukośnik i cudzysłów.

Listing 4.3. Napis ze znakami specjalnymi.

```
fun main() {  
    println("a\nb\t\"c\"")  
}
```

Po uruchomieniu na wyjściu jest litera a, nowy wiersz, litera b, tabulacja, cudzysłów, litera c, cudzysłów i jeszcze jeden nowy wiersz.

Operator + nie skleja napisów. Wyrażenie "a" + "b" daje błąd fazy type: operandy arytmetyczne muszą mieć ten sam typ liczbowy, a kompilator widzi dwa napisy. Do sklejania służy funkcja wbudowana concat. Jej zasady pamięci są w rozdziale 9.

Działania na liczbach i porównania

Operatory +, -, * i / wymagają dwóch operandów tego samego typu liczbowego. Nie ma automatycznego rozszerzenia i32 do i64.

Listing 4.4. Dodawanie i64 i i32 jest błędem, i to błędem podwójnym.

```
fun main(): i32 {  
    val a: i64 = 1  
    val b: i32 = 2  
    return a + b  
}
```

Pierwszy komunikat mówi, że operandy mają typy i64 i i32. Drugi, na tym samym miejscu, mówi, że zwracana wartość ma typ i64, a oczekiwano i32. Drugi komunikat jest skutkiem pierwszego. Przy niezgodnych operandach sprawdzanie typów zostawia typ lewej strony jako typ całego wyrażenia, a ten typ nie pasuje do wyniku funkcji.

Porównania >, <, >= i <= wymagają tego samego niepustego typu liczbowego. Dwie wartości bool dają komunikat, że uporządkowane porównanie wymaga typu liczbowego. Operatory == i != wymagają identycznego typu, ale typ może dopuszczać brak wartości. Zapis `val n: i32? = None` oraz porównanie `n == None` przechodzi sprawdzenie. Budowanie odrzuca None.

Operatory &&, || i ! działają na wartościach logicznych. Koniunkcja i alternatywa nie liczą prawej strony, gdy lewa już rozstrzyga wynik. Widać to w kodzie maszynowym i w teście `builds_logical_short_circuit`. Listing w rozdziale 6 uruchamia taki program.

Dzielenie całkowite przez zero przerywa program. Zbudowałem `return 1 / 0`. Budowanie kończy się sukcesem, a uruchomiony proces dostaje sygnał przerwania i w powłoce widać kod 134. Na standardowym wyjściu błędów nie ma komunikatu Borka. Wołana jest funkcja `abort` z biblioteki języka C. W kodzie generatora jest także strażnik

przed dzieleniem najmniejszej liczby typu ze znakiem przez minus jeden. Literału minus jeden nie da się napisać, więc tej drugiej ścieżki nie uruchamiałem z poziomu programu w Borku. Strażnik w funkcji `guard_int_div` mimo to w kodzie jest.

NOTA

NOTA. Liczby zmiennoprzecinkowe wolno dodawać i porównywać na etapie sprawdzania typów. Przy budowaniu dodawanie `f32` daje komunikat, że działanie zmiennoprzecinkowe nie jest jeszcze obsługiwane. Porównanie `f64` na ścieżce, którą uruchomiłem, nie daje komunikatu. Proces kompilatora przerywa się awaryjnie, bo wartość w reprezentacji LLVM jest liczbą zmiennoprzecinkową, a fragment kodu oczekuje liczby całkowitej. Traktuj arytmetykę zmiennoprzecinkową jako sprawdzaną, nie jako tłumaczoną na program.

Brak wartości

Zapis `T?` oznacza, że wartość typu `T` może nie istnieć. Dotyczy to typów prostych, napisu i typu funkcji. Tablica z pytajnikiem daje się zapisać w składni i jest odrzucana komunikatem `nullable array types [T]? are not supported`.

Wartość obecną buduje `Some(x)`, a brak wartości zapisuje `None`. Samo `None`, bez oczekiwanego typu, nie przechodzi. Komunikat brzmi `cannot infer type of None`.

Operator `?:` wymaga, żeby lewa strona mogła być pusta. Prawa strona musi pasować do typu po zdjęciu pytajnika. Zapis `1 ? : 0` przy `n` typu `i32` daje `left operand of ? : must be nullable, got i32`. Operator `!!` wymaga wartości, która może być pusta, i daje typ bez pytajnika. Na zwykłym `i32` dostaniesz `operand of !! must be nullable, got i32`.

Listing 4.5. Napis, który może nie istnieć, oraz długość odczytana tylko wtedy, gdy napis jest. Sprawdzenie przechodzi. Budowanie odrzuca każdy `?:`, `Some` i `None`.

```
fun processUser(name: String?, score: i32): i32 {
    val fallbackName: String = name ? : "Guest"
    val verifiedUser: String? = Some(fallbackName)
    val emptyMiddle: String? = None
    val verifiedLength: i32 = verifiedUser?.length ? : 0
    if (verifiedLength > 0) {
        return score
    }
    return 0
}
```

Zapis `?.` czyta pole tylko wtedy, gdy wartość po lewej nie jest pusta. Na typie, który pusty być nie może, sprawdzanie typów każe użyć zwykłej kropki. Jedyne pole napisu i tablicy nazywa się `length` i ma typ `i32`. Dla tablicy ta długość jest liczbą wpisaną w typ, a nie osobnym licznikiem trzymanym obok danych. Próba odczytu pola `foo` z napisu daje `unknown field foo on type String`.

OSTRZEŻENIE

OSTRZEŻENIE. Budowanie listingu 4.5 wypisuje osobny błąd fazy codegen dla każdego `?:`, `Some` i `None`. Nie ma częściowej obsługi braku wartości w gotowym programie. Kontrola przed generowaniem kodu odcina te konstrukcje, zanim powstanie moduł LLVM.

Typ funkcji też może dopuszczać brak wartości. Zapis `val f: ((i32) -> i32)? = None` przechodzi sprawdzenie. Wywołanie takiej nazwy bez `!!` nie jest tym, co generator kodu umie przetłumaczyć na instrukcje. Po `!!` kontrola przed generowaniem kodu i tak odrzuca asercję.

Typ funkcji i nawiasy

Typ funkcji zapisuje się w nawiasach, na przykład `(i32, i32) -> i32`. Wariant, który może być pusty, wymaga dodatkowych nawiasów: `((i32) -> i32)?`. Gramatyka odrzuca pytajnik przy nawiasach, które nie są typem funkcji. Komunikaty brzmią `nullable parentheses require one function type` oraz `only function types may be parenthesized for nullability`.

Nie ma krotek jako wartości. Nawias przy typie albo grupuje typ funkcji, albo w wyrażeniu grupuje działanie. Nie napiszesz `(1, 2)` jako pary liczb.

Typ nieznany

Gdy wyrażenie jest już błędne, sprawdzanie typów podstawia typ nieznany. Kolejne niezgodności z tym typem są pomijane, żeby jeden błąd nie produkował długiej kaskady. Analiza własności tego pominięcia nie dziedziczy. Typ, którego nie da się przenieść do jej środowiska, zachowuje się jak typ niekopiowany. Własność nadal może zgłosić błąd.

Podsumowanie

- Skróty `Int`, `Long`, `Byte`, `Float` i `Double` znikają już przy czytaniu programu. Dalej kompilator widzi typy `i32`, `i64`, `u8`, `f32` i `f64`.
- Kopiowane są niepuste typy proste. Napis, tablica, typ funkcji i każda wartość z pytajnikiem kopiowane nie są.
- Nie ma automatycznego rozszerzania liczb i nie ma operatora `+` dla napisów.
- `None` potrzebuje oczekiwanego typu. Operatory `?:` i `!!` wymagają wartości, która może być pusta.
- Jedyne pole napisu i tablicy nazywa się `length`.
- Brak wartości i liczby zmiennoprzecinkowe są sprawdzane. Generator kodu albo je odrzuca komunikatem, albo przy porównaniu `f64` przerywa się awaryjnie.

Rozdział 5. Funkcje, parametry i wywołania

Ten rozdział obejmuje

- jak zadeklarować funkcję, parametr i typ wyniku
- kiedy przy argumentie trzeba napisać `move`
- co robią trzy funkcje wbudowane
- jak dopisać funkcję na końcu wywołania
- dlaczego napis przekazany do własnej funkcji wywraca budowanie

Kształt funkcji

Funkcję zapisuje się jako `fun nazwa(parametry): TypWyniku`, a potem blok ciała. Pominięty typ wyniku oznacza typ pusty `unit`, a nie nazwę `Unit`. Parametr ma jedną z trzech postaci: `nazwa: Typ`, `val nazwa: Typ` albo `var nazwa: Typ`. Postać bez słowa jest stała, tak jak `val`.

Listing 5.1. Dwa rodzaje parametrów. Sprawdzenie przechodzi. W reprezentacji pośredniej pierwszy parametr jest stały, drugi zmienny, a wynik ma typ `i32`.

```
fun add(val a: Int, var b: Int): Int {  
    return a  
}
```

Słowo `var` przy parametrze znaczy, że wywołujący, który przekazuje wartość niekopiowaną, musi ją oddać słowem `move`. Dla `i32` różnica nie boli, bo liczba jest kopiowana. Dla napisu różnica jest sprawdzana.

Ciało funkcji jest blokiem. Ostatnie wyrażenie w bloku nie staje się samo wynikiem funkcji. Wynik przekazuje się słowem `return`. Jeśli funkcja deklaruje wynik inny niż `unit`, każda ścieżka musi wykonać `return`.

Listing 5.2. Funkcja z zadeklarowanym wynikiem, która do niego nie wraca.

```
fun f(): i32 {  
    val n = 1  
}  
  
fun main(): i32 {
```

```

    return f()
}

```

Komunikat nie ma numeru linii, bo błąd dotyczy całej funkcji:

```

missing_ret.bork: error: type: function must return a value of type i32 on
↳ all paths

```

Samotne return bez wartości ma typ unit. W funkcji o wyniku i32 dostaniesz informację, że pusty powrót ma typ unit, a oczekiwano i32. return z wartością sprawdza zgodność z zadeklarowanym wynikiem.

Nie ma przeciążania. Nazwy funkcji są trzymane w mapie po samym identyfikatorze. Komentarz w src/sema/env.rs mówi wprost, że to jest uproszczenie obecnej wersji i że lokalne przesłonięcie wywoływanej funkcji nie jest obsługiwane. Wywołanie funkcji zdefiniowanej niżej w pliku działa, bo sygnatury są zbierane z całego programu, zanim sprawdzane są ciała.

Własność na granicy wywołania

Dla wartości, która nie jest kopiowana, analiza własności stosuje prostą tabelę. Stałą nazwę wolno przekazać do parametru stałego. Przekazanie jej do parametru zmiennego wymaga move. Zmienną nazwę trzeba przenieść zarówno do parametru stałego, jak i do zmiennego. Literał, wywołanie concat i inne wyrażenie, które dopiero tworzy wartość, nie wymaga move.

Listing 5.3. Literał napisu przekazany do parametru zmiennego. Sprawdzenie przechodzi. Budowanie przerywa kompilator.

```

fun f(var a: String) {
    println(a)
}

fun main() {
    f("hello")
}

```

Brak słowa move jest zgodny z regułą sprawdzania. Świeży literał nie jest nazwą, którą trzeba przenieść. Generator kodu tej reguły nie dotrzymuje. Proces bork build kończy się kodem 101. Ślad wskazuje src/codegen/llvm/expr.rs, funkcję value_as_int, i mówi, że znaleziono strukturę, a oczekiwano liczby całkowitej. Ta sama awaria występuje przy f(move s) oraz przy przekazaniu stałego napisu do parametru funkcji użytkownika. println i concat działają, bo mają własne fragmenty generatora, a nie ogólną ścieżkę argumentu. Dopóki ta dziura istnieje, funkcja, która ma przejść przez bork build, powinna przyjmować i zwracać liczby. Napisy zostawiaj w main i przekazuj je do print, println albo concat.

Zwrócenie napisu, który już żyje na poziomie funkcji, sprawdzenie i budowanie akceptują.

Listing 5.4. Zwrot lokalnego napisu i jego wypisanie. Program został uruchomiony. Na wyjściu jest hi oraz nowy wiersz.

```
fun shout(): String {
    var s = "hi"
    return s
}
```

```
fun main() {
    println(shout())
}
```

Zapis `return "hi"` też działa i daje ten sam wydruk. Zapis `return name` dla parametru przechodzi sprawdzenie. Wywołanie `println(greet("Ada"))`, w którym `greet` przyjmuje napis, znowu przerywa kompilator na argumentach, nie na instrukcji `return`.

Zapis `return concat("a", "b")` jest odrzucany przy sprawdzaniu. Komunikat nie ma numeru linii:

```
err_ret_concat.bork: error: ownership: returning the result of `concat` is
↳ not supported yet: returned `String` bytes must outlive the callee arena
```

Podobny komunikat dotyczy `return move`. Tekst radzi użyć `return` ze zwykłą nazwą parametru albo nazwy lokalnej, albo zwrócić literal.

Funkcje wbudowane

Trzy nazwy nie są funkcjami, które wolno zdefiniować. Próba deklaracji jest błędem typu, zanim ciało w ogóle ma znaczenie. Dla `fun println(n: i32)` komunikat brzmi `cannot redefine builtin function println`.

`print` wypisuje jeden argument i opróżnia bufor wyjścia. `println` robi to samo i dodaje nowy wiersz. Argumentem może być `i32`, `i64` albo napis, chociaż w wewnętrznej tabeli sygnatura nominalna mówi o jednym `i32`. To jest specjalny przypadek, nie ogólna zasada, że każdy parametr `i32` przyjmie napis. `concat` przyjmuje dwa napisy i zwraca jeden nowy. Bufor wyniku powstaje w arenie miejsca, do którego wynik jest zapisywany. Gdy takiego miejsca nie ma, powstaje w arenie bieżącego bloku.

Listing 5.5. Kolejność wypisywania. Program został uruchomiony. Na wyjściu jest dokładnie cyfra 1, nowy wiersz, słowo `tail` i cyfra 7, bez końcowego nowego wiersza.

```
fun main() {
    println(1)
    print("tail")
    print(7)
}
```

Funkcja dopisana na końcu wywołania

Ostatni argument może być blokiem stojącym po nawiasie wywołania. Parametry tego bloku zapisuje się przed strzałką `->`.

Listing 5.6. Ostatni parametr ma typ funkcji. Sprawdzenie przechodzi. Budowanie odrzuca konstrukcję.

```

fun action(a: Int, b: Int, block: (Int, Int) -> Int): Int {
    return block(a, b)
}

fun main(): i32 {
    val threshold = 5
    return action(1, 2) { acc, current ->
        if (current > threshold) { acc + current } else { acc }
    }
}

```

Sprawdzanie typów pilnuje trzech rzeczy. Ostatni parametr wywoływanej funkcji musi być typem funkcji. Liczba parametrów bloku musi się zgadzać. Typ wyniku bloku musi pasować do typu wyniku tego parametru. Komunikaty mówią o tym wprost, po angielsku, słowami *trailing closure*. Wywołanie, którego celem nie jest nazwa funkcji, dostaje komunikat *only named functions can be called*. Generator kodu i tak odrzuca wywołanie pośrednie osobnym tekstem.

Analiza własności nie dostaje typów parametrów takiego bloku. Wpisuje im typ nieznany. Skutek widać w drzewie regionów. Parametry bloku są lokalne, a ich odczyt w gałęzi warunku bywa oznaczony jako współdzielenie, mimo że `Int` jest kopiowany. Program mimo to przechodzi sprawdzenie. Drzewo jest w tym miejscu mylące. Nie jest to błąd twojego programu.

Bloku funkcji nie zapiszesz w stałej poza wywołaniem. Gramatyka ma taką funkcję tylko jako argument końcowy albo jako blok *move*. Typ funkcji istnieje, ale nie ma osobnego literału funkcji.

Trzy formy przeniesienia przy bloku

Zwykły blok po wywołaniu niczego nie przenosi. Odczyt zewnętrznej zmiennej napisowej jest wtedy błędem. Zapis `move ()` przed blokiem też niczego nie przenosi, ale robi to jawnie. Zapis `move (a, b)` przenosi dokładnie wymienione nazwy. Zapis `move` bez nawiasu przenosi każdą żywą, niekopiowaną nazwę z regionu zewnętrznego, której blok używa.

To samo da się napisać jako instrukcję, bez wywołania. Rozdział 8 ma zbudowane programy dla tych instrukcji. Formy stojące po wywołaniu odpadają przy budowaniu komunikatem *trailing closures are not supported by codegen*.

Wnioskowanie, które nazwy przenieść, nie patrzy na wyrażenia `move nazwa` i `promote nazwa` jak na zwykłe użycie nazwy. W przeciwnym razie przeniesienie zapisane w wyrażeniu byłoby liczone drugi raz jako przeniesienie całego bloku.

Co z tego wynika dla programów, które mają się zbudować

Ciało wywołanej funkcji ma własne regiony. Argumenty są liczone w regionie wywołującego. Wynik liczbowy wraca w rejestrze. Wynik napisowy nie dostaje dziś bufora należącego do wywołującego, dlatego analiza ucieczki zabrania zwrócić świeże

bajty. Listing 5.4 działa, bo napis "hi" jest literałem albo wartością na poziomie funkcji, a `println` tylko czyta adres i długość.

WSKAZÓWKA

WSKAZÓWKA. W programie, który ma przejść przez bork `build`, trzymaj funkcje przy liczbach. Napisy wypisuj z `main`. To nie jest zalecenie na zawsze. To jest obejście błędu w funkcji `coerce_value_to_ty`, która wartość inną niż `bool` próbuje potraktować jako liczbę całkowitą. Liczba zmiennoprzecinkowa na części ścieżek dostaje komunikat, że nie jest obsługiwana. Napis dostaje awarię.

Podsumowanie

- Wynik funkcji zapisuje się słowem `return`. Brak powrotu na którejś ścieżce jest błędem typu.
- Parametr bez `val` i bez `var` jest stały. Parametr zmienny wymaga `move`, gdy przekazuje się istniejącą nazwę wartości niekopiowanej.
- Świeży literał nie wymaga `move`, ale napis jako argument funkcji użytkownika wywraca budowanie.
- `print`, `println` i `concat` są wbudowane. Nie wolno ich zadeklarować ponownie.
- Funkcja dopisana na końcu wywołania jest sprawdzana i nie jest tłumaczona na kod maszynowy.
- Analiza własności nie zna typów parametrów takiej funkcji, więc drzewo regionów potrafi pokazać współdzielenie tam, gdzie typ jest kopiowany.

Rozdział 6. Warunki, pętle i powrót z funkcji

Ten rozdział obejmuje

- kiedy warunek daje wartość, a kiedy tylko steruje
- jak działa pętla for po zakresie liczb
- jak działają while, break i continue
- dlaczego && i || nie zawsze liczą prawą stronę
- czego nie wolno zwrócić z wewnętrznego bloku

Warunek

Warunek zapisuje się jako `if (wyrażenie) { blok },` opcjonalnie z `else { blok }.` Wyrażenie w nawiasie musi mieć typ `bool`. Obie gałęzie, jeśli druga istnieje, są blokami. Typ wyniku zależy od tego, czy ktoś oczekuje wartości, i od tego, czy jest `else`. Reguły są w funkcji `check_if` w pliku `src/typeck/expr/control.rs`.

Gdy nie ma `else` i nikt nie oczekuje typu, wynik ma typ `unit`. Gdy nie ma `else`, a oczekiwany typ jest, na przykład przez adnotację `val x: i32`, kompilator zgłasza `value-producing if expression requires an else branch`. Gdy `else` jest i obie gałęzie mają ten sam typ, wynik ma ten typ. Gdy typy gałęzi się różnią, a wynik jest potrzebny, dostaniesz `else branch has type ..., expected ...`. Gdy `else` jest, ale całe wyrażenie stoi w miejscu, które nie prosi o wartość, wynik i tak jest `unit`. Wartości gałęzi są sprawdzane, ale nie wypływają na zewnątrz.

Listing 6.1. Warunek użyty jako wartość. Program został zbudowany i kończy się kodem 42.

```
fun pick(flag: i32): i32 {
    val v = if (flag == 1) { 40 } else { 7 }
    return v + 2
}

fun main(): i32 {
    return pick(1)
}
```

Nazwa `v` nie ma adnotacji. Obie gałęzie mają typ `i32`, więc cały warunek też ma typ `i32`. W drzewie regionów gałęzie są osobnymi węzłami `IfThen` oraz `IfElse`.

Listing 6.2. Brak else przy oczekiwaniu liczby.

```
fun main(): i32 {  
    val x: i32 = if (1 > 0) { 1 }  
    return x  
}
```

Komunikat nie wskazuje numeru linii:

```
if_ann.bork: error: type: value-producing if expression requires an else  
  ↳ branch
```

Bez adnotacji, przy `val x = if (n > 0) { n }`, błąd jest inny. Warunek staje się typem `unit`, bo nikt nie oczekiwał konkretnego typu, a dopiero `return x` mówi, że zwracana wartość ma typ `unit`, podczas gdy oczekiwano `i32`. Komunikat o brakującym `else` pojawia się wtedy, gdy oczekiwany typ jest znany.

Gałąź warunku jest regionem. Napis zbudowany tylko w jednej gałęzi nie może być wartością całego warunku, bo arena gałęzi ginie wraz z jej końcem. Analiza ucieczki mówi wtedy, że napis przeniesiony wewnątrz gałęzi nie może być wartością warunku, bo arena gałęzi jest zwalniana. Tekst komunikatu wymienia napis, choć warunek w kodzie patrzy szerzej, na każdą wartość trzymaną w arenie, a więc także na tablicę.

Analiza własności scala stan przeniesienia po warunku. Nazwa jest przeniesiona po całym `if` wtedy, gdy była przeniesiona już wcześniej albo gdy przeniosły ją obie gałęzie. Samo przeniesienie w gałęzi `then`, bez `else`, nie zostawia nazwy martwej po warunku. Inaczej gałąź, która może się nie wykonać, zabijałaby zmienną na stałe. Test `if_then_only_move_without_else_does_not_stick` zamyka tę zasadę.

Pętla po zakresie

Pętlę po liczbach zapisuje się jako `for (nazwa in początek..koniec)`. Zakres jest otwarty z prawej strony. Liczba `koniec` nie wchodzi do obiegu. Granice mają typ `i32` i są liczone raz. Nazwa pętli ma w sprawdzaniu typów typ `i32`.

Listing 6.3. Suma liczb od zera do czterech włącznie. Program został zbudowany i kończy się kodem 10.

```
fun main(): i32 {  
    var total = 0  
    for (i in 0..5) {  
        total = total + i  
    }  
    return total  
}
```

W drzewie regionów ciało pętli jest węzłem `ForLoop (i)`. Nazwa `i` jest lokalna. Nazwa `total` wewnątrz pętli jest oznaczona jako kopiowana, bo `i32` jest kopiowane. Przypisanie `total = ...` zmienia zmienną z regionu funkcji. Nie jest przeniesieniem. Generator kodu wchodzi do regionu ciała raz, a przy każdym obiegu czyści arenę tego ciała, zamiast oddawać bufor do puli i brać nowy. Dzięki temu pętla nie zużywa nowego bufora na każdy obieg.

Zakres o granicach i64 jest odrzucany. Komunikat mówi, że granice zakresu muszą mieć typ i32. Pętla, której prawa strona nie jest zakresem, dostaje for-loop iterator must be a range. Nie ma pętli po elementach tablicy.

Pętla dopóki i sterowanie obiegiem

Listing 6.4. Przerwanie pętli, gdy licznik dojdzie do trzech. Program został zbudowany i kończy się kodem 3.

```
fun main(): i32 {  
    var i: i32 = 0  
    while (i < 10) {  
        if (i == 3) {  
            break  
        }  
        i = i + 1  
    }  
    return i  
}
```

Warunek while musi mieć typ bool. break i continue poza pętlą są błędem fazy type, nie osobnej fazy sterowania. Dla samotnego break w main komunikat brzmi `break` outside of a loop. Sprawdzanie typów pamięta głębokość zagnieżdżenia pętli i podnosi ją zarówno przy for, jak i przy while.

Listing 6.5. continue pomija dodanie dwójki. Suma wynosi $0 + 1 + 3 + 4$, czyli 8. Program został zbudowany i kończy się tym kodem.

```
fun main(): i32 {  
    var s = 0  
    for (i in 0..5) {  
        if (i == 2) {  
            continue  
        }  
        s = s + i  
    }  
    return s  
}
```

continue działa także w while. Nie ma klauzuli else przy pętli. Pętlę bez warunku końca zapisuje się jako while (true).

Koniunkcja i alternatywa

Listing 6.6. Prawa strona koniunkcji nie powinna zmienić zmiennej, bo lewa strona jest fałszywa. Program został zbudowany i kończy się kodem 0.

```
fun side(): i32 {  
    return 1  
}
```

```

fun main(): i32 {
    var n: i32 = 0
    if (false && side() == 1) {
        n = 1
    }
    if (true || side() == 1) {
        return n
    }
    return 99
}

```

Funkcja `side` nie wypisuje nic, więc sam wydruk nie pokazuje, że prawe wywołanie zostało pominięte. Pokazuje to wynik: `n` zostaje zerem, a druga instrukcja wraca od razu, bo `true || ...` nie musi liczyć prawej strony. W kodzie generatora `&& i ||` dostają osobne bloki. Spacer, który uzgadnia drzewo regionów z generowaniem kodu, nie wchodzi w prawy operand zwykłą ścieżką, gdy ta ścieżka jest już odłożona do osobnego bloku. Dzięki temu decyzja, czy region potrzebuje własnego bufora, pozostaje zgodna z tym, co naprawdę zostanie wyemitowane.

Operator `!` wymaga wartości `bool`. Program, który liczy `val flag = !false` i zwraca 1 albo 0 w zależności od `flag`, został zbudowany i kończy się kodem 1.

Wczesny powrót i regiony

Powrót z wnętrza bloku musi zwolnić regiony, które funkcja otworzyła. Generator kodu zdejmuje je przy `return` i nie zdejmuje ich drugi raz na ścieżce, która już wróciła. Seria poprawek w historii repozytorium dotyczyła właśnie podwójnego zwolnienia albo zostawienia uchwytu bufora. Test `builds_nested_loops_with_regions_and_early_return` zwraca 63 i pilnuje, że wczesny powrót z zagnieżdżonych pętli daje oczekiwany wynik.

Dla programisty skutek jest taki. `return` w środku bloku jest legalny dla wartości kopiowanej i dla napisu, który żyje na poziomie funkcji. Nie jest legalny dla napisu, którego bajty żyją głębiej.

Listing 6.7. Zwrot napisu utworzonego w bloku wewnętrznym. Sprawdzenie i budowanie kończą się kodem jeden. Plik wykonywalny nie powstaje.

```

fun mk(): String {
    var s = "esc"
    {
        val x = move s
        return x
    }
}

fun main() {
    println(mk())
}

```

Komunikat fazy ownership mówi, że zwracany jest napis, którego bajty żyją w regionie

wewnętrznym, i że ta arena jest zwalniana, zanim wartość zostanie zwrócona. Faza nazywa się tak samo jak faza analizy własności, bo analiza ucieczki dokłada błędy do tej samej grupy. Gdy analiza ucieczki zgłosi błąd, reprezentacja pośrednia jest wyrzucana. Drzewo regionów zostaje.

Podsumowanie

- Warunek bez else, użyty tam, gdzie nikt nie oczekuje wartości, ma typ unit.
- Gdy oczekiwany typ jest znany, brak else jest osobnym błędem.
- Przeniesienie w jednej gałęzi nie zabija nazwy po warunku, jeśli druga gałąź jej nie ruszyła.
- Pętla for idzie po zakresie liczb i32, otwartym z prawej strony, i czyści arenę ciała przy każdym obiegu.
- break i continue poza pętlą są błędem typu.
- && i || nie liczą prawej strony, gdy lewa już rozstrzyga wynik.
- Nie zwrócisz napisu z bloku wewnętrznego. Kompilator odrzuca taki program, zanim powstanie plik wykonywalny.

Rozdział 7. Tablice o znanej z góry długości

Ten rozdział obejmuje

- jak zapisać typ tablicy i literał tablicy
- czym różni się odczyt elementu od wycinka o stałych granicach
- jak czytać długość
- jak własność dotyczy całej tablicy i pojedynczego napisu w tablicy
- co dzieje się w czasie działania, gdy indeks nie mieści się w tablicy

Dlaczego długość należy do typu tablicy

Tablicę zapisuje się jako `[T; N]`. `T` jest typem elementu, a `N` jest liczbową długością. Długość nie jest wyrażeniem liczonym w czasie działania. Jest częścią typu, tak jak szerokość liczby `i32` jest częścią typu, a nie wartością trzymaną obok liczby. Tablica trzech liczb `i32` ma inny typ niż tablica dwóch liczb `i32`. Nie przypiszesz jednej do drugiej.

`N` w zapisie typu musi być literałem całkowitym. Za duża liczba, która nie mieści się w `u32`, daje błąd gramatyki `array length is too large`. Pusta tablica nie ma skąd wziąć ani typu elementu, ani długości.

Listing 7.1. Pusta tablica z jawnym typem. Program został zbudowany. Długość wynosi zero, więc kod wyjścia też wynosi zero.

```
fun main(): i32 {  
    val a: [i32; 0] = []  
    return a.length  
}
```

Bez adnotacji zapis `val a = []` daje komunikat, że pusty literał wymaga jawnego typu, i podaje przykład `val a: [i32; 0] = []`.

Elementy literału muszą mieć jeden typ, a ich liczba musi równać się `N`.

Listing 7.2. Trzy elementy w tablicy, która według typu ma dwa.

Sprawdzenie wypisuje dwa błędy fazy `type`. Pierwszy mówi, że literał ma 3 elementy, a oczekiwano 2. Drugi mówi, że inicjalizator nazwy `a` ma typ `[i32; 3]`, a oczekiwano `[i32; 2]`. Źródło to `val a: [i32; 2] = [1, 2, 3]`.

Dozwolony element to wartość kopiowana albo niepusty napis. Napis, który może być pusty, nie może być elementem. Tablica, która sama może być pusta w sensie ?, też jest odrzucana.

Tablica nie jest kopiowana, nawet gdy jej elementy są. Przypisanie całych tablic wymaga identycznego typu elementu i identycznej długości.

Indeks

Indeks jest typu `i32` i jest liczony dopiero przy uruchomieniu. Nie wchodzi do typu, więc sprawdzenie nie odrzuca `a[9]` na tablicy trzelementowej. Zbudowałem taki program. Plik wykonywalny powstaje, a uruchomienie kończy się sygnałem przerwania, z kodem powłoki 134. Generator kodu wstawia sprawdzenie zakresu i woła `abort`.

Listing 7.3. Zapis elementu i wycinek, który widzi ten sam bufor. Program został zbudowany i kończy się kodem 12.

```
fun main(): i32 {  
    var a: [i32; 3] = [1, 2, 3]  
    a[1] = 9  
    val b = a[0..2]  
    return b[1] + a.length  
}
```

Po zapisie pod indeksem 1 leży 9. Wycinek od zera do dwóch, bez dwójki, obejmuje dwa pierwsze elementy i nie kopiuje ich do nowego bufora. Dlatego `b[1]` też jest 9. Długość `a` wynosi 3. Suma wynosi 12 i tyle wynosi kod wyjścia.

Nazwa wprowadzona przez `val` nie przyjmuje zapisu elementu. Komunikat jest ten sam co przy zwykłym przypisaniu do stałej: nie można przypisać do niezmienniczej nazwy `a`. Indeks złego typu daje `array index must be i32` albo, przy zapisie, informację, że indeks ma inny typ, a oczekiwano `i32`.

Wycinek o stałych granicach

Zapis `a[początek..koniec]` wymaga, żeby obie granice były literałami typu `i32`. Typ wyniku to tablica o długości `koniec - początek`. Wycinek nie kopiuje bufora. Nowy opis tablicy wskazuje w środek starej, a długość bierze z typu.

Wycinek `[1..9]` na tablicy `[i32; 3]` jest błędem typu: `slice [1..9] is out of bounds`. Granice, które nie są literałami, dają komunikat, że granice muszą być literałami całkowitymi, żeby typ wyniku miał znaną długość. Generator kodu nie wstawia drugiego sprawdzenia wycinka w czasie działania. Ufa sprawdzeniu typów. Indeks jest sprawdzany w czasie działania, bo nie da się go wpisać do typu.

Listing 7.4. Wycinek i jego długość. Program został zbudowany. Na wyjściu są wiersze 20 oraz 2.

```
fun main() {  
    val a = [10, 20, 30]  
    val b = a[0..2]  
}
```

```

    println(b[1])
    println(b.length)
}

```

Analiza ucieczki traktuje wycinek jak tablicę, z której powstał. Wycinek nie może przeżyć areny, która trzyma bufor. Test `rejects_slice_escaping_inner_region` oczekuje fazy ownership i tekstu `inner region`.

Napis jako element

Odczyt `a[i]`, gdy element jest napisem, jest oglądaniem napisu leżącego w buforze tablicy. Nie ma przeniesienia jednego elementu. Słowa `move` i `promote` dotyczą całej tablicy albo, przy zapisie do elementu, wartości po prawej stronie.

Zapis elementu, który nie jest kopiowany, wymaga `move` albo `promote` po prawej stronie. Arena tablicy jest miejscem, w którym lądują bajty nowego elementu.

Listing 7.5. Zapis przeniesionego napisu do tablicy. Program został zbudowany. Na wyjściu jest `z` oraz nowy wiersz.

```

fun main() {
    var a: [String; 2] = ["a", "b"]
    var s = "z"
    a[0] = move s
    println(a[0])
}

```

Bez `move` sprawdzenie mówi, że własność trzeba przenieść. Po `move` nazwa `s` jest martwa. W drzewie regionów `s` jest oznaczone jako przeniesione z regionu funkcji `main`.

Jak tablica wygląda w gotowym programie

W reprezentacji LLVM tablica i napis mają ten sam kształt. Jest to para: adres bufora i długość zapisana na 64 bitach. Dla tablicy długość w tej parze równa się `N` z typu. Bajty elementów leżą w arenie, w której tablica powstała, albo w arenie zmiennej, do której tablica jest od razu zapisywana.

Pole `length` w języku ma typ `i32`. Generator kodu wycina długość z pary i obcina ją do 32 bitów. Dopóki `N` mieści się w `i32`, wynik jest zgodny z typem. Gramatyka przyjmuje większe `N`, bo długość w typie jest liczbą `u32`. Testy nie sprawdzają tablicy dłuższej niż maksymalna wartość `i32`. Nie zakładaj, że pole `length` takiej tablicy jest dobrze określone.

Podsumowanie

- Długość tablicy jest częścią jej typu. Literał musi mieć dokładnie tyle elementów, ile mówi typ.
- Pusty literał `[]` wymaga adnotacji typu.
- Indeks spoza zakresu przerywa program. Zły wycinek jest błędem kompilacji.

- Wycinek o stałych granicach wskazuje w istniejący bufor i ma typ tablicy o długości równej różnicy granic.
- Element napisowy czyta się bez kopiowania. Zapis takiego elementu wymaga move.
- Całą tablicę wolno przenieść tylko do tablicy o tym samym typie elementu i tej samej długości.

Rozdział 8. Kopiowanie, współdzielenie i przeniesienie wartości

Ten rozdział obejmuje

- jak czytać drzewo regionów
- jak przenieść wartość słowem `move` i jak słowem `promote` skopiować ją do zmiennej z regionu zewnętrznego
- czym różnią się trzy formy bloku `move`
- dlaczego przypisanie do zmiennej z zewnątrz bloku nie jest odczytem tej zmiennej
- które błędy pochodzą z nazw, a które z czasu życia bajtów

Własność nazw i czas życia bajtów mają jedną fazę w komunikacie

Własność nazw sprawdza analiza w katalogu `src/sema`. To, czy bajty przeżyją użycie, sprawdza osobne przejście, analiza ucieczki, w pliku `src/escape.rs`. Analiza ucieczki uruchamia się tylko wtedy, gdy sprawdzanie typów i analiza nazw nie zgłosiły wcześniej żadnego błędu. Oba źródła drukują fazę `ownership`. Dlatego program, który przeszedł analizę nazw, nie musi jeszcze spełniać wszystkich reguł napisów. Znaczy to tyle, że nazwy nie są użyte po przeniesieniu i że wartość niekopiowana nie ucieka przez gołą nazwę. Czas życia bajtów sprawdza dopiero analiza ucieczki.

Drzewo regionów zostaje nawet przy błędach. Reprezentacja pośrednia zostaje tylko wtedy, gdy lista błędów jest pusta, i to po analizie ucieczki. Przy błędzie składni nie ma nawet drzewa.

Znaczniki w drzewie

Wypis `--dump-arenas` opisuje każdą nazwę krótkim znacznikiem. `Local` oznacza deklarację w tym regionie albo parametr. `Copy` oznacza odczyt wartości kopiowanej z regionu otaczającego. `Shared` z nazwą regionu oznacza odczyt stałej, która nie jest kopiowana. `Moved` z nazwą regionu oznacza, że własność została zabrana z tamtego regionu.

Węzły mają etykiety `fun` `nazwa`, `Block`, `ForLoop` (`i`), `WhileLoop`, `IfThen`, `IfElse`, `MoveBlock` (`move`), `Closure` i `Closure` (`move`). Sklejone nawiasy dopisują do etykiety informację, ile par zostało scalonych.

Przeniesienie zapisane przy nazwie

Zapis `move nazwa` zużywa nazwę. Późniejsze użycie daje `use of nazwa after move from etykieta`.

Sam zapis zmiennej napisowej po prawej stronie przypisania albo w argumencie nie dopisuje `move` za programistę.

Listing 8.1. Brak `move` przy przypisaniu napisu.

```
fun main() {  
    var s: String = "hi"  
    var x = s  
}
```

Komunikat brzmi `use move s to transfer ownership`. W wywołaniu tekst jest inny: `use move s to pass ownership`. Gdy analiza widzi przeniesienie między różnymi regionami, potrafi powiedzieć, że nazwa nie jest kopiowana i trzeba ją przenieść do nazwanego regionu słowem `move`.

Listing 8.2. Przeniesienie i wypisanie. Program został zbudowany. Na wyjściu są wiersze `ab` oraz `42`.

```
fun main() {  
    var s = "ab"  
    val t = move s  
    println(t)  
    print(4)  
    println(2)  
}
```

W drzewie `s` jest przeniesione z regionu `fun main`, a `t` jest nazwą lokalną.

Słowo `promote`

`promote nazwa` jest legalne tylko po prawej stronie przypisania do zmiennej z regionu ściśle zewnętrznego. Kopiuje bajty do areny tej zmiennej i unieważnia nazwę źródłową.

Listing 8.3. Podniesienie napisu, który już powstał w bloku wewnętrznym. Program został zbudowany. Na wyjściu jest `temp` oraz nowy wiersz.

```
fun main() {  
    var outer = "a"  
    {  
        var held = "temp"  
        outer = promote held  
    }  
}
```

```
println(outer)
}
```

W drzewie nazwa `held` pojawia się w bloku dwa razy. Raz jako deklaracja lokalna, raz jako przeniesienie po `promote`.

Są trzy typowe błędy. Użycie `promote` poza przypisaniem do zmiennej zewnętrznej daje komunikat, że `promote` jest legalne tylko przy takim przypisaniu. Użycie na wartości kopiowanej daje komunikat, że `promote` nie jest potrzebne. Użycie w tym samym regionie, na przykład `val x = promote held` obok deklaracji `held`, daje komunikat, że `promote held` nie może trafić do `fun main`, bo wartość już żyje w tej arenie albo jeszcze żyje. Ostatni komunikat sprawdziłem uruchomieniem.

`promote` nie występuje przy `return`. Nie ma miejsca przeznaczenia, do którego analiza mogłaby skopiować bajty wyniku.

Przypisanie do zmiennej z zewnątrz

Zapis `outer = wartość`, gdy `outer` jest zmienną z regionu otaczającego i nie została wcześniej przeniesiona, nie jest odczytem tej zmiennej w bloku wewnętrznym. Analiza nie traktuje lewej strony jak użycia, które wymagałoby `move`. Prawa strona dostaje jako miejsce alokacji arenę zmiennej `outer`, a nie arenę bloku, w którym stoi przypisanie.

Listing 8.4. Przypisanie literału do zmiennej z zewnątrz. Program został zbudowany. Na wyjściu jest `b` oraz nowy wiersz. W bloku wewnętrznym nie ma żadnej nazwy lokalnej.

```
fun main() {
    var outer = "a"
    {
        outer = "b"
    }
    println(outer)
}
```

To jest zmiana istniejącej zmiennej. Nie potrzebujesz nazwy pośredniej. W tym przykładzie dokument `docs/language.md` i kompilator mówią to samo.

Blok, który przenosi kilka nazw naraz

Listing 8.5. Dwa bloki `move`. Program został zbudowany. Na wyjściu są wiersze `A` oraz `B`.

```
fun main() {
    var a: String = "A"
    var b: String = "B"
    move (a) {
        val x = a
        println(x)
    }
    move {
        val y = b
    }
}
```

```

        println(y)
    }
}

```

Pierwszy blok wymienia `a` w nawiasie. Wewnątrz nazwa `a` jest już lokalna. Nie pisze się przy niej drugi raz `move`. Drugi blok nie ma listy. Kompilator przenosi te niekopiowane nazwy z zewnątrz, których blok używa. Używa `b`, więc przenosi `b`. W drzewie oba bloki nazywają się `MoveBlock` (`move`). Przy każdym widać, skąd nazwa została przeniesiona.

Nazwy z listy są już w regionie bloku. Ponowne `move` takiej nazwy w tym samym regionie jest błędem. Komunikat mówi, że nazwa została już przeniesiona do tego regionu jako przechwycenie.

Zapis `move () { }` nie zgaduje listy. Nic nie jest przenoszone. Odczyt zewnętrznej zmiennej napisowej wewnątrz takiego bloku jest zwykłym błędem mówiącym, że wartość nie jest kopiowana.

Jawna lista wygrywa ze zgadywaniem. Pusta jawna lista wyłącza zgadywanie. W drzewie składni różnica jest między brakiem listy a listą pustą. Nie wolno tych dwóch zapisów utożsamić, gdy będziesz czytał parser.

Pętla jeszcze raz

Zakaz z rozdziału 2 dotyczy także bloku `move`. Nazwę utworzoną w ciele pętli wolno przenieść.

```

for (i in 1..10) {
    var a: String = "A"
    move (a) {
        val t = a
    }
}

```

Test `move_loop_local_binding_is_ok` pilnuje, że takie przeniesienie przechodzi. Każdy obieg ma świeże `a`. Ta sama nazwa utworzona przed pętlą byłaby błędem.

Stała i zmienna

Odczyt stałej niekopiowanej w regionie wewnętrznym jest współdzieleniem i jest poprawny. Pokazuje to listing 2.2. Odczyt zmiennej niekopiowanej bez `move` jest błędem. Nie ma trzeciej możliwości w rodzaju chwilowego pożyczania zmiennej. Albo przenosisz własność, albo trzymasz wartość w stałej.

Ta sama zasada dotyczy wartości schowanej w `Some` i argumentów zagnieżdżonych w większym wyrażeniu. Zmienna napisowa użyta w środku konstruktora też chce `move`. Literał nie chce.

Podsumowanie

- Analiza nazw pilnuje nazw. Analiza ucieczki pilnuje bajtów. Obie wypisują fazę ownership.
- move nazwa zużywa nazwę. Zwykle użycie zmiennej niekopiowanej w przypisaniu i w wywołaniu jest błędem, a komunikat podpowiada move.
- promote kopiuje bajty do zmiennej z regionu zewnętrznego i też zużywa nazwę źródłową.
- Lewa strona przypisania do zmiennej z zewnątrz nie jest odczytem tej zmiennej.
- move (a, b), move () i move bez listy to trzy różne polecenia. Pusta lista nie jest tym samym co brak listy.
- Zgadywanie listy nie rusza wartości kopiowanych i nie rusza nazw, których blok nie wspomina.

Rozdział 9. Jak Bork przechowuje napisy w pamięci

Ten rozdział obejmuje

- czym w gotowym programie jest wartość napisu
- gdzie powstają znaki, gdy zapisujesz wynik do zmiennej
- kiedy kompilator buduje napis od razu w arenie zmiennej docelowej
- jak czytać przykłady z dokumentacji, które kompilator odrzuca
- czego jeszcze nie da się zwrócić z funkcji

Adres i długość, a osobno znaki

W kodzie maszynowym wartość typu String jest parą. Pierwszy element to adres znaków, drugi to długość zapisana na 64 bitach. Same znaki leżą albo w stałej programu, albo w buforze areny. Koniec regionu unieważnia bufor, a nie parę leżącą na stosie. Dlatego kompilator musi odrzucić parę, której adres wskazywałby w już zwolniony bufor, zanim generator kodu ją wyemituje. Robi to analiza ucieczki, funkcja `place` w `src/escape.rs`.

Głębokość zero oznacza region funkcji. Każdy region zagnieżdżony, który analiza ucieczki naprawdę otwiera, zwiększa głębokość. Funkcja `place` odpowiada na pytanie, w jakiej głębokości leżą znaki danego wyrażenia.

Znaki powstają tam, gdzie stoisz

Bez miejsca przeznaczenia literał i wynik concat biorą bieżącą arenę. Wyjście z bloku czyści tę arenę. Nazwa zadeklarowana w bloku i tak nie jest widoczna na zewnątrz. Problem zaczyna się wtedy, gdy parę adresu i długości wynosisz przypisaniem albo instrukcją `return`.

Zapis do zmiennej wybiera jej arenę

Gdy prawa strona jest zapisywana do zmiennej, znaki wyniku trafiają do areny tej zmiennej. Nie dotyczy to każdego podwyrażenia. Argumenty wywołania są liczone tak, jakby miejsca przeznaczenia chwilowo nie było, więc wartości tymczasowe zostają w bloku.

Listing 9.1. Literał zapisany wprost do zmiennej z zewnątrz. Program został zbudowany. Na wyjściu jest b oraz nowy wiersz.

```
fun main() {  
    var outer = "a"  
    {  
        outer = "b"  
    }  
    println(outer)  
}
```

Nie potrzebujesz zmiennej pośredniej. Zapis `outer = concat(lewy, prawy)` kładzie wynik w arenie zmiennej `outer`. Napisy `lewy` i `prawy` są czytane tam, gdzie już żyją.

Dwie sąsiednie instrukcje mogą uniknąć drugiej kopii

Gdy w bloku jest najpierw `var piece = literał` albo wywołanie, a zaraz potem `outer = move piece`, i nazwa `outer` jest już widoczna, kompilator ustawia przy deklaracji informację, że bajty mają powstać w arenie `outer`. W reprezentacji pośredniej to pole nazywa się `alloc_in_binding`. Druga instrukcja nie kopiuje wtedy znaków drugi raz.

Rozpoznanie jest celowo wąskie. Leży w `src/hoist.rs` i obejmuje tylko następną instrukcję, bez niczego pomiędzy. Inicjalizator musi być literałem napisu albo wywołaniem, nie tablicą i nie dowolnym wyrażeniem. Prawa strona przypisania musi być przeniesieniem tej samej nazwy. Cel musi być nazwą, nie elementem tablicy. Nazwa celu musi być parametrem albo zmienną zadeklarowaną wcześniej.

Pętla, warunek, funkcja dopisana na końcu wywołania i jakakolwiek instrukcja między tymi dwiema wyłącza to rozpoznanie. Zostaje kopia przy `move` albo jawnie `promote`.

NOTA

NOTA. Kompilator nie dowodzi, że nazwa pośrednia istnieje tylko po to, żeby nakarmić zmienną zewnętrzną. Patrzy na dwie sąsiednie instrukcje. Gdy je rozdzielisz, płacisz kopię.

Promote, gdy znaki już powstały za głęboko

Listing 8.3 pokazuje `promote`. Nazwa `held` powstała w arenie bloku. `promote` kopiuje znaki do areny zmiennej zewnętrznej i oznacza źródło jako przeniesione. Nie pomija kopii. Kopii unikasz wtedy, gdy wynik od razu zapisujesz do zmiennej zewnętrznej, albo gdy dwie sąsiednie instrukcje pozwolą zbudować wartość od razu w jej arenie.

Czego analiza ucieczki nie przepuści

Komunikaty z `src/escape.rs` sprawdzone na przykładach są takie. Zwrot wartości, której znaki żyją głębiej niż region funkcji, mówi, że zwracany napis żyje w regionie wewnętrznym i że arena zostanie zwolniona przed powrotem. Zwrot wyniku `concat`

mówi, że nie jest to jeszcze obsługiwane, bo znaki zwróconego napisu musiałyby przeżyć arenę wywoływanej funkcji. Zwrot napisu przeniesionego albo użytego ze słowem `promote` radzi użyć zwykłej nazwy albo literału. Przypisanie wartości, której bajty żyją głębiej niż zmienna docelowa, mówi, że taki zapis nie jest obsługiwany. Wartość warunku, która jest napisem przeniesionym tylko w gałęzi, jest odrzucana, bo arena gałęzi ginie razem z gałęzią.

Zapis `return if (c > 0) { concat("a", "b") } else { "x" }` wpada w komunikat o `concat`, nie w komunikat o gałęzi. Sprawdziłem to na osobnym pliku. Pozycja w pliku bywa pusta.

Przykład z dokumentacji, który się nie kompiluje

Plik `docs/language.md` pokazuje w bloku wywołanie `concat(left, right)`, gdy `left` i `right` są zmiennymi. Taki program nie przechodzi sprawdzenia. Obie nazwy są zmienne, więc w bloku wewnętrznym trzeba je przenieść. Komunikaty, które dostałem, mówią, że `left` nie jest kopiowane i trzeba je przenieść do bloku, i to samo o `right`.

Działają dwie poprawki. W obu wynik na wyjściu to LR oraz nowy wiersz, a nie `prefixLR`. `concat` nie dopisuje znaków do istniejącego bufora. Buduje nowy napis, a przypisanie zastępuje parę adresu i długości trzymaną w zmiennej. Stary napis `"prefix"` zostaje w arenie aż do końca regionu funkcji, ale nazwa już na niego nie wskazuje.

Listing 9.2. Stałe napisy są w bloku tylko oglądane. Program został zbudowany.

```
fun main() {
    var out = "prefix"
    val left = "L"
    val right = "R"
    {
        var piece = concat(left, right)
        out = move piece
    }
    println(out)
}
```

W drzewie regionów widać, że `piece` zostało przeniesione, a `left` i `right` są współdzielone z regionu funkcji. Dwie sąsiednie instrukcje pasują do rozpoznania z `hoist.rs`, więc znaki wyniku mogą powstać od razu w arenie `out`. Dla programisty skutek jest ten sam niezależnie od tego, czy kopiowanie zostało pominięte. Po bloku `out` da się wypisać.

Listing 9.3. Zmienne napisy trzeba przenieść jawnie. Program został zbudowany. Po bloku `left` i `right` są martwe.

```
fun main() {
    var out = "prefix"
    var left = "L"
    var right = "R"
    {
        var piece = concat(move left, move right)
    }
}
```

```
        out = move piece
    }
    println(out)
}
```

Czego jeszcze nie ma

Kompilator nie zgaduje więcej układów niż te dwie sąsiednie instrukcje. Nie ma bufora wyniku, który należałby do funkcji wywołującej. Nie ma `promote` przy `return`. Nawet `concat` stojący w jednej gałęzi warunku nie może być zwrócony.

Osobno, przy budowaniu, poprawne sprawdzenie nie wystarcza, żeby przekazać napis do funkcji użytkownika. Taki program przerywa kompilator. `concat` i `println` są wyjątkiem, bo mają własne fragmenty generowania kodu.

Podsumowanie

- Wartość napisu jest adresem i długością. Znaki leżą w stałej albo w buforze areny o pojemności 4096 bajtów.
- Przypisanie do zmiennej alokuje wynik w arenie tej zmiennej.
- Dwie sąsiednie instrukcje, deklaracja i zaraz potem przeniesienie do zmiennej zewnętrznej, mogą zbudować znaki od razu w arenie celu.
- `promote` kopiuje, gdy znaki już powstały w zbyt krótkim regionie.
- Przykład `concat` na dwóch zmiennych napisowych z dokumentacji języka nie kompiluje się. Trzeba użyć stałych albo słowa `move`.
- `concat` zastępuje wartość zmiennej. Nie dopisuje znaków do starego bufora.

Rozdział 10. Komunikaty kompilatora i błędy podczas działania

Ten rozdział obejmuje

- jak czytać linię błędu i co znaczą cztery fazy
- czego język nie oferuje w miejsce wyjątków
- które błędy przerywają kompilację, a które przerywają gotowy program
- dlaczego czasem kompilator kończy się awarią zamiast komunikatem
- w jakiej kolejności warto naprawiać kilka błędów naraz

Język nie ma instrukcji obsługi błędu

Nie ma `throw`, `try`, `catch` ani typu `Result`. Pytajnik przy typie nie jest modelem błędu. Jest wartością, którą sprawdzenie rozumie, a generowanie kodu odrzuca. Program albo daje się skompilować, albo nie. W czasie działania są trzy awarie, które kod naprawdę wywołuje.

Dzielenie przez zero woła `abort` z biblioteki języka C. Proces kończy się sygnałem przerwania. W powłoce widać kod 134, a na wyjściu błędów nie ma tekstu Borka. Ten sam mechanizm przerywa program, gdy indeks tablicy nie mieści się w długości zapisanej w typie. Trzecia awaria dotyczy alokacji większej niż 4096 bajtów w jednym buforze areny. Biblioteka wykonawcza, napisana w Ruście, przerywa się wtedy komunikatem `arena overflow`, z liczbą potrzebnych bajtów i pojemnością 4096. Program w Borku nie może tego złapać.

NOTA

NOTA. W kodzie generatora jest także przerwanie przy dzieleniu najmniejszej liczby ze znakiem przez minus jeden. Literału ujemnego nie zapiszesz, więc tej ścieżki nie uruchamiałem z pliku `.bork`. Strażnik w funkcji `guard_int_div` porównuje dzielnik z zerem, a dla typów ze znakiem także parę złożoną z minimum typu i z minus jeden.

Jak wygląda linia błędu

Funkcja `print_diagnostics` w `src/main.rs` drukuje ścieżkę, numer linii, numer kolumny, słowo `error`, nazwę fazy i treść. Gdy błąd nie ma zakresu w pliku źródłowym, numer linii i kolumny znikają. Zostaje ścieżka: `error: faza: treść`.

Zakres źródłowy, po angielsku `span`, jest parą pozycji bajtowych w pliku. Kolumna na wydruku liczy znaki od ostatniego przejścia do nowego wiersza, plus jeden. Przy tekście ASCII, a taki jest cały dzisiejszy lekser, znak i bajt wypadają w tym samym miejscu.

Fazy nazywają się `parse`, `ownership`, `type` i `codegen`. W strukturze błędu jest też poziom ostrzeżenia, ale programy, które uruchamiałem, produkują wyłącznie błędy. Wiersz poleceń i tak drukuje słowo `error`.

Serwer edytora dokłada nazwę fazy jeszcze raz, na początku treści, i przelicza bajty na pozycje wymagane przez protokół. Rozdział 18 opisuje różnicę.

Błędy składni

Tłumaczenie błędu parsera jest w `diag::from_parse`. Niepoprawny znak daje `invalid token`. Nagły koniec pliku daje `unexpected end of file` i listę rzeczy oczekiwanych. Nieoczekiwany token daje `unexpected token` oraz tę samą listę. Dodatkowy token na końcu daje `unexpected extra token`. Błąd zgłoszony wprost przez gramatykę, na przykład zły cel przypisania, dostaje pozycję na końcu pliku.

Listing 10.1. Niedomknięta lista parametrów.

Źródło `fun oops` daje:

```
err_parse.bork:1:10: error: parse: unexpected end of file; expected
↪  r#[a-zA-Z_][a-zA-Z0-9_]*"#, ")", "val", "var"
```

Lista oczekiwanych rzeczy jest surowa i zawiera wyrażenie regularne nazwy. Przy błędzie składni nie ma drzewa regionów ani dalszych faz. Polecenie `--dump-arenas` nie wypisuje wtedy drzewa.

Błędy typów

Komunikaty powstają w katalogu `src/typeck`. Rozdziały 4–7 cytowały te, które uruchomiłem. Kilka dotyczy całej funkcji, nie jednego wyrażenia. Brak powrotu na wszystkich ścieżkach, ponowna deklaracja funkcji wbudowanej, nieznana nazwa, nieznana funkcja oraz zła liczba argumentów należą do tej grupy.

Sprawdzanie typów nie zatrzymuje się na pierwszym błędzie. Zbiera listę. Potem i tak uruchamia się analiza własności. Dlatego jeden plik potrafi mieć zarówno fazę `type`, jak i fazę `ownership`. Test `reports_ownership_and_type_together` łączy dodawanie liczby do napisu z użyciem nazwy po przeniesieniu.

Błędy własności

Mają dwóch autorów. Pierwszym jest analiza nazw. Drugim jest analiza ucieczki. Druga nie startuje, gdy wcześniejsze błędy nie są puste. Przy błędzie typu możesz nie zobaczyć błędu ucieczki, który w poprawionym programie by się pojawił. Najpierw napraw typy i brakujące move, potem czytaj komunikaty o czasie życia bajtów.

Drzewo regionów powstaje także przy błędzie własności. Wypis przy użyciu nazwy po move pokazuje, że nazwa jest przeniesiona, i pokazuje deklarację, która tej nazwy użyła. Drzewo jest obrazem tego, co przejście zdążyło zbudować. Nie jest świadectwem, że program jest poprawny.

Błędy generowania kodu

Pojawiają się tylko przy bork build. Samo sprawdzenie ich nie produkuje. Zanim powstanie moduł LLVM, funkcja gate w src/codegen/gate.rs odrzuca konstrukcje, których generator nie umie przetłumaczyć. Należą do nich funkcja dopisana na końcu wywołania, None, Some, !!, operator ?:, pole inne niż length na napisie albo tablicy oraz operator, który nie jest na liście obsługiwanych.

Późniejsza emisja dokłada między innymi brak funkcji main, wynik main o typie innym niż i32 i unit, wywołanie pośrednie oraz działanie na liczbach zmiennoprzecinkowych. Ostatni tekst potrafi przyjść z dopiskiem internal arena schedule mismatch. Ten dopisek powstaje, gdy błąd spaceru po regionach jest opakowywany jako niezgodność harmonogramu. Treść po dwukropku mówi, czego naprawdę brakuje. Brzmi to surowo. Czasem jest wewnętrzną niezgodnością drzew, a czasem tylko opakowaniem zwykłego „jeszcze nie obsługujemy”.

Błąd tej fazy ma kod wyjścia jeden i nie zostawia pliku wykonywalnego. Pilnują tego testy odrzucenia None oraz braku main.

Awaria kompilatora

To nie jest komunikat dla programisty Borka. Proces bork kończy się kodem 101 i śladem stosu Rusta. Sprawdziłem dwa przypadki. Argument napisowy funkcji użytkownika oraz porównanie wartości f64 wchodzi w value_as_int. Tam wywołanie into_int_value zakłada, że wartość LLVM jest liczbą całkowitą. Dla napisu jest strukturą adresu i długości. Dla porównania f64 jest liczbą zmiennoprzecinkową. Inkwell, czyli biblioteka Rusta, przez którą kompilator woła LLVM, w takiej sytuacji przerywa proces, zamiast zwrócić błąd.

Przyczyna leży w coerce_value_to_ty. Dla bool wartość jest zwężana. Dla liczby zmiennoprzecinkowej część ścieżek zwraca komunikat, że argument nie jest obsługiwany. Dla reszty kod woła konwersję na liczbę całkowitą bez sprawdzenia, czym wartość naprawdę jest.

OSTRZEŻENIE

OSTRZEŻENIE. Awaria kompilatora na programie, który sprawdzenie uznało za poprawny, jest błędem generatora kodu. Nie czytaj jej jako zakazu języka. Sprawdzanie typów i analiza własności tego zapisu nie zabraniają. Dodatek C trzyma ten przypadek razem z innymi zaległościami.

Błąd narzędzia, a nie programu, kończy się kodem dwa. Należy tu złe polecenie, brak pliku, nakładające się ścieżki, brak clang albo brak opcji codegen. Tekst zaczyna się od `error:` i nie ma fazy.

Gdy błędów jest kilka

Kompilator nie wybiera jednego błędu i nie milczy o reszcie. Dostajesz wszystkie, które dana faza zdążyła zebrać. Praktyczna kolejność naprawy jest taka. Najpierw faza `parse`, bo przy błędzie składni reszty nie ma. Potem faza `type`, zwłaszcza nieznane nazwy i niezgodne typy. Potem faza `ownership` dotycząca `move`. Faza `codegen` ma sens dopiero wtedy, gdy sprawdzenie jest czyste. Budowanie najpierw sprawdza program. Przy błędach sprawdzenia w ogóle nie wchodzi w LLVM. Wypisuje te błędy i kończy się kodem jeden.

Sprawdziłem budowanie programu z `None`. Faza to `codegen`, kod wyjścia jeden, pliku wynikowego nie ma. Program, który zwraca napis z regionu wewnętrznego, też kończy budowanie kodem jeden, z tekstem `inner region`, bez pliku wynikowego. Analiza ucieczki jest częścią sprawdzenia, więc budowanie nie udaje, że to problem LLVM.

Podsumowanie

- Język nie ma wyjątków. Ma komunikaty kompilacji i trzy awarie w czasie działania.
- Linia błędu podaje fazę. Brak zakresu źródłowego usuwa numer linii.
- Faza `ownership` pochodzi albo z analizy nazw, albo z analizy ucieczki. Druga milczy, gdy wcześniej są inne błędy.
- Kontrola przed generowaniem kodu mówi wprost, że konstrukcja nie jest obsługiwana. Część późniejszej emisji mówi, że nie jest obsługiwana jeszcze.
- Przekazanie napisu do funkcji użytkownika nie daje komunikatu. Przerywa proces kompilatora.
- Kod jeden oznacza zły program. Kod dwa oznacza złe wywołanie albo brak narzędzia. Kod 101 oznacza awarię procesu kompilatora.

Rozdział 11. Funkcje wbudowane i sposób pisania programów

Ten rozdział obejmuje

- dlaczego w języku nie ma modułów
- jakie funkcje wbudowane naprawdę istnieją
- jak pisać programy, które dają się zbudować
- jak pisać programy, które na razie tylko przechodzą sprawdzenie
- których zapisów kompilator nie przyjmie, choć wyglądają znajomo

Modułów nie ma

Parser na poziomie programu oczekuje nowej linii albo słowa `fun`. Słowa `module` i `struct` są błędami składni. Nie ma `import`, ścieżek plików, widoczności ani osobnych jednostek kompilacji po stronie języka. Jeden plik zawiera jedną listę funkcji i jedną mapę nazw.

Słowo `moduł` wraca w tej książce tylko przy opisie dwóch skrzynek Rusta, `bork` i `bork_runtime`. Skrzynka, po angielsku `crate`, jest jednostką kompilacji w Cargo. To podział kodu kompilatora, nie mechanizm języka Bork. Serwer edytora i rozszerzenie w `tools/bork-lsp-extension` są narzędziami obok języka. Nie dodają składni.

Cała biblioteka, która istnieje

W `src/builtins.rs` są trzy nazwy. Nic więcej nie jest zarejestrowane.

`print` wypisuje jedną wartość `i32`, `i64` albo napis i opróżnia bufor standardowego wyjścia. `println` robi to samo i dodaje nowy wiersz. `concat` przyjmuje dwa napisy i zwraca nowy napis. Bufor powstaje w arenie miejsca, do którego wynik jest zapisywany, a gdy takiego miejsca nie ma, w arenie bieżącego bloku.

Nie ma asercji, formatowania, czytania standardowego wejścia, plików, zegara ani argumentów wiersza poleceń. Funkcja `main` nie dostaje `argv`. Generator kodu odrzuca `main` z parametrami już przy deklaracji funkcji LLVM.

Biblioteka wykonawcza ma funkcje, których nie wołasz z Borka wprost. Należą do nich pobranie bufora areny, jego zwrot, wyczyszczenie wskaźnika, alokacja w buforze oraz wypisywanie liczb i napisów. Rozdział 17 wymienia ich nazwy w C, bo taki jest sposób połączenia wygenerowanego kodu z biblioteką wykonawczą.

Jak pisać programy, które kompilator potrafi zbudować

Wynik liczbowy jako kod wyjścia jest wygodny, gdy ćwiczysz budowanie bez wypisywania. Funkcja `main`, która zwraca 7, została zbudowana i kończy się kodem 7. Tak samo robi `test builds_return_constant`.

Pętla z liczbowym akumulatorem nie walczy z własnością. Listing 6.3 jest wzorem. Indeks i suma są typu `i32`, więc ciało pętli je kopiuje.

Napis, który ma przeżyć blok, trzymaj w `main` i wypisz na końcu. Listing 9.1 pokazuje przypisanie literału do zmiennej z zewnątrz. Nie zwracaj świeżego napisu z funkcji pomocniczej, dopóki nie ma bufora należącego do wywołującego.

Stała napisowa, którą blok wewnętrzny tylko czyta, jest prostsza niż przenoszenie w tę i z powrotem. Listing 2.2 jest wzorem współdzielenia.

Gdy chcesz, żeby kompilator zbudował napis od razu w arenie zmiennej docelowej, napisz deklarację i w następnym wierszu przeniesienie. Nie wstawiaj między nie `println`.

Tablicy używaj z wycinkiem o stałych granicach, gdy długość jest znana w typie. Listing 7.4 to pokazuje. Wycinka o granicach trzymany w zmiennych sprawdzanie typów nie przyjmie, bo typ wyniku nie miałby znanej długości.

Warunek, który ma dać liczbę, zapisuj z dwiema gałęziami-blokami. Listing 6.1 jest wzorem.

Programy tylko do sprawdzenia

Pisz je, gdy ćwiczysz kompilator albo edytor. Nie wkładaj ich do programu, który ma być plikiem wykonywalnym.

Należą tu funkcja dopisana na końcu wywołania, blok `move` stojący po wywołaniu, `Some`, `None`, `?:`, `!!` i `?..`. Należy tu także próba trzymania funkcji w stałej i wywołania jej pośrednio. Osobno należy tu napis jako parametr funkcji użytkownika. Sprawdzenie go przyjmuje, a budowanie przerywa kompilator. W bork `build` trzymaj się od tej kombinacji z daleka.

Listing 11.1. Fragment próbki `PROCESS_USER_SAMPLE` z `src/lib.rs`. Cała próbka przechodzi sprawdzenie. Budowanie odrzuca `?:`, `Some` i `None`.

```
fun processUser(name: String?, score: i32): i32 {
    val fallbackName: String = name ?: "Guest"
    val verifiedUser: String? = Some(fallbackName)
    val emptyMiddle: String? = None
    val verifiedLength: i32 = verifiedUser?.length ?: 0
    if (verifiedLength > 0) {
        return score
    }
    return 0
}
```

Oryginalna próbka w `lib.rs` jest dłuższa. Ma dodatkową nazwę i zagnieżdżony blok. Też przechodzi sprawdzenie. Zostawiam ją w źródle kompilatora jako test, nie jako program do zbudowania.

Zapisy, które wyglądają znajomo i są błędami

Dodawanie dwóch napisów operatorem `+` jest błędem typu. Operator nie skleja tekstu. `return -3` jest błędem składni, bo minus nie jest operatorem jednoargumentowym. `val n = 1;` jest błędem składni. Przypisanie `var x = s` przy zmiennej napisowej `s` prosi o move. Wywołanie `concat` na dwóch zmiennych napisowych wewnątrz bloku prosi o przeniesienie każdej z nich. `return concat(a, b)` odpada w analizie ucieczki. Indeks spoza tablicy kompiluje się i przerywa program. Dzielenie `1 / 0` kompiluje się i przerywa program. Druga deklaracja `println` jest błędem typu. Słowa `struct` i `module` są błędami składni na poziomie programu.

Styl, który pasuje do regionów

Krótki blok robi jedną arenę na wartości tymczasowe. Długi blok funkcji trzyma dane, które mają przeżyć pętlę. Pętla nie powinna przenosić stanu utworzonego przed nią. Powinna czytać wartości kopiowane albo stałe współdzielone i zapisywać wynik do zmiennej z zewnątrz przez przypisanie.

Stałych używaj do progów i do napisów, które są tylko czytane. Zmiennych używaj do akumulatorów. To nie jest rada stylistyczna obok kompilatora. To jest różnica, którą analiza własności sprawdza.

Nie ma odpowiednika rustowego `clone`. Świadoma kopia napisu wymaga zbudowania nowego napisu, na przykład przez `concat`, gdy oba argumenty są w danym regionie legalne, albo przez przypisanie literału. Nie ma ogólnego kopiowania tablicy poza zbudowaniem nowego literału. Wycinek nie jest kopią.

Podsumowanie

- Biblioteka języka to `print`, `println` i `concat`.
- Modułów, struktur i `import` nie ma. Program jest jednym plikiem.
- Program, który ma się zbudować, trzyma funkcje przy liczbach, napisy w `main`, stałe przy samym odczycie i zmienne przy akumulatorach.
- Brak wartości i funkcja dopisana na końcu wywołania są prawdziwą częścią sprawdzanego języka i nie są częścią bork `build`.
- Operator `+` na napisach, średnik, minus jednoargumentowy i gołe użycie zmiennej napisowej są błędami kompilacji.

Rozdział 12. Od pliku źródłowego do gotowego programu

Ten rozdział obejmuje

- w jakiej kolejności kompilator sprawdza program
- co zapamiętuje każda pośrednia struktura danych
- dlaczego sprawdzanie typów wykonuje się przed analizą własności
- kiedy znika reprezentacja pośrednia, a zostaje drzewo regionów
- jak z poprawnego programu powstaje plik wykonywalny

Dwa wejścia, jedna wspólna kontrola

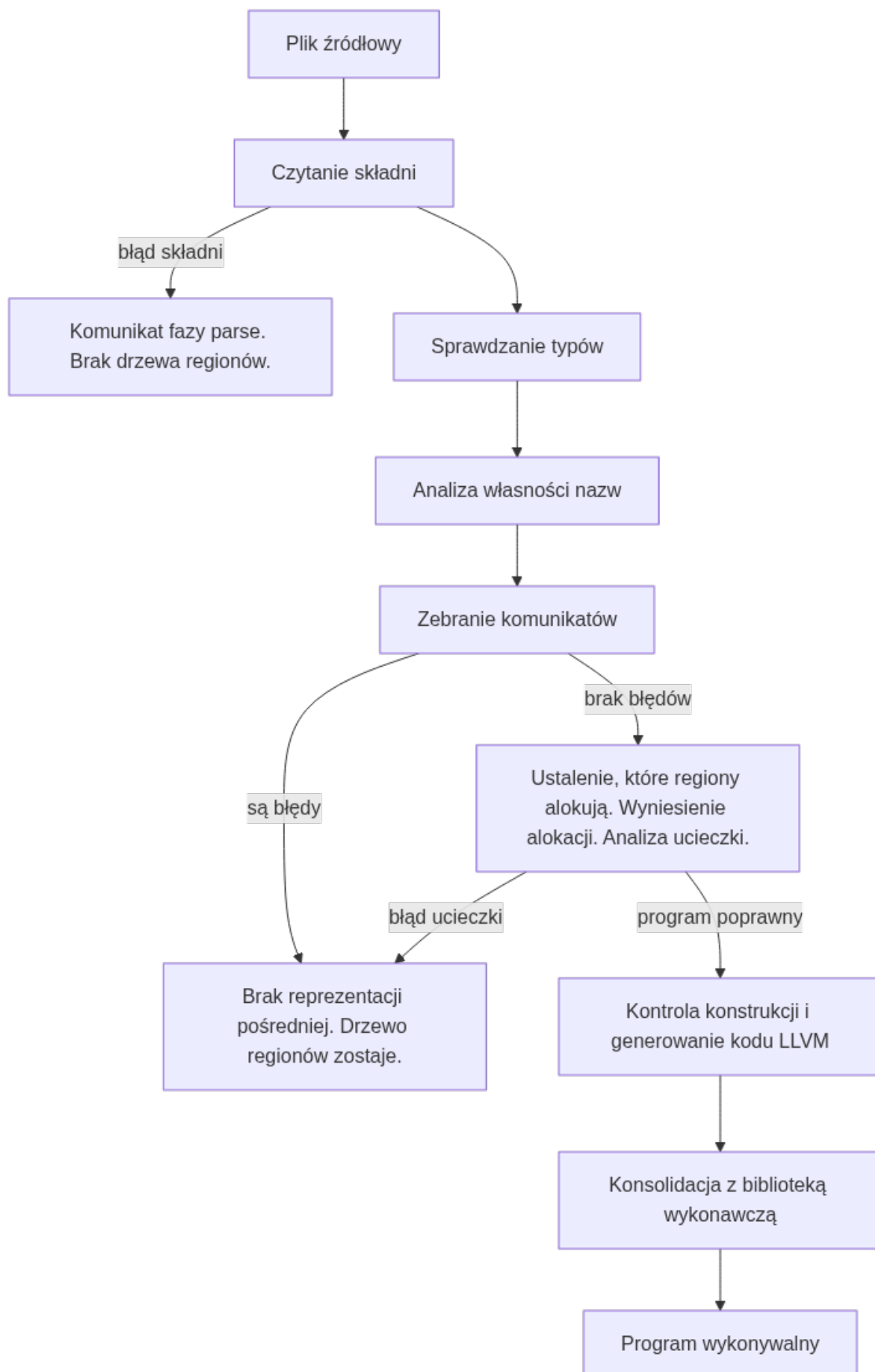
Kompilator nie ma interpretera i nie ma osobnej reprezentacji maszynowej niższego poziomu niż LLVM. Są dwa wejścia. Polecenie `bork plik.bork` kończy się na sprawdzeniu. Polecenie `bork build` woła to samo sprawdzenie, a potem, tylko gdy nie ma błędów, tłumaczy program na kod.

Komentarz na początku `src/frontend.rs` streszcza kolejność pracy jako czytanie, sprawdzanie typów, analizę własności i reprezentację pośrednią. To jest skrót. Gdy nie ma błędów, dochodzą jeszcze trzy przejścia. Pierwsze oznacza, które regiony naprawdę potrzebują bufora. Drugie przenosi alokację napisu do areny zmiennej docelowej, jeśli widzi opisany wcześniej układ dwóch instrukcji. Trzecie jest analizą ucieczki. Reprezentacja pośrednia w wyniku sprawdzenia jest obecna tylko wtedy, gdy po analizie ucieczki lista błędów nadal jest pusta.

Co zwraca sprawdzenie

Funkcja `frontend::check` zwraca strukturę `CheckResult`. Ma ona trzy pola. Pole `report` to drzewo regionów. Jest obecne, gdy składnia się udała, także przy błędach typów, własności i ucieczki. Pole `hir` to reprezentacja pośrednia z typami. Jest obecna tylko przy pustej liście błędów. Pole `diagnostics` zbiera komunikaty.

Skrót HIR oznacza tę reprezentację pośrednią. Po angielsku *high-level intermediate representation*, czyli pośrednią postać programu, która ma już typy, ale nie ma jeszcze instrukcji maszynowych. Dalej piszę o niej jako o reprezentacji pośredniej, a skrót HIR zostawiam przy nazwach typów w kodzie Rusta, bo tak nazywają się struktury `HirProgram` i `HirExpr`.



Rysunek 1: Diagram 1

Kolejność w liście komunikatów jest odwrotna do kolejności pracy. Najpierw dopisywane są błędy analizy własności, potem błędy typów. Sprawdzanie typów wykonało się wcześniej, bo analiza własności potrzebuje typów deklaracji. Wypis idzie w drugą stronę. Robią to linie 38-42 w `src/frontend.rs`.

Dlaczego typy są przed własnością

Starsza notatka projektowa opisuje kolejność odwrotną, najpierw własność, potem typy. Kod robi inaczej. Analiza własności czyta wektor typów w kolejności deklaracji `val` i `var`. Przy każdej deklaracji zdejmuję kolejny typ. Bez wcześniejszego sprawdzenia typów nie wie, czy nazwa jest kopiowana. Funkcja `sema::analize`, używana w części testów, uruchamia sprawdzanie typów wewnętrznie jeszcze raz. Ścieżka `frontend::check` używa `analize_with_decl_tys`, żeby nie płacić dwa razy.

Analiza własności nie dostaje całej reprezentacji pośredniej. Dostaje drzewo składni i cienki wektor typów deklaracji. Sygnatury wywołań buduje sobie z drzewa składni. Informacja, czy parametr jest stały, czy zmienny, jest informacją o własności, nie tylko o typie.

Co która struktura pamięta

Drzewo składni pamięta kształt zapisu, pozycje w pliku i to, czy nazwa jest stała. Nie pamięta typów wywnioskowanych ani regionów. Reprezentacja pośrednia pamięta typ przy każdym wyrażeniu, sposób użycia nazwy oraz, po wyniesieniu alokacji, nazwę zmiennej, w której arenę mają trafić bajty. Nie pamięta numeru regionu. Drzewo regionów pamięta zagnieżdżenie, własność nazw i później znacznik, czy region dostanie bufor w czasie działania. Nie pamięta pełnych typów reprezentacji pośredniej. Część typów spłaszcza do typu nieznanego. Moduł LLVM pamięta instrukcje i stałe literały. Nie pamięta już tekstu źródłowego. Komunikaty powstają wcześniej.

Komentarz w `src/hir/mod.rs` jest normą dla reszty kompilatora. Reprezentacja pośrednia nie niesie tożsamości regionu. Regiony żyją w drzewie z analizy własności. Generator kodu idzie po obu strukturach równocześnie, wspólnym spacerem. Gdy liczba dzieci w drzewie regionów nie zgadza się z liczbą miejsc w reprezentacji pośredniej, budowanie kończy się błędem wewnętrznym o niezgodności harmonogramu regionów. Nie jest to błąd, który programista Borka popełnił w składni. Jest to błąd zgodności dwóch przejść kompilatora.

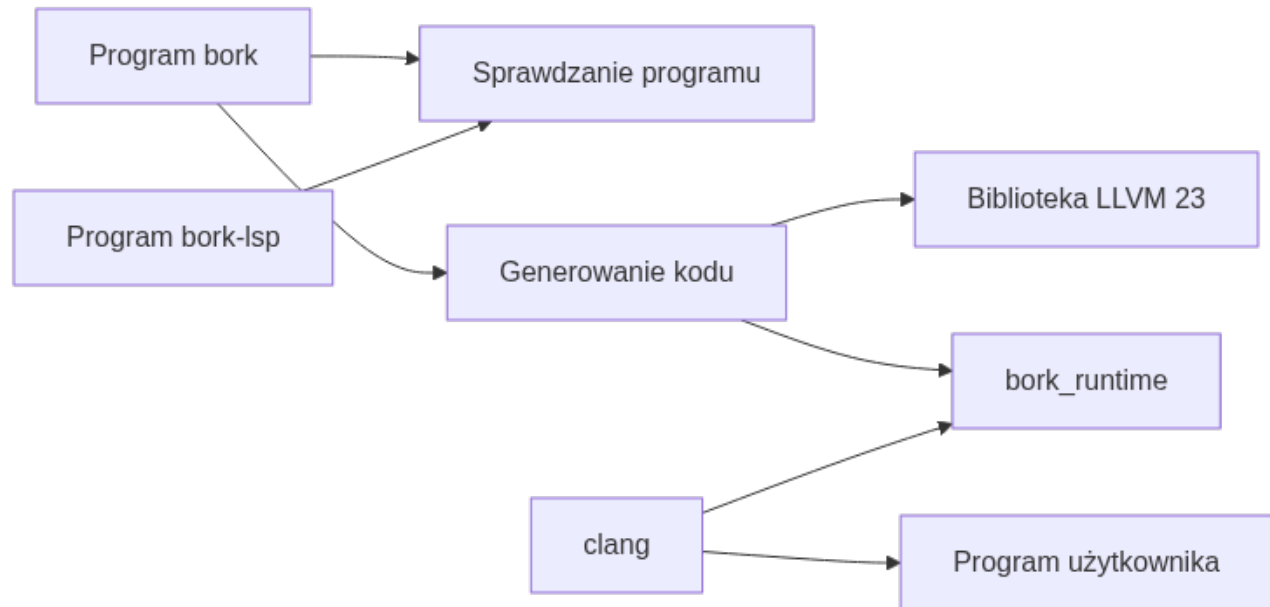
Co dzieje się przy budowaniu

Funkcja `codegen::build` w `src/codegen/mod.rs` wymaga czystego wyniku sprawdzenia. W przeciwnym razie zwraca komunikaty wcześniejszych faz. Potem funkcja `gate`, czyli kontrola przed generowaniem kodu, odrzuca konstrukcje spoza obsługiwanego zestawu. Następnie powstaje kontekst LLVM, moduł i plik obiektowy. Na końcu `link::link_executable` woła `clang` z plikiem obiektowym i z archiwum `libbork_runtime.a`.

Błąd konsolidacji i brak `clang` są błędem narzędzia. Kod wyjścia wynosi dwa, a tekst zaczyna się od `error:` bez fazy. Błąd funkcji `gate` jest błędem programu fazy `codegen` i

ma kod jeden.

Jak podzielony jest projekt



Rysunek 2: Diagram 2

`bork_runtime` nie jest zwykłą zależnością, którą kod Rusta importuje słowem `use`. W Cargo jednostka kompilacji nazywa się skrzynką, po angielsku `crate`. Plik `build.rs`, przy opcji `codegen`, kompiluje skrzynkę `bork_runtime` jako bibliotekę statyczną i przekazuje ścieżkę przez zmienną `BORK_RUNTIME_LIB`. Program użytkownika jest z nią konsolidowany. Sam kompilator ładuje `libLLVM` dynamicznie.

Opcja `lsp` jest domyślna. Dokłada biblioteki serwera językowego i program `bork-lsp`. Opcja `codegen` dokłada `Inkwell`, czyli bibliotekę Rusta, przez którą kompilator woła LLVM. Da się złożyć sam program sprawdzający, bez żadnej z tych opcji. Wtedy `bork build` odmawia pracy kodem dwa.

Czego w tej kolejności pracy nie ma

Nie ma osobnego optymalizatora poza tym, co LLVM zrobi z gotowym modulem. Nie ma wklejania funkcji dopisanych na końcu wywołania, bo te funkcje nie dochodzą do LLVM. Nie ma osobnych kopii funkcji dla różnych typów, bo nie ma typów ogólnych. Jedyną adnotacją dokładaną do reprezentacji pośredniej po sprawdzeniu typów jest nazwa zmiennej, w której arenie mają powstać bajty napisu. Jest lokalna i dotyczy dwóch sąsiednich instrukcji.

Plik `src/arena.rs` nie jest jedną z faz tej kolejności. To model bufora o pojemności 4096 bajtów. Analiza własności `go` nie woła. Biblioteka wykonawcza ma własną kopię stałej pojemności.

Podsumowanie

- Sprawdzenie czyta składnię, sprawdza typy, analizuje własność, a przy braku błędów ustala bufory regionów, wynosi alokację i sprawdza ucieczkę.
- Reprezentacja pośrednia zostaje w wyniku tylko dla programu bez komunikatów.
- Drzewo regionów przeżywa błędy znaczenia i znika tylko przy błędzie składni.
- Sprawdzanie typów jest przed analizą własności, bo ta druga potrzebuje typów deklaracji. Starsza notatka projektowa opisuje kolejność odwrotną.
- Reprezentacja pośrednia nie ma numerów regionów. Generator kodu uzgadnia ją z drzewem regionów wspólnym spacerem.
- Plik wykonywalny jest obiektem LLVM skonsolidowanym z `libbork_runtime.a` przez clang.

Rozdział 13. Jak kompilator czyta tekst programu

Ten rozdział obejmuje

- czym jest lekser i czym jest parser w tym projekcie
- w jakiej kolejności wiążą się operatory
- jakie węzły ma drzewo składni
- po co kompilator przerabia nowe linie wewnątrz nawiasów
- jak błąd czytania staje się komunikatem

Jedna gramatyka czyta i składa

Lekser zamienia tekst na tokeny, czyli słowa, liczby, operatory i końce wierszy. Parser układa tokeny w drzewo. W Borku obie te rzeczy są zapisane w jednym pliku, `src/parser.lalrpop`. Narzędzie LALRPOP generuje z tej gramatyki kod Rusta. Plik `build.rs` uruchamia to generowanie przy każdej kompilacji kompilatora. Wejście dla reszty programu to funkcja `parse` w `src/lib.rs`. Najpierw normalizuje nowe linie w nawiasach, potem woła parser programu. Błąd ma typ `ParseError` z biblioteki LALRPOP.

Nie ma osobnego pliku z listą tokenów. Lekser jest blokiem dopasowań na początku gramatyki.

Co lekser pomija, a co zamienia na token

Pomijane są spacje, tabulatory i komentarz `//` do końca wiersza. Znak powrotu karetki też jest odstępem. Dzięki temu zamiana nowych linii wewnątrz nawiasów na odstęp naprawdę ukrywa je przed lekserem. Przejście do nowego wiersza, które nie stoi tuż przed `else`, staje się tokenem `NL`. Słowo `else` wchłania poprzedzające puste wiersze i komentarze, żeby warunek mógł złamać linię przed drugą gałęzią.

Słowa kluczowe w lekserze to `if`, `fun`, `val`, `var`, `for`, `in`, `return`, `Some`, `None`, `move` i `promote`. Reszta, w tym `while` i `true`, idzie zwykłą ścieżką i jest porównywana jako konkretny napis w regule gramatyki.

Liczba całkowita musi zmieścić się w `i64`. W przeciwnym razie gramatyka zgłasza `integer literal out of range for i64`. Liczba z kropką, która nie da się odczytać, daje

invalid float literal. Napis przechodzi przez funkcję usuwającą sekwencje ucieczki, zapisaną w `src/ast.rs`.

Kolejność operatorów

Najmocniej wiążą się wywołanie, indeks, wycinek, odczyt pola i przyrostkowe `!!`. Potem jest negacja `!`. Potem mnożenie i dzielenie. Potem dodawanie i odejmowanie. Potem porównania. Potem zakres `...`. Potem koniunkcja `&&`. Potem alternatywa `||`. Najślabiej wiąże się `?:`.

Operator `?:` jest prawostronnie łączny, bo prawa strona reguły woła z powrotem tę samą regułę. Pozostałe operatory dwuargumentowe są lewostronnie łączne.

Dokument `docs/language.md` w jednym zdaniu o kolejności wymienia wywołania, mnożenie, dodawanie, porównania, zakres i `?:`. Pomija `&&`, `||` i `!`, które w gramatyce są. Ta książka trzyma się gramatyki. Koniunkcja wiąże mocniej niż alternatywa. Obie wiążą słabiej niż porównanie i mocniej niż `?:`.

Indeks i granice wycinka używają reguły porównania, nie pełnego wyrażenia. Bez dodatkowych nawiasów nie włożysz `?:`, `||`, `&&` ani `..` do środka nawiasów kwadratowych.

Drzewo składni

Plik `src/ast.rs` definiuje drzewo. Program ma listę funkcji. Funkcja ma nazwę, parametry, typ wyniku i ciało. Parametr ma rodzaj wiązania, stały albo zmienny, nazwę z zakresem źródłowym i typ.

Typ w drzewie składni jest albo typem prostym, albo nazwą, albo tablicą o długości, albo typem funkcji. Każdy z nich może mieć znacznik pustej wartości, choć później sprawdzanie typów odrzuca pustą tablicę.

Instrukcja jest blokiem, deklaracją, przypisaniem, pętlą `for`, pętlą `while`, `break`, `continue`, blokiem `move`, powrotem albo wyrażeniem użytym jako instrukcja. Cel przypisania jest albo nazwą, albo indeksem. Inny cel ginie już w gramatyce.

Wyrażenie jest literałem, indeksem, wycinkiem, nazwą, przeniesieniem, promocją, `None`, `Some`, działaniem, negacją, odczytem pola, wywołaniem albo warunkiem. Wywołanie może nieść funkcję dopisaną na końcu. Ta funkcja ma parametry, ciało, znacznik `move` i opcjonalną listę przenoszonych nazw.

Lista przenoszonych nazw jest wartością opcjonalną. Brak listy oznacza zgadywanie. Lista pusta oznacza jawne „nic nie przenoś”. Tej różnicy nie wolno spłaszczyć do jednego pustego wektora.

Zakres źródłowy jest parą `start` i `end` w bajtach. Notatka projektowa pierwszej wersji parsera mówiła, że zakresów nie będzie. To jest nieaktualne. Zakresy są w całym drzewie.

Po co przerabiać nowe linie

Funkcja w `src/layout.rs` zamienia `\n` na `\r` wewnątrz nawiasów okrągłych, na głębokości nawiasów klamrowych z chwili otwarcia nawiasu. Pomija napisy i komentarze. Długość pliku w bajtach się nie zmienia, więc pozycje komunikatów się nie przesuwają. Testy w `tests/parser.rs` sprawdzają pusty program, próbkę z funkcją na końcu wywołania, przypisanie do indeksu, parametry stałe i zmienne, bloki `move`, komentarze, kolejność operatorów, pętlę `while` i miejsca błędów. To jest pierwszy plik testowy, który warto czytać, gdy ruszasz gramatykę.

Od błędu parsera do komunikatu

Funkcja `from_parse` w `src/diag.rs` zamienia warianty błędu LALRPOP na fazę `parse`. Błąd zgłoszony przez regułę gramatyki dostaje zakres o początku i końcu równym długości pliku, czyli pozycję na końcu. Dlatego zdanie `assignment target must be a name or name[index]` wskazuje koniec pliku. Poprawka byłaby lokalna albo w tłumaczeniu błędu, albo w akcji gramatyki, która dziś nie niesie pozycji. Nikt jej jeszcze nie zrobił.

Funkcja `frontend::check` przy błędzie składni wraca natychmiast. Sprawdzanie typów i analiza własności się nie wykonują.

Podsumowanie

- Gramatyka LALRPOP jest jedynym lekserem i jedynym parserem.
- Nowa linia jest tokenem, chyba że wewnątrz nawiasów została zamieniona na odstęp albo stoi przed `else`.
- Brak listy przy `move` oznacza zgadywanie. Pusta lista oznacza świadomą rezygnację ze zgadywania.
- Kolejność operatorów obejmuje negację, koniunkcję i alternatywę, nawet jeśli skrót w dokumentacji języka je pomija.
- Błąd zgłoszony przez regułę gramatyki ma pozycję na końcu pliku.
- Drzewo składni nie niesie typów wywnioskowanych. To robota sprawdzania typów.

Rozdział 14. Jak kompilator sprawdza własność nazw

Ten rozdział obejmuje

- czym jest analiza własności i czym różni się od bufora pamięci
- jak powstaje drzewo regionów dla każdej funkcji
- według jakiej reguły nazwa jest kopiowana, współdzielona albo musi być przeniesiona
- jak pętla i warunek zmieniają to, co wolno przenieść
- czego ta analiza świadomie nie sprawdza

Analiza własności opisuje nazwy, a nie bajty

Analiza własności odpowiada na pytanie, czy daną nazwę wolno w tym miejscu odczytać. W kodzie katalog tej analizy nazywa się `sema`, od angielskiego `semantic analysis`, czyli analizy znaczenia nazw. Dalej piszę o niej jako o analizie własności. Nie przydziela ona pamięci. Plik `src/arena.rs`, który opisuje bufor o pojemności 4096 bajtów, zaczyna się komentarzem, że ta analiza go nie używa. Zamiast bufora powstaje drzewo. Każdy węzeł drzewa opisuje jeden region: które nazwy w nim powstały i które nazwy z zewnątrz zostały w nim użyte.

Węzeł nazywa się w kodzie `ArenaNode`. Ma numer, etykietę tekstową, listę wiązań, listę obserwacji i listę dzieci. Numer pochodzi z licznika w strukturze `Analyser` i jest zwykłą liczbą całkowitą bez znaku. Nie ma osobnego typu identyfikatora regionu. Nie ma też wyliczenia rodzaju regionu, choć starsze notatki projektowe o takim wyliczeniu wspominały. Region jest albo zwykły, albo jest blokiem `move`. Różnicę widać po funkcji, która go otwiera. Zwykły region otwiera `open_ordinary`. Blok `move` otwiera `open_move`. Etykieta, na przykład `fun main`, `Block` albo `ForLoop (i)`, jest napisem do wydruku i do komunikatów.

Korzeń drzewa dla całego programu trzyma struktura `ArenaReport`. Ma ona po jednym korzeniu na funkcję.

Co pamięta jedno wiązanie

Wiązanie w środowisku analizy, w kodzie typ `EnvBinding`, pamięta numer regionu, etykietę tego regionu, typ, rodzaj własności, flagę przeniesienia i pochodzenie nazwy.

Typ jest albo znany, albo nieznany. Znany typ pochodzi z drzewa składni. Typ nieznany, w kodzie Unknown, dostają parametry funkcji dopisanej na końcu wywołania. Analiza własności nie czyta typów, które sprawdzanie typów wyliczyło dla tych parametrów. Pochodzenie nazwy jest albo deklaracją, albo przechwyceniem w bloku move.

Rodzaj własności, w kodzie typ Ownership, ma cztery warianty. Local oznacza nazwę powstałą w bieżącym regionie. Copy oznacza odczyt wartości, którą wolno skopiować. Shared oznacza odczyt nazwy stałej, której nie wolno skopiować, i pamięta etykietę regionu, z którego nazwa pochodzi. Moved oznacza, że własność została przeniesiona, i też pamięta etykietę źródła.

Wydruk drzewa pomija obserwacje lokalne, żeby nie powtarzać deklaracji. Obserwacje kopiowania, współdzielenia i przeniesienia zostają na wydruku.

Jak otwiera się region

Wejście do analizy jednej funkcji nazywa się `analyze_with_decl_tys`. Najpierw zapisuje, czy parametry każdej funkcji są stałe, czy zmienne. Potem woła `open_ordinary` z etykietą złożoną ze słowa `fun` i nazwy funkcji, z ciałem funkcji i z listą parametrów.

Otwarcie regionu, czy zwykłego, czy blokiem `move`, zaczyna się od zdjęcia opakowań. Funkcja `peel_blocks` usuwa bloki, które zawierają tylko jeden wewnętrzny blok, i liczy, ile takich nawiasów zdjęła. Licznik ładuje w polu `compacted_braces`. Dzięki temu zapis `{ { instrukcje } }` nie tworzy dwóch regionów. Potem analiza bierze nowy numer, buduje ramkę regionu i wiąże parametry. Dla bloku `move` dodatkowo przechwytyje nazwy z listy. Przechwycenie przenosi źródło, cieniuje nazwę jako przechwyconą, dopisuje przeniesienie do dziecka, a po zamknięciu regionu oznacza źródło u rodzica jako przeniesione. Następnie analiza schodzi w instrukcje. Na końcu zdejmuje cienie, czyli przywraca nazwy, które były widoczne przed wejściem.

Lista nazw przy `move` jest rozstrzygana w funkcji `resolve_move_captures`. Jeśli programista podał listę, wygrywa ona ze zgadywaniem, także gdy jest pusta. Jeśli listy nie ma, kompilator zbiera nazwy wolne w bloku, odrzuca parametry i zostawia nazwy żywe, których typ nie jest kopiowalny. Wyrażenie `move` i wyrażenie `promote` nie liczą się jako zmienne wolne. Ta różnica jest zapisana w drzewie składni. Brak listy to wartość pusta. Pusta lista to lista podana jawnie. Nie wolno ich spłaszczyć do jednego pustego wektora, bo znaczą co innego.

Reguła jednego odczytu

Decyzja o kopiowaniu, współdzieleniu i przeniesieniu mieści się w funkcji `classify_use` w pliku `src/sema/policy.rs`. Kolejność sprawdzeń jest stała.

Jeśli wiązanie jest już przeniesione, odczyt jest błędem. Komunikat mówi, że nazwy użyto po przeniesieniu, i podaje etykietę regionu, z którego ją przeniesiono.

Jeśli numer regionu wiązania jest taki sam jak numer bieżącego regionu, odczyt jest lokalny. Nazwa żyje w tym regionie, więc nie przekracza jego granicy.

Jeśli typ jest kopiowalny, odczyt jest kopią. Źródło zostaje żywe. Kopiowalne są tylko typy proste, które nie mogą być puste. Mowa o tym była w rozdziale 4.

Jeśli wiązanie jest stałe, odczyt jest współdzieleniem. Region wewnętrzny widzi bajty regionu zewnętrznego, ale ich nie kopiuje i nie unieważnia nazwy.

W pozostałych przypadkach odczyt jest błędem. Chodzi o zmienną z regionu zewnętrznego, której nie wolno skopiować. Komunikat każe przenieść nazwę słowem `move` do regionu o bieżącej etykietce.

Przypisanie do zmiennej z regionu zewnętrznego omija tę regułę po lewej stronie. Jeśli cel jest zmienną, nie został przeniesiony, a jego region ma mniejszy numer niż region bieżący, analiza nie traktuje lewej strony jako odczytu. Prawa strona jest sprawdzana osobno. Dzięki temu zapis `outer =` wyrażenie zmienia istniejącą zmienną, a nie próbuje jej przenieść.

Krótsze podpowiedzi, gdy programista napisał gołą nazwę tam, gdzie potrzebne jest `move`, buduje funkcja `bare_ident_move_message`. Dla argumentu wywołania tekst każe użyć `move`, żeby przekazać własność. Dla przypisania w tej samej okolicy każe użyć `move`, żeby własność przenieść. Gdy regiony się różnią, zostaje dłuższy tekst o braku kopiowania.

Pętla zabrania przenosić to, co było widać na wejściu

Przy pętli `for` i przy pętli `while` analiza odkłada na stos zbiór wszystkich nazw widocznych w tej chwili. Zbiór nazywa się w kodzie `loop_move_ban`. Po ciele pętli analiza go zdejmuje. Próba przeniesienia nazwy z tego zbioru kończy się błędem. Komunikat tłumaczy, że kolejna iteracja zobaczyłaby nazwę już przeniesioną. Nazwa utworzona wewnątrz pętli do zbioru nie należy, więc wolno ją przenieść w tej samej iteracji, w której powstała.

Warunek scala przeniesienia z obu gałęzi

Przy `if` analiza zapamiętuje, które nazwy są przeniesione, schodzi w gałąź prawdziwą, przywraca flagi, schodzi w gałąź `else`, jeśli druga gałąź istnieje, przywraca flagi jeszcze raz i scala wynik funkcją `apply_moved_merge`. Nazwa jest przeniesiona po całym warunku, gdy była przeniesiona już przed nim albo gdy została przeniesiona w obu gałęziach. Sama gałąź prawdziwa, bez `else`, nie zostawia przeniesienia na zewnątrz. Nie ma tu śledzenia, że nazwa jest martwa tylko przy prawdziwym warunku. Albo obie gałęzie ją przenoszą, albo po warunku nazwa jest nadal żywa, o ile żyła wcześniej.

Czego ta analiza nie sprawdza

Nie sprawdza, czy bajty wyniku `concat` przeżyją powrót z funkcji. To robi analiza ucieczki, opisana w rozdziale 16. Nie nadaje typów parametrom funkcji dopisanej na końcu wywołania i zostawia je jako nieznane. Typ nieznany nie jest kopiowalny. Test `unknown_type_is_not_treated_as_copy` pilnuje tego wyboru. Lepiej dostać zbędny komunikat o braku kopiowania przy programie, który i tak ma błąd typu, niż puścić wartość, która mogłaby uciec z regionu. Skutek uboczny widać w wydruku drzewa. Parametr takiej funkcji, który sprawdzanie typów uznało za liczbę całkowitą, w wydruku potrafi wyglądać jak współdzielenie.

Indeks pętli `for` jest w tej analizie zwykłą liczbą całkowitą. Nie ma ścieżek wyjątków, bo język ich nie ma. Pole `compacted_braces` liczy zdjęte nawiasy, a nie bajty w buforze.

Słowo `promote` jest obsługiwane przy zejściu w wyrażenie, w funkcji `apply_expr_promote`. W katalogu testów jednostkowych analizy nie ma osobnego pliku o promocji. Zachowanie jest w zejściu i w testach wyższego poziomu, które budują program. Milczenie testów jednostkowych nie znaczy, że promocja jest niezaimplementowana. Program z rozdziału 8, który przypisuje `promote`, daje się zbudować.

Wydruk drzewa nie powstaje w tej analizie. Powstaje w pliku `src/dump.rs`. Serwer edytora używa tej samej funkcji.

Podsumowanie

- Analiza własności buduje drzewo regionów i flagi przeniesienia. Nie przydziela buforów o pojemności 4096 bajtów.
- Funkcja `classify_use` jest definicją kopiowania, współdzielenia i przeniesienia.
- Brak listy przy `move` oznacza zgadywanie. Pusta lista oznacza rezygnację ze zgadywania.
- Pętla zabrania przenosić nazwy widoczne na wejściu. Warunek zostawia przeniesienie tylko wtedy, gdy zrobiły je obie gałęzie.
- Analiza ucieczki i wyniesienie alokacji są później i nie leżą w tym katalogu.
- Parametry funkcji dopisanej na końcu wywołania mają tu typ nieznany, więc wydruk drzewa potrafi pokazać współdzielenie tam, gdzie sprawdzanie typów widzi kopię.

Rozdział 15. Jak kompilator sprawdza typy

Ten rozdział obejmuje

- po co obok drzewa składni powstaje drugie drzewo, już z typami
- jakie typy istnieją tylko po sprawdzeniu, a jakich nie ma w składni
- jak oczekiwany typ zmienia znaczenie literału, pustej wartości i warunku
- dlaczego funkcje wbudowane przyjmują więcej, niż mówi ich nominalna sygnatura
- dlaczego samo drzewo z typami nie wystarcza generatorowi kodu

Drugie drzewo opisuje program po typach

Drzewo składni, opisane w rozdziale 13, ma kształt tekstu. Nie wie jeszcze, jaki typ ma literał 0 ani czy nazwa została skopiowana. Sprawdzanie typów buduje drugie drzewo. W kodzie nazywa się ono reprezentacją pośrednią. Skrót HIR, od angielskiego high-level intermediate representation, pojawia się w nazwach struktur HirProgram i HirExpr. W zdaniach zostają przy reprezentacji pośredniej. Rozdział 12 wprowadził to pojęcie przy opisie kolejności pracy kompilatora.

Każde wyrażenie w nowym drzewie niesie typ, zakres źródłowy i rodzaj wyrażenia. Wyrażenia move i promote z drzewa składni stają się zwykłymi nazwami. Rodzaj użycia, w kodzie UseKind, mówi wtedy, że było to przeniesienie albo promocja. Pozostałe warianty tego rodzaju to użycie lokalne, współdzielenie i kopia. Sprawdzanie typów wpisuje rodzaj użycia w funkcji check_ident. Nie zgłasza przy tym błędów własności. Własność zgłasza analiza opisana w poprzednim rozdziale. Reprezentacja pośrednia tylko zapisuje, jak sprawdzanie typów widziało odczyt. Generator kodu później ufa temu zapisowi, gdy kopiuje deskryptor napisu. Na liczbach i na napisach używanych w main testy trzymają oba opisy razem. Na parametrach funkcji dopisanej na końcu wywołania analiza własności i tak widzi typ nieznany, o czym była mowa w rozdziale 14.

Wywołanie traci ciało funkcji dopisanej na końcu jako osobną wartość i zostawia flagę, że taka funkcja była. Ciało i tak jest sprawdzane tam, gdzie sprawdzanie typów schodzi w blok. Flaga służy później kontroli przed generowaniem kodu. Deklaracja zmiennej dostaje typ oraz pole alloc_in_binding. Na tym etapie pole jest puste. Wypełni je dopiero wyniesienie alokacji, opisane w następnym rozdziale. Parametry w reprezentacji pośredniej nie mają zakresów źródłowych. Zostają same napisy z nazwami.

Jakie typy pojawiają się dopiero teraz

Typ w reprezentacji pośredniej, struktura Ty w pliku `src/hir/ty.rs`, ma rodzaj i znacznik, czy wartość może być pusta. Rodzaj jest typem prostym, nazwą, tablicą o długości, typem funkcji, zakresem albo typem nieznanym.

Zakres i typ nieznanymi nie mają odpowiednika w drzewie składni. Zakres istnieje tylko jako typ wyrażenia `lo..hi`. Typ nieznanymi tłumaczy dalsze komunikaty, gdy wcześniejszy błąd i tak zepsuł wyrażenie. Porównanie z typem nieznanymi nie dokłada kolejnego oczekiwania. Nie tłumaczy natomiast analizy własności.

Funkcja `is_copy` jest prawdziwa dla typu prostego, który nie może być pusty. Funkcja `uses_arena_storage` jest prawdziwa dla napisu i dla tablicy, która nie może być pusta. Drugi predykat mówi analizie ucieczki i generatorowi kodu, że wartość ma bajty w buforze regionu, a nie samą liczbę w rejestrze.

Tłumaczenie typu ze składni, funkcja `lower_type`, odrzuca pustą tablicę, czyli zapis `[T]?`, oraz każdą nazwę poza `String`.

Jak sprawdzanie typów schodzi po programie

Funkcja `typeck::check` zbiera sygnatury, odmawia ponownego zdefiniowania funkcji wbudowanych, a potem sprawdza ciała. Środowisko ma stos zakresów, mapę funkcji, listę komunikatów, wektor typów deklaracji i głębokość pętli. Wektor typów deklaracji, w kodzie `decl_tys`, rośnie przy każdej deklaracji, w kolejności źródła. Analiza własności zdejmuję z niego typy po kolei. Dlatego sprawdzanie typów musi skończyć się wcześniej. Rozdział 12 tłumaczy, czemu komunikaty i tak wypisują się w odwrotnej kolejności.

Instrukcje są w pliku `src/typeck/stmt.rs`. Wyrażenia są podzielone. Plik `expr/mod.rs` rozdziela rodzaje i obsługuje nazwy. Plik `expr/binary.rs` obsługuje arytmetykę, porównania, koniunkcję, alternatywę i zakres. Plik `expr/call.rs` obsługuje wywołania, funkcję dopisaną na końcu i funkcje wbudowane. Plik `expr/control.rs` obsługuje warunek i blok użyty jako wartość. Plik `expr/array.rs` obsługuje literał tablicy, indeks i wycinek. Plik `expr/field.rs` obsługuje odczyt pola i odczyt z operatorem `?.`

Ostatnia instrukcja bloku, który ma dać wartość, musi być gołym wyrażeniem. W przeciwnym razie typ bloku to `unit`. Robi to funkcja `check_value_block_in_current_scope`. Dlatego `return` w gałęzi warunku nie jest wartością tej gałęzi. Jest instrukcją. Gałąź jako wyrażenie ma typ `unit`, jeśli ostatnią rzeczą w niej nie jest gołe wyrażenie.

Oczekiwany typ steruje literałem i pustą wartością

Sprawdzanie typów przekazuje w dół oczekiwany typ, jeśli otoczenie go zna. Literał całkowity bez oczekiwania ma typ `i32`. Gdy otoczenie oczekuje typu całkowitego, literał przyjmuje ten typ. Literał zmiennoprzecinkowy bez oczekiwania ma typ `f64`. Słowo `None` bez oczekiwanego typu jest błędem. Nie staje się „jakimś” typem pustym.

Warunek bez `else`, gdy nikt nie oczekuje wartości, ma typ `unit`. Gdy otoczenie oczekuje typu, a drugiej gałęzi nie ma, komunikat mówi, że warunek produkujący wartość wymaga gałęzi `else`. Zakres tego komunikatu bywa pusty.

Funkcje wbudowane są specjalnym przypadkiem

Plik `expr/call.rs` rozpoznaje `print`, `println` i `concat`, zanim sprawdzi liczbę argumentów zwykłej funkcji. Dlatego `print` przyjmuje napis, choć tabela sygnatur w `src/builtins.rs` mówi, że `print` bierze `i32` i zwraca `unit`. To jest przypadek szczególny, a nie ogólna zasada, że jeden typ można podstawić pod drugi. Inna funkcja o parametrze `i32` napisu nie przyjmie.

Funkcja dopisana na końcu wywołania dokleja się jako ostatni argument typu funkcyjnego. Jej ciało jest sprawdzane w nowym zakresie. Analiza własności robi osobne zejście po drzewie składni i nie widzi typów wyliczonych dla parametrów tej funkcji. Dwa zejścia mają dwa środowiska. Stąd wydruk drzewa regionów potrafi pokazać współdzielenie tam, gdzie sprawdzanie typów widzi kopię. Rozdział 14 opisał ten skutek od strony analizy własności.

Dlaczego generator kodu nie może iść tylko po tym drzewie

Reprezentacja pośrednia nie ma numerów regionów. Komentarz w `src/hir/mod.rs` mówi, że regiony żyją w drzewie z analizy własności. Generator kodu ma iść wspólnym spacerem, opisanym w następnym rozdziale. Na każdym bloku, pętli, warunku i funkcji dopisanej na końcu bierze kolejne dziecko węzła regionu. Jeśli sprawdzanie typów wyrzuci albo wstawi blok inaczej niż analiza własności, spacer nie znajdzie dziecka.

Funkcja `peel_blocks` jest skopiowana w reprezentacji pośredniej i w analizie własności. Komentarz przy kopii każe trzymać obie wersje w zgodzie. Nie ma jednego wspólnego traitu. Jest konwencja. Zmiana tylko w jednym miejscu psuje uzgodnienie drzew.

Pole `alloc_in_binding` jest puste po sprawdzeniu typów. Wypełnia je wyniesienie alokacji, już na gotowym drzewie, w funkcji `frontend::check`. To jedyna adnotacja dokładana między sprawdzeniem typów a generowaniem kodu.

Testy w `src/typeck/tests/mod.rs` idą zwykle przez `frontend::check`, a nie przez samo sprawdzanie typów. Dzięki temu łapią także własność. Plik `nullable.rs` trzyma operator `?:`, wykrzykniki `!!` i porównania wartości pustych. Plik `closures.rs` trzyma liczbę argumentów funkcji dopisanej na końcu wywołania. Gdy dodajesz operator, dopisujesz tu przypadek z dokładnym tekstem komunikatu. Rozdział 21 wraca do tej kolejności pracy.

Podsumowanie

- Reprezentacja pośrednia niesie typ i rodzaj użycia nazwy. Nie niesie numeru regionu.
- Słowo `move` w drzewie składni staje się rodzajem użycia na nazwie.
- Sprawdzanie typów nie zgłasza błędów własności. Analiza własności nie czyta rodzaju użycia z reprezentacji pośredniej.
- Oczekiwany typ steruje literałami, słowem `None` i tym, czy warunek bez `else` jest błędem, czy ma typ `unit`.
- Dwa zejścia, sprawdzanie typów i analiza własności, spotykają się dopiero we wspólnym spacerze. Zgodność `peel_blocks` jest warunkiem tego spotkania.

Rozdział 16. Jak kompilator wybiera miejsce na napis

Ten rozdział obejmuje

- kiedy kompilator w ogóle zajmuje się miejscem bajtów
- jak dwie sąsiednie instrukcje przenoszą alokację do zmiennej docelowej
- jak głębokość regionu decyduje, czy napis wolno zwrócić albo przypisać
- po co oznaczenie regionów, analiza ucieczki i generator kodu idą jednym spacerem
- które regiony naprawdę pobierają bufor w czasie działania

Te trzy przejścia biegą tylko na programie bez wcześniejszych błędów

Gdy lista komunikatów po sprawdzeniu typów i po analizie własności jest pusta, funkcja `frontend::check` robi jeszcze trzy rzeczy. Najpierw `region_walk::stamp_codegen_push` oznacza, które regiony mają pobrać bufor. Potem `hoist::annotate` dopisuje do deklaracji informację, w której zmiennej zewnętrznej alokować napis. Na końcu `escape::check_function` sprawdza każdą funkcję. Jeśli analiza ucieczki dopisze komunikat, reprezentacja pośrednia znika z wyniku, a drzewo regionów zostaje.

Przy błędzie typu wyniesienie alokacji się nie wykona. Nie zobaczysz drugiego komunikatu o ucieczce obok błędu typu. Przejścia zakładają drzewo, które sprawdzanie typów uważa za spójne. Łączona diagnostyka, w której jeden plik pokazuje naraz błąd typu i błąd ucieczki, w tej wersji nie powstaje.

Wyniesienie alokacji rozpoznaje dokładnie dwie sąsiednie instrukcje

Plik `src/hoist.rs` rozwiązuje następujący układ. Programista deklaruje zmienną wewnętrzną, a w następnym wierszu przenosi ją do zmiennej, która żyje dalej. Bajty i tak mają wylądować w arenie tej dalszej zmiennej. Kompilator może więc zbudować je od razu tam, zamiast budować je w bieżącym buforze i zaraz kopiować.

Funkcja `try_hoist` patrzy na instrukcję o numerze `i` oraz na następną. Pierwsza musi być deklaracją zmiennej o nazwie wewnętrznej, jeszcze bez wskazania miejsca alokacji. Druga musi być przypisaniem do nazwy, a nie do indeksu tablicy. Wartość deklaracji

musi być literałem napisowym albo wywołaniem. Prawa strona przypisania musi być nazwą wewnętrzną użytą jako przeniesienie. Nazwa celu musi być już w zakresie: wśród parametrów, wśród nazw z zewnątrz albo wśród nazw zadeklarowanych wcześniej w tym bloku.

Gdy te warunki są spełnione, pole `alloc_in_binding` dostaje nazwę celu. Generator kodu czyta to pole przy emisji deklaracji i każe alokować wynik w arenie tej nazwy. Jeśli pole jest puste, inicjalizator idzie do bieżącej areny, a późniejsze przeniesienie kopiuje bajty funkcją `copy_into_arena`.

Zbiór nazw zewnętrznych, przy zejściu w blok, w pętlę `for` i w pętlę `while`, jest sumą nazw widzianych do tej pory. Wyniesienie nie przeskakuje między gałęziami warunku. Nie patrzy, która instrukcja dominuje nad inną w grafie sterowania. Albo dwie sąsiednie linie pasują do wzorca, albo alokacja zostaje w bieżącym regionie.

Analiza ucieczki liczy głębokość bajtów

Plik `src/escape.rs` pyta, na jakiej głębokości leżą bajty i czy odbiorca leży nie głębiej niż one. Tylko wtedy bajty jeszcze istnieją w chwili użycia. Głębokość zero to region funkcji. Każdy region wewnętrzny jest o jeden głębszy.

Wiązanie pamięta dwie liczby. `decl_depth` to głębokość deklaracji nazwy. `value_depth` to głębokość bajtów. Druga liczba potrafi być mniejsza od pierwszej, gdy wyniesienie alokacji albo miejsce przeznaczenia położyło bajty wyżej, czyli bliżej funkcji. Samo przejście nie zapisuje tej poprawki z powrotem do analizy własności. Albo odrzuca program, albo go przepuszcza.

Funkcja `place` wylicza głębokość wyrażenia. Literał liczbowy, logiczny, napisowy i słowo `None` mają głębokość zero, bo nie zależą od bufora wewnętrznego regionu. Literał tablicy bierze maksimum z głębokości elementów i z głębokości bufora tablicy. Bufor idzie do miejsca przeznaczenia, a gdy go nie ma, do bieżącej głębokości. Indeks wartości trzymanej w arenie ma głębokość tablicy, a nie głębokość bloku, w którym stoi indeks. Wycinek ma głębokość odbiorcy. Przeniesienie i promocja przy znanym miejscu przeznaczenia mają głębokość tego miejsca. Przy powrocie z funkcji, gdzie miejsce przeznaczenia ma głębokość zero, zostaje głębokość źródła. Wynik `concat` ma głębokość miejsca przeznaczenia albo głębokość bieżącą. Inne wywołanie typu trzymanego w arenie bierze maksimum głębokości argumentów. Warunek bierze maksimum obu gałęzi.

Powrót wartości trzymanej w arenie, gdy jej głębokość jest większa od zera, jest błędem. Potem funkcja `check_return_string_form` osobno zabrania zwrócić wynik `concat` oraz wartość przeniesioną albo promowaną, nawet gdy wyliczona głębokość wyszła zero. Dlatego `return concat(...)` pada także w funkcji bez zagnieżdżonego bloku. Nie ma jeszcze areny wyniku po stronie wywołującego, więc świeży napis nie ma gdzie przeżyć powrotu.

Przypisanie liczy głębokość z miejscem przeznaczenia równym głębokości deklaracji celu. Jeśli bajty byłyby głębsze niż deklaracja celu, komunikat mówi, że przypisujesz wartość, której bajty żyją w regionie wewnętrznym.

Gałąź warunku otwarta jako region, który produkuje wartość, bez miejsca przeznaczenia,

z bajtami na głębokości tej gałęzi, daje błąd. Tekst komunikatu mówi o napisie w gałęzi. Warunek w kodzie jest szerszy: obejmuje każdy typ, dla którego `uses_arena_storage` jest prawdziwe, a więc także tablicę. Słowo w komunikacie jest węższe niż sprawdzany warunek.

Komunikaty analizy ucieczki mają fazę `ownership`, tę samą co analiza własności. Część z nich nie ma zakresu źródłowego. Dotyczy to między innymi zwrotu wyniku `concat`. Wiersz poleceń pomija wtedy numer linii i kolumny.

Jeden spacer trzyma oznaczenie regionów i generator kodu w zgodzie

Plik `src/region_walk.rs` istnieje po to, żeby oznaczanie regionów, układ wywołań w generatorze kodu i emisja instrukcji LLVM nie miały trzech lekko różnych pętli po reprezentacji pośredniej. Komentarz na górze pliku mówi, że jest jeden spacer zsynchronizowany z dziećmi węzła regionu.

Miejsca, które otwierają dziecko raportu, nazywa typ `RegionSite`. Są to blok, pętla `for`, pętla `while`, gałęzie warunku i funkcja dopisana na końcu wywołania. Etykiety muszą pasować do tych, które wpisała analiza własności. Inaczej funkcja `take_child` nie znajdzie węzła i kompilator zgłosi wewnętrzną niezgodność harmonogramu regionów.

Gość spaceru, w kodzie typ `RegionVisitor`, to obiekt, który odwiedza węzły i w wybranych miejscach wykonuje własną czynność. Ma osobne punkty na wejście w funkcję, na wejście w region, na zatrząsk pętli, na wyjście z regionu, na pominięcie funkcji dopisanej na końcu i na chwilę po wyrażeniu. Domyślna ścieżka instrukcji nie obsługuje bloku i pętli. Te idą przez region. Jeśli ktoś wywoła je jak zwykłą instrukcję, dostanie `unreachable!`. To jest kontrola programisty kompilatora, a nie komunikat dla autora programu w Borku.

Koniunkcja i alternatywa zwierają się. Prawa strona jest w osobnym bloku podstawowym, więc emisja nie schodzi w nią zwykłą ścieżką. Oznaczenie regionów i tak musi wiedzieć, czy prawa strona alokuje. Dlatego ogląda prawą stronę, a emisja jej nie emituje drugi raz.

Który region pobiera bufor

Funkcja `codegen_push_for_region` decyduje o wywołaniu `bork_arena_push` w wygenerowanym kodzie. Etykieta funkcji dopisanej na końcu wywołania nigdy nie pobiera bufora. Dla pozostałych regionów decyduje predykat `block_may_allocate_sink` na ciele po zdjęciu opakowań.

Predykat jest prawdziwy, gdy w bloku jest literał napisu albo tablicy, przeniesienie lub promocja typu trzymanego w arenie, wywołanie `concat`, przypisanie albo powrót czegoś, co może alokować, zagnieżdżony blok albo gałąź, która może alokować. Jest fałszywy dla samych liczb, dla `break`, dla `continue` i dla słowa `None`.

Oznaczanie ustawia korzeniowi funkcji flagę pobrania bufora zawsze. Dziecku ustawia wynik predykatu. Funkcji dopisanej na końcu wywołania ustawia tę flagę na fałsz wprost.

Skutek, opisany w docs/memory-model.md i zrealizowany w src/codegen/regions.rs, jest taki. Wydruk drzewa nadal ma węzeł na każdy region, ale wygenerowany kod woła `bork_arena_push` tylko wtedy, gdy flaga jest prawdziwa. Blok, w którym są same liczby i32, ma węzeł w wydruku i nie ma własnego bufora w czasie działania. Funkcja zawsze pobiera jeden bufor na wejściu, nawet gdy nic nie alokuje. To jest uproszczenie tej wersji kompilatora. Bufor funkcji i tak wraca na listę wolnych przy wyjściu.

Pętla, jeśli flaga jest prawdziwa, wchodzi raz, na zatrasku czyści wskaźnik bufora bez oddawania go do puli i wychodzi raz. Plik TODO.md prosi, żeby ten kontrakt nie rozszedł się między gośćmi spaceru. Dziś emisja i oznaczanie dzielą `region_walk`, więc utrzymanie zgodności jest prostsze niż przy dwóch ręcznych pętlach. Nadal da się ją zepsuć, nadpisując hak pętli i zapominając o zatrasku.

Podsumowanie

- Wyniesienie alokacji, oznaczenie buforów i analiza ucieczki biegną tylko po czystym sprawdzeniu typów i własności.
- Wyniesienie rozpoznaje dokładnie dwie sąsiednie instrukcje i każe zbudować napis od razu w arenie celu.
- Analiza ucieczki liczy głębokość bajtów. Zabrania zwrócić świeży napis, nawet z głębokości zero, gdy formą wyniku jest `concat`, `move` albo `promote`.
- Wspólny spacer trzyma oznaczenie regionów i generator kodu przy tych samych dzieciach drzewa regionów.
- Flaga pobrania bufora odcina puste regiony od `bork_arena_push`. Funkcje dopisane na końcu wywołania nie pobierają bufora, bo i tak nie dochodzą do emisji.

Rozdział 17. Jak powstaje kod maszynowy

Ten rozdział obejmuje

- co kontrola przed generowaniem kodu przepuszcza, a co odrzuca komunikatem
- jak wygląda moduł LLVM i jak nazywają się funkcje
- jak emisja traktuje liczby, napisy, tablice i regiony
- jakie funkcje ma biblioteka wykonawcza i jak program jest z nią konsolidowany
- które programy przechodzą kontrolę, a potem wywracają kompilator

Kontrola przed generowaniem kodu odrzuca konstrukcje, których emisja nie zna

Zanim powstanie moduł LLVM, funkcja `gate` w pliku `src/codegen/gate.rs` schodzi po reprezentacji pośredniej i zbiera komunikaty fazy `codegen`. Nie próbuje przetłumaczyć programu częściowo. Pierwsze trafienie zostaje komunikatem. Reszta konstrukcji i tak jest odwiedzana, więc jeden plik może dostać kilka komunikatów o braku wsparcia.

Bez własnego komunikatu tej kontroli przechodzą literały, nazwy, tablice, indeks, wycinek, arytmetyka całkowita, porównania liczb całkowitych, koniunkcja, alternatywa, zakres, negacja, wywołanie bez funkcji dopisanej na końcu, warunek oraz pole `length` na typie utrzymanym w `arenie`.

Odrzucane są między innymi funkcja dopisana na końcu wywołania, słowa `None` i `Some`, operator `?:`, wykrzykniki `!!` oraz pole inne niż `length`. Dokładne teksty są w rozdziale 10. Ta kontrola nie wie o awarii, która powstaje później, gdy argumentem wywołania jest napis. Taki program przechodzi kontrolę i wywraca się w emisji. Plik `TOD0.md` prosi, żeby kontrola była zsynchronizowana z tym, co emisja umie, albo żeby istniał test mówiący, które programy poprawne dla sprawdzania są świadomie odrzucane. Listy dla napisu przekazanego do funkcji użytkownika nie ma.

Moduł wymaga funkcji `main` i deklaruje pozostałe funkcje wcześniej

Funkcja `emit_module` w `src/codegen/llvm/mod.rs` wymaga funkcji o nazwie `main`. Potem tworzy moduł LLVM o nazwie `bork` i budowniczego instrukcji. Deklaruje każdą funkcję

z reprezentacji pośredniej, zanim wyemituje którekolwiek ciało. Dzięki temu wywołanie funkcji zdefiniowanej niżej w pliku ma już symbol. Następnie emiter regionów dostaje drzewo regionów i deklaracje wywołań biblioteki wykonawczej. Potem emitowane są ciała. Na końcu emiter regionów kończy pracę, a moduł jest weryfikowany.

Symbol `main` jest funkcją `main` w konwencji C. W LLVM ma typ `i32` bez parametrów. Wynik `unit` i tak zwraca stałe zero. Inna funkcja nazywa się `bork`. z dopisaną nazwą z programu i ma powiązanie wewnętrzne. Nie jest eksportowana.

Funkcja `main`, która zwraca coś poza `i32` i `unit`, odpada komunikatem, że taki wynik nie jest jeszcze obsługiwany przez generowanie kodu. Sprawdziłem to dla `i64`, `f64` i `bool`. Parametry funkcji `main` są odrzucane już przy deklaracji.

Liczby, napisy i miejsce przeznaczenia

Liczba całkowita jest wartością LLVM o odpowiadającej szerokości. Wartość `bool` jest liczbą całkowitą. Liczba zmiennoprzecinkowa, tam gdzie emisja w ogóle dojdzie do instrukcji arytmetycznej, używa operacji zmiennoprzecinkowych LLVM. Dodawanie `f32` po spacerze regionów zwraca błąd: wewnętrzna niezgodność harmonogramu, a w treści informacja, że dodawanie zmiennoprzecinkowe po spacerze nie jest jeszcze obsługiwane. Porównanie `f64` wpada w funkcję, która oczekuje liczby całkowitej, i kompilator kończy się awarią. Arytmetyka `i64` w funkcji pomocniczej, porównana potem w `main`, działa. Program z odejmowaniem dwóch wartości `i64` i progiem w warunku kończy się kodem 7. Ograniczenie liczb zmiennoprzecinkowych nie jest ograniczeniem szerokich liczb całkowitych.

Napis i tablica są strukturą z wskaźnika i długości typu `i64`. Literał napisu jest prywatną stałą globalną, a w wartości ładuje stały deskryptor. Pole `length` w języku ma typ `i32`. W deskrypcorze długość jest typu `i64`.

Lokalny slot to alokacja na stosie funkcji, instrukcja `alloca`, w bloku wejścia. Dla typu trzymanego w arenie slot pamięta identyfikator regionu, w którym bajty powinny żyć. Na czas emisji wyniku przypisania, powrotu i wywołania `concat` emiter ustawia miejsce przeznaczenia alokacji. Na czas argumentów to miejsce czyści. Dzięki temu bajty wyniku idą do areny odbiorcy, a podwyrażenia nie alokują się wszystkie w tym samym miejscu. Rozdział 9 opisał tę regułę od strony języka.

Przeniesienie i promocja deskryptora wołają kopiowanie do areny celu. Dla tablicy jest osobna funkcja kopiująca. Wyniesienie alokacji sprawia, że kopii czasem nie trzeba, bo bajty już powstały we właściwej arenie. Osobnego przejścia, które usuwałoby zbędne kopiowanie pamięci, nie ma. Model pamięci wymienia je jako rzecz do zrobienia.

Regiony w wygenerowanym kodzie są stosem uchwytów

Emiter regionów tłumaczy wejście, czyszczenie i wyjście na wywołania biblioteki wykonawczej. Wejście woła `bork_arena_push` i kładzie uchwyt na stos. Czyszczenie woła `bork_arena_reset`: wskaźnik w buforze wraca do zera, a bufor zostaje. Wyjście woła `bork_arena_pop` i zwraca bufor do puli. Na martwym punkcie wstawienia, na przykład po instrukcji powrotu, czyszczenie i zdjęcie są pomijane.

Powrót z funkcji zdejmuję uchwyty, które funkcja jeszcze trzyma. Historia poprawek w repozytorium dotyczyła podwójnego zdjęcia albo zostawienia uchwytu. Testy generowania kodu liczą pary wejść i wyjść. Gdy ruszasz instrukcję `return`, uruchom te testy, nie tylko gotową binarkę.

Alokacja użytkowa to `bork_arena_alloc`. Bierze uchwyt, rozmiar i wyrównanie. Wyrównanie musi być potęgą dwójki. Biblioteka wykonawcza przerywa program, gdy koniec alokacji przekroczy 4096 bajtów.

Co emisja naprawdę tłumaczy

Pliki `src/codegen/llvm/expr.rs` i `src/codegen/llvm/array/emit.rs` zawierają tłumaczenie wyrażeń. Następujące zachowania zostały sprawdzone zbudowanym programem.

Arytmetyka i32 i i64 oraz porównania liczb całkowitych działają. Dzielenie ma strażnika, który woła `abort`, gdy dzielnik jest zerem, a także przy skrajnym przypadku minimalnej liczby całkowitej dzielonej przez minus jeden. Warunek jest albo wartością scaloną z dwóch gałęzi, albo sterowaniem ze skokiem. Pętle `for` i `while` obsługują `break` i `continue`. Koniunkcja i alternatywa zwierają się. `println` i `print` wypisują liczby, literały napisowe i napisy lokalne w `main`. `concat` dwóch napisów, które da się załadować jako struktury w `main`, buduje jeden bufor w miejscu przeznaczenia i kopiuje do niego oba argumenty. Nie ma osobnej areny tymczasowej na czas zwykłego wywołania. Literał tablicy, indeks ze strażnikiem, wycinek jako przesunięcie wskaźnika plus nowy deskryptor oraz pole `length` działają. Przypisanie elementu tablicy, w tym elementu napisowego przez `move` w `main`, też działa.

Wywołanie funkcji użytkownika idzie albo przez emisję, która najpierw liczy argumenty, albo przez ścieżkę po spacerze, która dostaje gotowe wartości. Ta druga ścieżka woła `coerce_value_to_ty`. Dla `bool` zwręza wartość do liczby całkowitej. Dla liczby zmiennoprzecinkowej zwraca błąd, że argument wywołania po spacerze nie jest obsługiwany. Dla reszty woła `value_as_int`. Ta funkcja zakłada, że wartość LLVM jest liczbą całkowitą, i woła `into_int_value` bez sprawdzenia wariantu. Struktura napisu powoduje awarię kompilatora. W tej rewizji jest to linia 901 w `src/codegen/llvm/expr.rs`. Nie traktuję śladu biblioteki Inkwell jako kontraktu języka. Traktuję go jako objaw tej luki.

Wywołanie, którego wywoływaną rzeczą nie jest nazwa, daje komunikat, że wywołania pośrednie nie są jeszcze obsługiwane, o ile ścieżka w ogóle tam wejdzie. Funkcja dopisana na końcu wywołania zwykle odpada wcześniej, w kontroli przed generowaniem kodu.

Biblioteka wykonawcza trzyma pulę buforów

Plik `crates/bork_runtime/src/lib.rs` trzyma pulę w muteksie, czyli w blokadzie, którą inicjuje się raz. Pobranie zdejmuję bufor z wektora albo alokuje nowy. Zwrot czyści wskaźnik i odkłada bufor. Regiony żywe w tym samym czasie dostają różne bufory. Regiony sąsiednie w czasie mogą dostać ten sam.

Funkcje widoczne z C są następujące. `bork_arena_push` zwraca wskaźnik na bufor z puli. `bork_arena_reset` zeruje przesunięcie i zostawia bufor. `bork_arena_pop`

zwraca bufor do puli. `bork_arena_alloc` przesuwa wskaźnik i zwraca adres. `bork_print_i64` i `bork_println_i64` piszą liczbę na standardowe wyjście i opróżniają bufor. `bork_print_str` i `bork_println_str` piszą napis podany wskaźnikiem i długością.

Nie ma symbolu `bork_abort`. Generator kodu deklaruje `abort` z biblioteki C.

Testy biblioteki wykonawczej sprawdzają wyrównanie i przepełnienie. Są w tym samym pliku, pod warunkiem kompilacji testów, i wchodzi w `cargo test --workspace` nawet bez opcji `codegen` kompilatora. Archiwum do konsolidacji powstaje dopiero przy tej opcji, w pliku `build.rs`.

Konsolidacja używa clang i statycznego archiwum

Plik `src/codegen/link.rs` woła `clang`. W uproszczeniu polecenie łączy plik obiektowy z archiwum `libbork_runtime.a` oraz z bibliotekami `gcc_s`, `util`, `rt`, `pthread`, `m` i `dl`. Ścieżka archiwum pochodzi ze zmiennej środowiskowej `BORK_RUNTIME_LIB` albo z wartości wkompileowanej pod tą samą nazwą. Plik `build.rs` ustawia ścieżkę wyszukiwania bibliotek na katalog, w którym leży sama binarka `bork`, żeby `libLLVM` mogło leżeć obok niej. Skrypt `scripts/bundle-llvm.sh` kopiuje używany plik `libLLVM`.

Maszyna, która tylko uruchamia skompilowany program w Borku, nie potrzebuje LLVM. Maszyna, która uruchamia `bork build`, potrzebuje `clang`. Jeśli biblioteka LLVM nie została spakowana obok binarki, potrzebuje też instalacji LLVM 23.

Co zostało uruchomione przy pisaniu tej książki

Kompilator złożony z opcją `codegen`, na LLVM 23.1.2, zbudował i uruchomił między innymi zwrot 42 z dodawania, sumę pętli równą 10, pętlę `while` z `break` o wyniku 3, pętlę z `continue` o sumie 8, wypisanie `hi`, współdzielenie napisu, przeniesienie napisu dające `ab`, konkatencję dającą `LR`, promocję dającą `temp`, przypisanie napisu, sekwencje ucieczki, dzielenie $8/2$ z kodem 4, tablicę z wycinkiem i kodem 12, wycinek drukujący 20 i 2, indeks poza zakresem oraz dzielenie przez zero jako przerwanie procesu, a także brak funkcji `main` jako komunikat. Wyniki są powtórzone w `docs/book/przykłady/WYNIKI.md`.

Podsumowanie

- Kontrola przed generowaniem kodu odcina wartości puste, operator `?:`, wykrzykniki `!!` i funkcję dopisaną na końcu wywołania zwykłym komunikatem.
- Nie odcina napisu jako argumentu funkcji użytkownika. Emisja wtedy kończy się awarią kompilatora.
- `main` to funkcja `main` w konwencji C, zwracająca `i32`. Pozostałe funkcje nazywają się `bork` z kropką i nazwą.
- Regiony w LLVM to stos uchwytów do biblioteki wykonawczej. Pętla czyści bufor, a nie oddaje go do puli przy każdym obrocie.
- Napis i tablica to struktura ze wskaźnika i długości. Literał napisu leży w stałej globalnej.
- Program jest konsolidowany przez `clang` ze statyczną biblioteką wykonawczą. Sam kompilator ładuje `libLLVM` w wersji 23.

Rozdział 18. Komunikaty błędów, polecenia i edytor

Ten rozdział obejmuje

- jedną strukturę komunikatu dla wiersza poleceń i dla edytora
- co robią polecenia `bork` i `bork build`
- co naprawdę ogłasza serwer językowy
- co pokazuje rozszerzenie Cursora i Visual Studio Code
- czego odpowiedź pod kursorem nie zawiera

Jeden komunikat ma fazę, treść i opcjonalny zakres

Plik `src/diag.rs` definiuje strukturę `Diagnostic`. Są cztery fazy: czytanie składni, własność, typ i generowanie kodu. Są dwa poziomy: błąd i ostrzeżenie. Jest treść i opcjonalny zakres źródłowy. Nie ma kodów numerycznych, nie ma linii zaczynających się od `help:` i nie ma linii zaczynających się od `note:`. Jeden komunikat niesie jedną myśl. Gdy kompilator chce powiedzieć dwie rzeczy, dostajesz dwa komunikaty. Program z pętlą i przeniesieniem z rozdziału 2 jest takim przypadkiem.

Czytanie składni zamienia błąd parsera funkcją `from_parse` i dostaje fazę `parse`. Błędy analizy własności dostają fazę `ownership` przez `from_sema`. Błędy sprawdzania typów powstają w środowisku sprawdzania i mają fazę `type`. Analiza ucieczki też dostaje fazę `ownership`. Kontrola przed generowaniem kodu i błędy emisji mają fazę `codegen`.

Serwer edytora nie uruchamia `bork build`. Fazy generowania kodu w edytorze nie zobaczysz, dopóki ktoś nie wstawi tej kontroli do `frontend::check`. Dziś kontrola jest tylko w `codegen::build`. Program ze słowem `None` w edytorze wygląda na poprawny, a `bork build go` odrzuca. Zielone podkreślenie w edytorze nie znaczy, że powstanie plik wykonywalny.

Dwa polecenia i kody zakończenia

Plik `src/main.rs` przyjmuje dwie formy. Pierwsza to `bork`, opcjonalnie z `--dump-arenas`, i ścieżka pliku. Druga to `bork build`, opcjonalnie z `-o` i ścieżką wyjścia, oraz ścieżka pliku.

Wydruk drzewa regionów pojawia się na standardowym wyjściu tylko wtedy, gdy pole

raportu w wyniku sprawdzenia jest obecne. Przy błędzie składni jest cicho, poza komunikatami na standardowym wyjściu błędów. Przy błędzie typu drzewo zostaje. Prośba o pomoc kończy się kodem zero. Nieznany znacznik kończy się kodem dwa.

Polecenie build bez opcji kompilacji codegen nie próbuje wołać LLVM. Mówi, że ta opcja jest wymagana. Z opcją najpierw sprawdza program, potem buduje. Sukces kończy się kodem zero, także gdy program tylko wypisuje tekst i funkcja main nie ma zadeklarowanego wyniku. Komunikaty programu kończą się kodem jeden. Problem z narzędziami, na przykład brak clang, kończy się kodem dwa. Awaria samego kompilatora, czyli panika Rusta, kończy się kodem 101. Przerwanie procesu użytkownika przez abort daje w powłoce kod 134.

Nakładanie ścieżki wejścia i wyjścia jest sprawdzane najpierw porównaniem ścieżek, a potem kanonizacją, gdy oba pliki istnieją. Nie jest to pełne rozwiązanie dowiązań symbolicznych w każdą stronę. Wystarczy, żeby nie nadpisać źródła domyślnym wyjściem wtedy, gdy to naprawdę ten sam plik. Domyślne wyjście ma obcięte rozszerzenie, więc a.bork staje się a. Konflikt trzeba wymusić ręcznie, podając -o a.bork przy wejściu a.bork.

Serwer liczy sprawdzenie od zera przy każdej zmianie

Pliki src/bin/bork_lsp.rs i src/lsp.rs składają serwer, gdy włączona jest opcja lsp. Transport idzie przez standardowe wejście i wyjście. Biblioteka to tower-lsp.

Ogłaszane możliwości to synchronizacja pełnym tekstem dokumentu, odpowiedź pod kursorem oraz polecenie bork.dumpArenas. Nie ma uzupełniania, przechodzenia do definicji, wyszukiwania referencji, formatowania, listy symboli ani szybkich poprawek. Powiadomienie o zmianie bierze ostatnią wersję tekstu i liczy analizę od zera. Nie ma parsera przyrostowego.

Analiza woła lsp::analyzer::source, a ta woła frontend::check. To ta sama kontrola co w wierszu poleceń. Specyfikacja rozszerzenia z 21 września 2026 opisywała komunikaty samego parsera. To jest nieaktualne. Plik README w jednym nagłówku nadal mówi o diagnostyce składni, a trzy zdania niżej opisuje pełne sprawdzenie. Wierz funkcji analyzer::source, a nie nagłówkowi.

Komunikat w edytorze ma źródło bork i treść złożoną z fazy oraz wiadomości. Pozycje są w jednostkach UTF-16, zgodnie z protokołem. Pusty zakres, w którym początek równa się końcowi, jest rozszerzany o jedną jednostkę, żeby podkreślenie było widoczne.

Licznik epoki na adresie dokumentu odrzuca spóźnione publikowanie komunikatów, gdy użytkownik zdążył wpisać kolejną wersję. To jest jedyna współbieżność, o którą serwer dba. Nie ma puli procesów roboczych.

Podpowiedź pokazuje region i własność, a nie typ

Podpowiedź pod kursorem szuka słowa złożonego ze znaków ASCII, cyfr i podkreślenia. Trafienie porównuje z zakresem wiązania w drzewie regionów. Tekst w Markdownie mówi, przy jakiej etykietce regionu leży nazwa, i jaki jest rodzaj własności. Nie ma typu z reprezentacji pośredniej. Specyfikacja zostawiła typ na podpowiedzi jako rzecz

do zrobienia. Commit, który podpiął serwer do pełnego sprawdzenia, też traktował typ jako opcjonalny. Nadal go nie ma.

Słowo arena w tym zdaniu podpowiedzi oznacza etykietę regionu z drzewa analizy własności. Nie oznacza bufora 4096 bajtów. Rozdział 2 rozdziela te dwa znaczenia. W tekście podpowiedzi angielskie słowo arena zostaje, bo tak formatuje je serwer.

Polecenie `bork.dumpArenas` bierze opcjonalny adres dokumentu albo pierwszy otwarty dokument. Wynik to JSON z polem `dump` oraz komunikat w oknie dziennika. Przy błędzie składni JSON ma pole `error`. Wydruk przy błędach znaczenia dopisuje linie zaczynające się od `# semantic errors`. To ten sam tekst co `--dump-arenas`, plus ten dopisek.

Rozszerzenie uruchamia serwer z katalogu projektu

Katalog `tools/bork-lsp-extension` aktywuje się dla języka bork i dla plików o rozszerzeniu `.bork`. Gramatyka kolorowania jest w `syntaxes/bork.tmLanguage.json`. Konfiguracja nawiasów i komentarza `//` jest w `language-configuration.json`.

Plik `server.js` uruchamia `target/debug/bork-lsp` z katalogu otwartego projektu. Gdy tej binarki nie ma, woła `cargo run --quiet --bin bork-lsp`. Bez otwartego folderu rozszerzenie nie startuje serwera. Polecenie w interfejsie nazywa się `bork.dumpArenasView` i woła polecenie serwera `bork.dumpArenas`. Test `test/server.test.js` pilnuje tej różnicy nazw oraz wyboru binarki.

W pliku `package.json` nie ma ustawień. Nie ma adaptera debuggera. Kolorowanie składni nie wie o błędach typów. Od tego są komunikaty serwera.

Podsumowanie

- Jedna struktura komunikatu zasila wiersz poleceń i edytor. Generowanie kodu do edytora nie dochodzi.
- Plik bez podkreśleń w edytorze może nadal paść przy `bork build`, albo komunikatem kontroli, albo awarią kompilatora.
- Podpowiedź pod kursorem pokazuje etykietę regionu i własność, a nie typ.
- Wydruk w edytorze i znacznik `--dump-arenas` dzielą funkcję `dump_arenas`.
- Serwer liczy sprawdzenie od zera przy każdej zmianie i wyrzuca spóźnione odpowiedzi licznikiem epoki dokumentu.

Rozdział 19. Jak czytać repozytorium

Ten rozdział obejmuje

- które katalogi mają znaczenie przy czytaniu kompilatora
- za co odpowiada każdy większy plik
- w jakiej kolejności rosły warstwy, które widać w kodzie
- od czego zacząć, gdy chcesz zrozumieć drogę pliku źródłowego
- które dokumenty wyglądają na opis bieżącego kodu, a opisują wcześniejszy etap

Katalogi, które mają znaczenie

W korzeniu plik `Cargo.toml` opisuje przestrzeń roboczą: skrzynkę `bork` i skrzynkę `crates/bork_runtime`. Plik `build.rs` uruchamia generowanie parsera. Przy opcji `codegen` buduje też statyczną bibliotekę wykonawczą. Plik `src/lib.rs` wystawia funkcję `parse`, eksporty modułów i próbki używane w testach. Plik `src/main.rs` jest wierszem poleceń. Plik `src/bin/bork_lsp.rs` jest binarką serwera. Gramatyka jest w `src/parser.lalrpop`. Drzewo składni jest w `src/ast.rs`. Zamiana nowych linii w nawiasach jest w `src/layout.rs`. Zakres źródłowy jest w `src/span.rs`. Komunikat jest w `src/diag.rs`. Sklejenie faz jest w `src/frontend.rs`.

Sprawdzanie typów leży w `src/typeck/`. Reprezentacja pośrednia leży w `src/hir/`. Analiza własności leży w `src/sema/`. Analiza ucieczki jest w `src/escape.rs`. Wyniesienie alokacji jest w `src/hoist.rs`. Wspólny spacer regionów jest w `src/region_walk.rs`. Wydruk drzewa jest w `src/dump.rs`. Funkcje wbudowane są w `src/builtins.rs`. Model bufora 4096 bajtów, nieużywany przez analizę własności, jest w `src/arena.rs`. Adapter protokołu edytora jest w `src/lsp.rs`. Generowanie kodu jest w `src/codegen/`. Biblioteka wykonawcza jest w `crates/bork_runtime/`.

Testy czytania składni są w `tests/parser.rs`. Testy tablic przez samo sprawdzenie są w `tests/arrays.rs`. Testy budowania i uruchamiania binarki są w `tests/build.rs`. Klient edytora jest w `tools/bork-lsp-extension/`. Opis języka jest w `docs/language.md`. Model pamięci jest w `docs/memory-model.md`. Starsze specyfikacje i plany są w `docs/superpowers/`. Lista znanych braków jest w `TOD0.md`. Skrypt kopiujący bibliotekę LLVM obok binarki `bork` jest w `scripts/bundle-llvm.sh`.

Moduły `escape`, `region_walk`, `layout` i `builtins` są prywatne. Widać je wewnątrz skrzynki, a nie w publicznym API. Z zewnątrz wchodzisz przez `parse`, `frontend`, `sema`,

hir, typeck, dump, diag, lsp, codegen, arena, span i ast.

Za co odpowiada każdy moduł

Moduł ast trzyma nietypowane drzewo po parserze. Moduł layout zamienia znak nowej linii na znak powrotu karetki wewnątrz nawiasów i nie zmienia długości pliku. Moduł span trzyma parę przesunięć bajtowych. Moduł diag trzyma fazę, treść i zakres. Moduł frontend skleja fazy w wynik sprawdzenia. Moduł typeck produkuje reprezentację pośrednią i błędy typu. Moduł hir trzyma typowane drzewo bez numerów regionów. Moduł sema buduje drzewo regionów i błędy nazw. Plik `escape.rs` odrzuca bajty, które byłyby użyte po zwolnieniu ich bufora. Plik `hoist.rs` rozpoznaje dwie sąsiednie instrukcje i ustawia miejsce alokacji. Plik `region_walk.rs` jest jednym spacerem reprezentacji pośredniej razem z raportem regionów. Plik `dump.rs` zamienia drzewo na tekst. Plik `builtins.rs` deklaruje `print`, `println` i `concat`. Plik `arena.rs` opisuje bufor i pulę jako model w kompilatorze. Katalog `codegen` ma kontrolę przed generowaniem kodu, emisję LLVM i konsolidację. Katalog `codegen/llvm` tłumaczy funkcje, wyrażenia, tablice i uchwyt regionów przez bibliotekę `Inkwell`. Moduł `lsp` liczy pozycje UTF-16, podpowiedź i wydruk. Skrzynka `bork_runtime` trzyma bufor i wypisywanie dla zlinkowanego programu.

Jak rosły warstwy widoczne w kodzie

Kolejność na gałęzi `main`, skrócona do rzeczy, które widać w źródłach, jest następująca. Najpierw był parser i drzewo składni, z próbką funkcji dopisanej na końcu wywołania. Serwer edytora najpierw wołał samo czytanie składni. Dziś woła pełne sprawdzenie. Potem doszła składnia wartości pustych, napisów oraz słów `Some` i `None`. Potem analiza regionów, wydruk drzewa i podpowiedź własności. Potem wyrażeniowe `move` oraz `val` i `var` na parametrach. Potem typowana reprezentacja pośrednia i funkcja `frontend::check` z fazą w komunikacie. Potem LLVM, polecenie `bork build`, biblioteka wykonawcza i kontrola przed generowaniem kodu. Potem miejsce przeznaczenia alokacji, wyniesienie, słowo `promote`, funkcja `concat` i analiza ucieczki. Potem tablice o długości w typie, wycinki, pętle `while` z `break` i `continue`, koniunkcja i alternatywa oraz wspólny spacer regionów. Na końcu przypięcie LLVM 23 i skrypt pakowania biblioteki.

Specyfikacje w `docs/superpowers/specs/` opisują wczesne kroki i projekt serwera. Nie opisują tablic ani pętli `while`. Plik `TODO.md` jest świeższy i mówi wprost, żeby plany w `docs/superpowers/plans/` przenieść albo usunąć, gdy zostaną wchłonięte. Czytaj je jako historię decyzji. Kontraktem jest kod. Dodatek C wymienia miejsca, w których dokument i kod mówią co innego.

Od czego zacząć czytanie

Gdy celem jest zrozumienie, co dzieje się z plikiem o rozszerzeniu `.bork`, idź w tej kolejności.

Najpierw przeczytaj `docs/language.md` i `docs/memory-model.md`, a potem tę książkę w miejscach, gdzie dokument rozmija się z kompilatorem. Najważniejszy przykład to

concat na dwóch zmiennych napisowych. Potem otwórz `src/lib.rs` i zobacz funkcję `parse` oraz dwie próbki. Próbką pierwszej wersji sprawdza, że parser i analiza własności dają radę. Nie jest testem generowania kodu. Potem przeczytaj `src/frontend.rs`. Cała kolejność faz mieści się tam w kilkudziesięciu liniach.

Drzewo składni w `src/ast.rs` czytaj równolegle z początkiem gramatyki, czyli z produkcjami programu, instrukcji i wyrażenia. Nie czytaj kilkuset linii gramatyki ciągiem, dopóki nie znasz wariantów drzewa. Potem przeczytaj `src/hir/ty.rs`, zwłaszcza `is_copy` i `uses_arena_storage`. Do tych dwóch predykatów wraca reszta kompilatora. Potem `src/sema/policy.rs` i funkcję `classify_use`. Zejście w `src/sema/walk.rs` czytaj najpierw wokół otwarcia zwykłego regionu, wokół przypisania i wokół warunku.

Wyrażenia w `src/typeck/expr/control.rs` i `src/typeck/expr/call.rs` pokazują miejsca, w których składnia najbardziej przypomina Kotliną. Funkcja `place` w `src/escape.rs` i funkcja `try_hoist` w `src/hoist.rs` są krótkie. Spacer w `src/region_walk.rs` zacznij od typu gościa i od stempla pobrania bufora, a nie od makr kursora. Makra `arena_cursor` są mechaniką pożyczania w Ruście. Plik `TOD0.md` sam proponuje je kiedyś zastąpić typem kursora.

Kontrolę przed generowaniem kodu czytaj w `src/codegen/gate.rs`, potem wejście budowania w `src/codegen/mod.rs`, potem `emit_module` w `src/codegen/llvm/mod.rs`, potem emisję funkcji i wyrażeń. Emisję tablic zostaw na koniec, gdy wiesz, jak wygląda deskryptor. Biblioteka wykonawcza w `crates/bork_runtime/src/lib.rs` ma około stu pięćdziesięciu linii i jest całym kodem wykonawczym. Testy w `tests/build.rs` mówią, co naprawdę zwraca proces.

Gdy celem jest tylko język, zatrzymaj się przed generowaniem kodu i czytaj testy analizy własności oraz sprawdzania typów.

Pułapki przy nawigacji

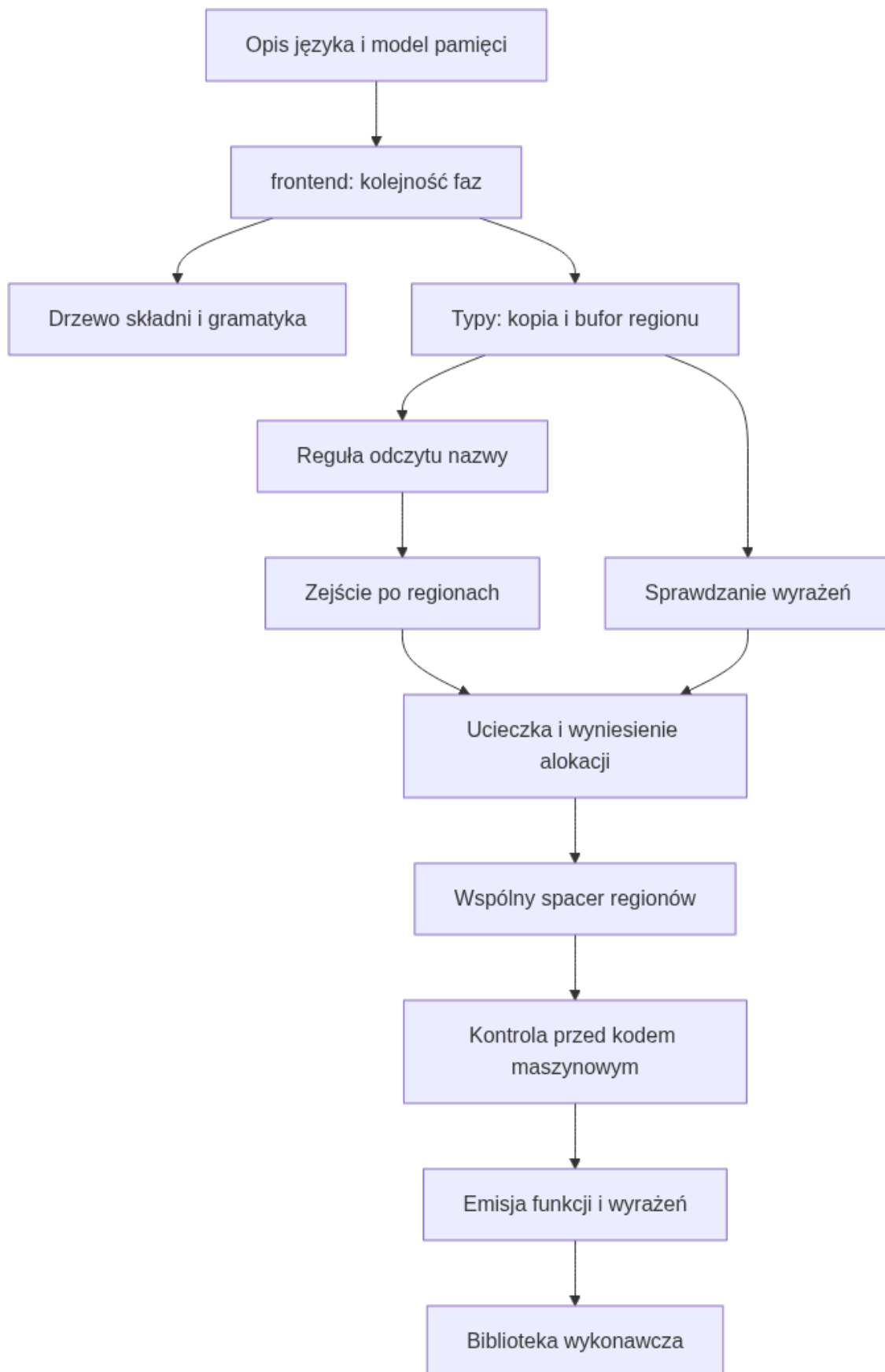
Plik `src/arena.rs` i węzeł `ArenaNode` nie są tym samym obiektem. Pierwszy ma przesunięcie i pojemność 4096 bajtów. Drugi ma dzieci i wiązania w drzewie regionów.

Funkcja `sema::analyzer` i funkcja `frontend::check` nie są wymienne. Pierwsza liczy sprawdzanie typów wewnętrznie jeszcze raz. Druga podaje gotowy wektor typów deklaracji.

Rodzaj użycia w reprezentacji pośredniej i rodzaj własności w raporcie mają podobne nazwy, między innymi kopia, współdzielenie i przeniesienie. Nie są tym samym typem w Ruście.

Opcja `codegen` wyłącza część plików z kompilacji, gdy jest wyłączona. Ostrzeżenia o martwym kodzie na spacerze regionów bez tej opcji są oczekiwane. Spacer jest wtedy używany przez oznaczenie regionów, a część metod tylko przez emisję LLVM.

Plany w `docs/superpowers/plans/` potrafią opisywać LLVM 18. Plik `Cargo.toml` przypina Inkwell z opcją `llvm23-1-force-dynamic`. Wierz manifestowi.



Rysunek 3: Diagram 3

Podsumowanie

- Kolejność faz mieści się w `src/frontend.rs`. Reszta katalogu `src` to poszczególne fazy.
- Regułę odczytu nazwy czytaj przed zejściem po regionach, a predykaty typu przed jednym i przed drugim.
- Specyfikacje w `docs/superpowers` są historią. Plik `TOD0.md` i kod są stanem.
- Plik `src/arena.rs` nie stoi na ścieżce sprawdzenia.
- Testy w `tests/build.rs` definiują to, co robi binarka. Komentarze przy emisji LLVM tego nie zastępują.

Rozdział 20. Jeden program od źródła do uruchomienia

Ten rozdział obejmuje

- krótki program, który został zbudowany i uruchomiony
- co widzi czytanie składni
- co dopisuje sprawdzanie typów
- jakie drzewo regionów wypisuje analiza własności
- dlaczego wyniesienie alokacji i analiza ucieczki w tym programie milczą
- które funkcje wygenerowanego kodu i biblioteki wykonawczej naprawdę pracują

Program jest krótki celowo. Ma funkcję, pętlę, wywołanie, stały napis i blok, który ten napis tylko czyta. Polecenie `bork build` tłumaczy go na plik wykonywalny. Nie ma w nim napisu przekazywanego do funkcji użytkownika, bo ta ścieżka kończy się awarią kompilatora i ślad urwałby się w emisji.

Program, który zwraca trzy i wypisuje sumę

Listing 20.1. Uruchomiony. Standardowe wyjście to `sum` i nowa linia. Kod wyjścia to 3, bo suma `0 + 1 + 2` wynosi 3.

```
fun add(a: i32, b: i32): i32 {
    return a + b
}

fun main(): i32 {
    var total = 0
    for (i in 0..3) {
        total = add(total, i)
    }
    val label = "sum"
    {
        println(label)
    }
    return total
}
```

Zakres `0..3` jest otwarty z prawej strony. Są trzy iteracje, a nie cztery. Indeks przyjmuje

wartości 0, 1 i 2.

Czytanie składni widzi dwie funkcje

W pliku nie ma nawiasu, który łamałby linię, więc funkcja normalizująca nowe linie zwraca brak zmiany i parser dostaje oryginał. Parser buduje dwie funkcje.

Dla `add` drzewo składni ma dwa parametry. Nie napisano przy nich `val`, a goła forma parametru jest stała. Typ obu to `i32`. Ciało ma jeden powrót i dodawanie.

Dla `main` jest deklaracja zmiennej `total` bez adnotacji typu, pętla `for` z zakresem, deklaracja stałej `label`, blok z wywołaniem `println` oraz powrót.

Gdyby w pliku był średnik, czytanie składni zwróciłoby nierozpoznany token. Sprawdzenie skończyłoby się jednym komunikatem fazy `parse`, bez drzewa regionów. Średnika nie ma.

Wejście jest w funkcji `bork::parse` w `src/lib.rs`. Reguły gramatyki to produkcje funkcji, instrukcji, zakresu i atomu.

Sprawdzanie typów widzi liczby i napis

Sprawdzanie typów widzi dwie sygnatury. Funkcja `add` bierze dwie liczby `i32` i zwraca `i32`. Funkcja `main` zwraca `i32`. Nazwa `println` jest wbudowana i nie koliduje.

Zmienna `total` nie ma adnotacji. Inicjalizator `0` bez oczekiwanego typu jest literałem całkowitym, więc dostaje `i32`. Ten typ wpada do wektora typów deklaracji. Stała `label` ma typ `String`. Indeks `i` w pętli dostaje `i32`, bo granice zakresu są typu `i32`. Warunek pętli nie jest osobnym wyrażeniem logicznym. Zakres jest osobnym typem, który istnieje dopiero po sprawdzeniu typów.

Wywołanie `add(total, i)` ma nazwę jako rzecz wywoływana i dwa argumenty typu `i32`. Odczyt `total` w ciele pętli dostaje rodzaj użycia oznaczający kopię, bo region jest inny, a typ jest kopiowalny. Wywołanie `println(label)` jest funkcją wbudowaną. Argument jest napisem. Odczyt `label` dostaje rodzaj użycia oznaczający współdzielenie.

Powrót `total` zgadza się z `i32`. Ścieżka funkcji zawsze wraca. Reprezentacja pośrednia funkcji `add` to jeden powrót z dodawaniem.

Pliki tego kroku to `src/typeck/mod.rs`, `src/typeck/stmt.rs`, `src/typeck/expr/call.rs` i `src/typeck/expr/binary.rs`.

Analiza własności widzi kopię w pętli i współdzielenie w bloku

Analiza dostaje kolejkę typów deklaracji: najpierw `total`, potem `label`. Pamięta, że oba parametry `add` są stałe.

Korzeń `fun add` wiąże `a` i `b` jako nazwy lokalne. Wydruk nie pokazuje osobnych obserwacji kopiowania dla parametrów użytych w tym samym regionie. Reguła odczytu zwraca użycie lokalne, a wydruk pomija lokalne obserwacje przy deklaracji.

Korzeń fun main wygląda tak. Ten tekst pochodzi z uruchomienia ze znacznikiem --dump-arenas, a nie z rekonstrukcji na papierze.

```
Arenas
├── fun add
│   ├── a [Local]
│   └── b [Local]
└── fun main
    ├── total [Local]
    ├── label [Local]
    ├── ForLoop (i)
    │   ├── i [Local]
    │   └── total [Copy]
    └── Block
        └── label [Shared ← fun main]
```

Węzeł pętli powstał przy zejściu w instrukcję for. Indeks i jest parametrem tego regionu. Nazwa total w ciele przechodzi regułę odczytu: inny region i typ kopiowalny, więc obserwacja to kopia. Przypisanie total = ... idzie do zmiennej z regionu zewnętrznego. Cel jest zmienną, a numer jego regionu jest mniejszy, więc lewa strona nie jest błędem użycia zmiennej rodzica. Prawa strona to i32, więc i tak jest kopia.

Blok po pętli otwiera zwykły region o etykiecie Block. Stała label nie jest kopiowalna i żyje w innym regionie, więc obserwacja to współdzielenie z etykietą fun main.

Na czas pętli zbiór zakazu przeniesienia zawiera total. Nikt nie woła move, więc zakaz milczy. Zdjęcie opakowań nic nie skleja, bo żaden blok nie jest gołym opakowaniem drugiego bloku.

Pliki tego kroku to src/sema/analize.rs, src/sema/walk.rs, src/sema/policy.rs, src/sema/region.rs i src/dump.rs.

Oznaczenie buforów, wyniesienie i ucieczka nie mają tu pracy poza korzeniem funkcji

Komunikatów nie ma, więc oznaczanie schodzi wspólnym spacerem. Korzeń każdej funkcji dostaje flagę pobrania bufora.

Ciało pętli to przypisanie liczby i32 i wywołanie add. Ani jedno, ani drugie nie alokuje napisu w miejscu przeznaczenia. Predykat block_may_allocate_sink dla ciała złożonego z samej arytmetyki jest fałszywy. Węzeł pętli zostaje w raporcie, ale nie prosi o bork_arena_push. Wydruk ma węzeł, a w czasie działania pętla nie dostaje własnego bufora.

Blok z println(label) nie tworzy nowego napisu. Czyta wartość współdzieloną. Samo wywołanie println nie jest ani concat, ani literałem napisu. Literał "sum" stoi przy deklaracji label w ciele funkcji, a nie w bloku wewnętrznym. Blok wewnętrzny też nie musi pobierać własnego bufora.

Wyniesienie alokacji nie znajduje pary: deklaracja zmiennej wewnętrznej i zaraz potem przeniesienie do zmiennej zewnętrznej. Pole miejsca alokacji zostaje puste.

Analiza ucieczki widzi powrót liczby `total`, a nie typu trzymanego w buforze. Stała `label` nie wraca z regionu wewnętrznego. Funkcja licząca głębokość nic nie zgłasza.

Reprezentacja pośrednia zostaje w wyniku sprawdzenia.

Kontrola, moduł i wygenerowane wywołania

Kontrola przed generowaniem kodu nie znajduje funkcji dopisanej na końcu wywołania, słów `None` i `Some`, operatora `?:`, wykrzykników `!!` ani obcego pola. Przechodzi.

Emisja modułu deklaruje `main` jako funkcję zwracającą `i32` bez parametrów oraz `bork.add` jako funkcję wewnętrzną. Potem emituje ciała.

Dla `main` otwiera blok wejścia, rezerwuje slot na `total` i slot na deskryptor `label`. Region funkcji pobiera bufor, bo korzeń zawsze ma flagę pobrania. Literał `"sum"` jest stałą globalną. Deskryptor ze wskaźnikiem i długością 3 ładuje w slot. Bufor funkcji w tym programie może zostać nietknięty przez alokację, bo znaki są w stałej globalnej. Pobranie bufora i tak jest.

Pętla trzyma indeks, porównuje go z 3, woła `bork.add` i przypisuje wynik. Zatrask nie musi czyścić bufora pętli, jeśli wejście w pętlę w ogóle nie położyło uchwytu. Spacer i tak woła `hak` zatrasku. Emiter na braku uchwytu tego regionu nie czyści cudzego bufora.

Wypisanie schodzi do `bork_println_str`. Po bloku, jeśli nie było pobrania, nie ma zwrotu. Powrót ładuje `total` i zwraca. Zdjęcie uchwytów zdejmuje uchwyt funkcji.

Funkcja `add` ładuje dwa parametry, dodaje je instrukcją całkowitą i wraca. Napisów w niej nie ma.

Plik obiektowy, konsolidacja i proces

Zapis obiektu bierze natywny cel. Na maszynie, na której książka była sprawdzana, był to Linux x86-64. Ustawia trójkę docelową i układ danych, potem pisze plik obiektowy. Konsolidacja woła `clang` z archiwum biblioteki wykonawczej.

Proces zwraca 3. Wypisanie pisze trzy bajty słowa `sum` i nową linię, potem opróżnia bufor wyjścia. Pula buforów w bibliotece wykonawczej żyje w statycznym muteksie. Po zwrocie bufor funkcji wraca na listę. Proces się kończy, więc pula znika razem z pamięcią procesu.

Czego ten ślad nie pokrywa

Nie było wyniesienia alokacji, promocji, przeniesienia napisu, wycinka, pętli `while`, zwierania ani błędu ucieczki. Każde z nich ma listing w części pierwszej i test w `tests/build.rs` albo w `src/escape.rs`. Ślad miał pokazać udaną drogę przez funkcje graniczne, a nie każdy `hak`.

Gdy będziesz powtarzał ślad na programie, który deklaruje kawałek napisu wynikiem `concat` i w następnym wierszu przenosi go do zmiennej zewnętrznej, zatrzymaj się dłużej

przy wyniesieniu. Pole miejsca alokacji zostanie ustawione, a emisja deklaracji ustawi miejsce przeznaczenia na dom zmiennej zewnętrznej, zanim wyemituje concat. Reszta kroków jest ta sama.

Podsumowanie

- Śledzony program zwraca 3 i drukuje sum. Sprawdzenie i bork build są zgodne.
- Parser widzi dwie funkcje i zakres. Sprawdzanie typów widzi i32 i String.
- Analiza własności oznacza total jako kopię w pętli i label jako współdzielenie w bloku.
- Pętla bez alokacji nie dostaje własnego bufora, choć ma węzeł w wydruku.
- Wyniesienie alokacji i analiza ucieczki nie mają tu nic do zrobienia. To też jest wynik.
- Wygenerowany kod woła bork.add i bork_println_str. Biblioteka wykonawcza dostaje deskryptor literału ze stałej globalnej.

Rozdział 21. Testy, szukanie przyczyny błędu i nowa konstrukcja języka

Ten rozdział obejmuje

- które polecenie uruchamia który zestaw testów
- gdzie szukać, gdy binarka się wywraca, a sprawdzenie milczy
- w jakiej kolejności warstw doszła do języka pętla while
- które braki są już zapisane w TODO.md
- czego nie robić w pierwszej poprawce

Dwa polecenia testów, bo generowanie kodu jest osobną opcją

Z pliku README i z `.github/workflows/ci.yml` wynikają dwa polecenia. Pierwsze to `cargo test --workspace`. Drugie to `cargo test --workspace --features codegen`.

Pierwsze polecenie nie składa LLVM. Łapie parser, analizę własności, sprawdzanie typów, analizę ucieczki przez `frontend::check`, serwer edytora oraz testy jednostkowe puli w bibliotece wykonawczej. Kontrolę przed generowaniem kodu łapie tylko o tyle, o ile test tej kontroli jest za warunkiem opcji `codegen`. Zadanie ciągłej integracji o nazwie `codegen` jest osobne. Instaluje LLVM 23 i clang, puszcza przestrzeń roboczą z tą opcją i jeszcze sprawdza spakowaną bibliotekę LLVM.

Testy w `tests/parser.rs` utwierdzają tokeny, nowe linie, słowo `move`, kolejność operatorów i zakresy błędów. Testy w `src/sema/tests` utwierdzają własność nazw. Testy w `src/typeck/tests` utwierdzają typy, często przez pełne sprawdzenie. Plik `tests/arrays.rs` sprawdza tablice bez uruchamiania binarki. Plik `tests/build.rs` buduje program i sprawdza kod wyjścia albo standardowe wyjście. Plik `src/codegen/gate.rs` trzyma teksty o braku wsparcia. Plik `src/lsp.rs` sprawdza pozycje, podpowiedź i wydruk. Skrzynka `bork_runtime` sprawdza przesunięcie wskaźnika, wyrównanie i przepełnienie. Test `tools/bork-lsp-extension/test/server.test.js` pilnuje ścieżki binarki wobec `cargo run`.

Plik `tests/build.rs` jest kompilowany tylko z opcją `codegen`. Bez niej znika z zestawu. Nie dziw się, że samo `cargo test` nie widzi testu sumy pętli `for`.

Testy budowania piszą źródło do katalogu tymczasowego cargo, wołają binarkę bork z poleceniem build i ścieżką -o, a potem uruchamiają powstały program. Gdy dodasz zachowanie widoczne w czasie działania, dopisuj tu oczekiwany kod albo tekst wyjścia. Test jednostkowy samej emisji tego nie zastępuje. Listingi w tej książce opierają się na tym samym kontrakcie.

Gdzie szukać, gdy wynik nie zgadza się z oczekiwaniem

Gdy drzewo regionów nie ma węzła, którego oczekujesz, błąd jest w zejściu analizy własności, w `src/sema/walk.rs`, a nie w LLVM. Gdy drzewo jest, a bork build kończy się awarią kompilatora, błąd jest w emisji albo w funkcji `coerce_value_to_ty`. Wydruk drzewa dostaniesz poleceniem `cargo run --bin bork -- --dump-arenas plik.bork`.

Prefiks fazy mówi, którego katalogu nie czytać w pierwszej kolejności. Faza `parse` prowadzi do gramatyki. Faza `type` prowadzi do `src/typeck`. Faza `ownership` bez słów o użyciu po przeniesieniu i bez słów o braku kopiowania często pochodzi z `src/escape.rs`, a nie z `src/sema/policy.rs`. Słowa o regionie wewnętrznym, o `concat` i o wartości przeniesionej albo promowanej są z analizy ucieczki. Faza `codegen` prowadzi do kontroli albo do emisji.

Komunikat o wewnętrznej niezgodności harmonogramu regionów znaczy, że wspólny spacer nie dostał dziecka węzła albo dostał je w złym miejscu. Porównaj etykietę w analizie własności z miejscem w typie `RegionSite`. Najczęstsza przyczyna przy nowej konstrukcji jest taka, że analiza własności otwiera region, a gość emisji o nim nie wie, albo odwrotnie.

Awaria w `into_int_value` znaczy, że wartość LLVM nie jest liczbą całkowitą. Patrz, jaki typ ma wyrażenie. Napis i liczba zmiennoprzecinkowa na ścieżce `emit_call_with_values` są znanymi ofiarami. Poprawka należy do `coerce_value_to_ty`: dopasowanie wariantu wartości, a dla struktury przekazanie deskryptora bez rzutowania na `i64`. Nie zakrywaj tego łapaniem paniki.

Przerwanie procesu użytkownika bez komunikatu kompilatora to zwykle indeks albo dzielenie. Uruchom binarkę pod debuggerem i zobacz, czy stanęła w abort. Przepełnienie bufora regionu daje panikę Rusta z tekstem `arena overflow`, bo biblioteka wykonawcza jest pisana w Rustie i ta panika nie jest łapana przez program w Borku.

Weryfikacja modułu LLVM po emisji wywraca budowanie, gdy reprezentacja pośrednia LLVM jest zepsuta, na przykład przy złym typie scalenia gałęzi albo przy braku powrotu. To lepsze niż cichy zły kod. Gdy weryfikacja pada, błędu nie szukaj w clang.

WSKAZÓWKA

WSKAZÓWKA. Binarka bork bez opcji `codegen` jest szybka, gdy poprawiasz komunikaty sprawdzenia. Włączaj tę opcję dopiero wtedy, gdy sprawdzenie jest czyste i chcesz zobaczyć kontrolę przed generowaniem kodu albo sam proces.

Jak do języka doszła pętla while

Poniżej jest kolejność, którą widać po fakcie na while, break i continue. Nie jest to przepis skopiowany z planu. Jest to kolejność warstw, które musiały ruszyć, bo każda z nich w kodzie o pętli while wie.

Najpierw gramatyka. Dochodzi terminal i produkcja instrukcji. Dla while warunek jest w nawiasach, a ciało jest blokiem. break i continue są instrukcjami z zakresem źródłowym. Test w tests/parser.rs sprawdza, że nowa linia działa i że słowo nie jest identyfikatorem w złym miejscu.

Potem drzewo składni dostaje warianty instrukcji while, break i continue. Na tym etapie nie ma typów.

Potem sprawdzanie typów. Warunek oczekuje bool. Głębokość pętli rośnie tak samo jak przy for. break poza pętlą jest błędem typu. Dochodzi nowy wariant instrukcji w reprezentacji pośredniej. Test trzyma dokładny tekst komunikatu.

Potem analiza własności. Region WhileLoop otwiera się zwykłą ścieżką. Obowiązuje ten sam zakaz przeniesienia co przy for. Jeśli zapomnisz zakazu, istniejący test przeniesienia nazwy zewnętrznej w pętli nie pokryje while, dopóki go nie skopiujesz. Warto go skopiować.

Jeśli ciało jest blokiem, nie dokładaj drugiego regionu w sprawdzaniu typów. Analiza własności i reprezentacja pośrednia muszą mieć po jednym dziecku. Inaczej spacer się nie zepnie.

Nowa instrukcja musi być odwiedzona w analizie ucieczki i w wyniesieniu alokacji. Inaczej deklaracja wewnątrz while nie będzie widziana. Dla tej pętli wystarczy zejść w ciało tak, jak przy for.

We wspólnym spacerze dochodzi miejsce regionu i hak pętli while. Oznaczenie pobrania bufora zaczyna widzieć alokacje w ciebie. Bez tego wygenerowany kod i raport przestaną pasować do siebie w chwili, gdy ciało alokuje napis.

Pętla while nie potrzebuje odmowy w kontroli przed generowaniem kodu, bo emisja ją umie. Nowa konstrukcja, której emisja nie umie, musi dostać odmowę w src/codegen/gate.rs, żeby użytkownik dostał komunikat zamiast awarii kompilatora. Lekcją jest napis jako argument funkcji użytkownika. Kontrola go nie zna, a emisja kończy się awarią.

Emisja dokłada bloki podstawowe: nagłówek, ciało, zatrask, czyszczenie bufora, jeśli region go pobrał, oraz break jako skok do bloku wyjścia. Potem test w tests/build.rs ma konkretny kod wyjścia. Dla while z break przy wartości 3 jest to 3.

Na końcu dokument. docs/language.md, a jeśli ruszasz pamięć, także docs/memory-model.md. Plik TODO.md prosi, żeby dokument nadążał za kontrolą przed generowaniem kodu. Ta książka opisuje jedną rewizję i nie zaktualizuje się sama. Przy zmianie semantyki aktualizuj co najmniej docs/language.md.

Konstrukcja, która jest tylko skrótem składniowym, może skończyć się w parserze zamianą na istniejące drzewo. W tym repozytorium prawie nic się tak nie dzieje. Wyrażeniowe move i regionalne move są osobnymi węzłami. Nowy skrót lepiej zostawić osobnym węzłem, jeśli ma własny komunikat.

Braki, które są już zapisane

Plik `TODO.md` wymienia między innymi rozcięcie gościa generowania kodu, żeby emiter funkcji nie był jednocześnie gościem spaceru regionów. Wymienia kursor regionów zamiast makra, gdy makro dalej urośnie. Wymienia test, że harmonogram pętli to jedno wejście, wiele czyszczeń i jedno wyjście. Wymienia synchronizację kontroli przed generowaniem kodu z frontendem. Wymienia błędy spaceru zawsze jako błąd z komunikatem, bez gubienia tekstu użytkownika w opakowaniu o niezgodności harmonogramu. Wymienia więcej testów par wejście-wyjście na zagnieżdżonym warunku i na miejscach przeznaczenia napisów. Prosi też, żeby nie commitować luźnych plików `smoke.bork` w korzeniu repozytorium.

Komunikat o niezgodności harmonogramu wokół liczb zmiennoprzecinkowych jest przykładem dwóch z tych punktów. Tekst użytkownika, że dodawanie zmiennoprzecinkowe nie jest obsługiwane, utonął w opakowaniu. Kontrola przed generowaniem kodu w ogóle nie powiedziała, że liczba zmiennoprzecinkowa nie jest tłumaczona.

Czego nie robić w pierwszej poprawce

Nie zaczynaj od LLVM. Dopisz test parsera albo sprawdzania typów, który na czerwono nazywa zachowanie, i doprowadź `frontend::check` do tego tekstu. Odmowę w kontroli przed generowaniem kodu dodaj w tym samym patchu, jeśli emisji nie będzie. Brak takiej odmowy przy nowej składni zamienia przyszłą awarię kompilatora w coś, co użytkownik zobaczy dopiero jako panikę.

Nie zmieniaj `peel_blocks` tylko w jednym z dwóch miejsc.

Nie naprawiaj awarii `into_int_value` przez odrzucenie wszystkich napisów w kontroli, jeśli w `main` napisy działają. To obciążłoby listingi, które są legalne. Wąskie miejsce to rzutowanie argumentu wywołania.

Podsumowanie

- Polecenie `cargo test --workspace` nie uruchamia LLVM. Opcja `codegen` uruchamia testy w `tests/build.rs`.
- Wydruk regionów debuguje analizę własności. Awaria `into_int_value` debuguje emisję. Niezgodność harmonogramu debuguje wspólny spacer.
- Nowa konstrukcja idzie warstwami: gramatyka, drzewo składni, sprawdzanie typów, analiza własności, ucieczka i wyniesienie, spacer, kontrola, emisja, test binarki.
- Jeśli emisji nie ma, odmowa w kontroli przed generowaniem kodu jest obowiązkowa.
- Plik `TODO.md` jest listą znanych braków. Sprawdź ją, zanim opiszesz różnicę dokumentu i kodu jako własne odkrycie.

Rozdział 22. Ćwiczenia

Ten rozdział obejmuje

- ćwiczenia na samo sprawdzenie programu, bez generowania kodu
- ćwiczenia na bork build i na kod wyjścia procesu
- ćwiczenia czytania kodu kompilatora
- odpowiedzi, które da się zestawić z komunikatem albo z plikiem

Odpowiedzi są w tym samym rozdziale, pod treścią zadań. Najpierw uruchom kompilator. Komunikaty z części pierwszej wolno potraktować jako ściągę.

Część A. Samo sprawdzenie

A1. Napisz funkcję main, która zwraca i32 i nie ma ani jednego return. Jaki jest dokładny początek komunikatu, czyli faza i pierwsze słowa?

A2. Zadeklaruj `var s = "hi"` i `var t = s`. Popraw program najmniejszą zmianą tak, żeby sprawdzenie przeszło, a nazwy `s` nie dało się użyć niżej.

A3. Poniższy program jest poprawny i odejmuje trzy od zera. Zamień `0 - 3` na `-3` i nazwij fazę błędu.

```
fun main(): i32 {  
    val n = 0 - 3  
    return n  
}
```

A4. Program `val a: [i32; 2] = [1, 2, 3]` daje więcej niż jeden komunikat. Ile ich jest, jakiej są fazy i czego dotyczą? Nie poprawiaj programu.

A5. Napisz pętlę for po zakresie typu i64. Jakiego typu granic wymaga sprawdzanie?

A6. Weź fragment z dokumentacji języka, dopisz brakującą deklarację out i spraw, żeby sprawdzenie przeszło na dwa sposoby: przez stałe val oraz przez move.

```
var left = "L"  
var right = "R"  
{  
    var piece = concat(left, right)  
    out = move piece  
}
```

A7. W funkcji zwracającej String napisz `return concat("a", "b")`. Czy komunikat ma numer linii?

Część B. Zbudowany program

Uruchom bork build i powstały proces.

B1. Ile wynosi kod wyjścia sumy `for (i in 0..5)`, gdy w ciele jest `total = total + i`, a `total` zaczyna się od zera?

B2. Pętla `while` podbija `i` od zera i robi `break` przy `i == 3`, zanim zwiększy licznik. Co zwraca `main`?

B3. W funkcji `main` bez zadeklarowanego typu wyniku jest samo `println("hi")`. Co jest na standardowym wyjściu i jaki jest kod wyjścia?

B4. Program zwraca `1 / 0`. Czy bork build kończy się błędem? Co robi proces?

B5. Program ma `val a = [10, 20, 30]` i odczyt `a[9]`. Czy sprawdzenie przechodzi?

B6. Weź listing 20.1 i zmień zakres na `0..4`. Jaki kod wyjścia przewidujesz, zanim uruchomisz program, i dlaczego?

B7. Program to `fun main(): i64 { return 1 }`. Czy pada sprawdzenie, czy dopiero budowanie, i jaka jest faza?

Część C. Kod kompilatora

C1. Wskaż plik i funkcję, które ustawiają kolejność wypisu: najpierw komunikaty analizy własności, potem komunikaty typów.

C2. Gdzie jest pojemność 4096? Podaj dwa pliki. Który wykonuje się w procesie użytkownika?

C3. Dlaczego `--dump-arenas` przy urwanym `fun oops` (nie drukuje drzewa)?

C4. Pole przechwycenia bloku `move` bywa puste albo bywa pustą listą. Który wariant zgaduje nazwy?

C5. Dlaczego reprezentacja pośrednia nie ma pola z numerem regionu? Skąd generator kodu bierze region?

C6. Która funkcja w `src/sema/policy.rs` zwraca współdzielenie?

C7. Co trzeba zmienić, żeby przekazanie napisu do funkcji użytkownika dało komunikat zamiast awarii kompilatora? Nie musisz pisać poprawki. Nazwij funkcję, która się wywraca, i powiedz, czemu kontrola przed generowaniem kodu tego nie łapie.

C8. Masz nowy operator dwuargumentowy, którego LLVM jeszcze nie tłumaczy. Wymień warstwy z rozdziału 21 w kolejności i zaznacz, która jest obowiązkowa, żeby nie było awarii kompilatora.

Odpowiedzi do części A

- A1.** Początek komunikatu to `type: function must return a value of type i32 on all paths`. Zakres bywa pusty, więc numer linii może zniknąć z prefiksu.
- A2.** Zapis `var t = move s` przenosi własność. Przy gołym `s` komunikat każe użyć `move s`, żeby przenieść własność.
- A3.** Zapis `return -3` jest fazą `parse` i mówi o nieoczekiwanym tokenie. Zapis `0 - 3` jest odejmowaniem. Dla `main` zwracającego `i32` sprawdzenie jest zadowolone. Kod wyjścia procesu dla wartości ujemnej owija się w zakresie `i32`.
- A4.** Są dwie diagnostyki fazy `type`. Literał ma 3 elementy, a oczekiwano 2. Inicjalizator ma typ `[i32; 3]`, a oczekiwano `[i32; 2]`.
- A5.** Komunikat mówi, że granice zakresu muszą mieć typ `i32`.
- A6.** Na zewnątrz bloku deklarujesz `var out` z jakimś napisem początkowym, na przykład "prefix". Albo `left` i `right` są stałymi `val`, albo wywołanie ma postać `concat(move left, move right)`. Wynikowy napis to `LR`, a nie `prefixLR`, bo `concat` zastępuje deskryptor, a nie dokleja znaki do starego bufora zmiennej `out`.
- A7.** Nie. Analiza ucieczki stawia pusty zakres. Zostaje komunikat fazy `ownership` o zwracaniu wyniku `concat`, bez numeru linii.

Odpowiedzi do części B

- B1.** Kod wyjścia to 10.
- B2.** Kod wyjścia to 3.
- B3.** Standardowe wyjście to `hi` i nowa linia. Kod to 0, bo `main` bez typu zwraca `unit`, a generowanie kodu zamienia to na `i32` równe zero.
- B4.** Budowanie się udaje. Proces kończy się sygnałem przerwania, w powłoce kodem 134.
- B5.** Sprawdzenie przechodzi. Proces jest przerywany. Indeks nie jest częścią typu. Częścią typu jest długość.
- B6.** Suma `0 + 1 + 2 + 3` wynosi 6, bo zakres `0..4` jest otwarty z prawej strony.
- B7.** Sprawdzenie przechodzi. Budowanie zgłasza fazę `codegen` i mówi, że `main` zwracające `i64` nie jest jeszcze obsługiwane.

Odpowiedzi do części C

- C1.** Robi to `frontend::check` w `src/frontend.rs`. Sprawdzanie typów wykonuje się wcześniej i zwraca komunikaty. Analiza własności zwraca własne błędy. Wektor buduje się najpierw z nich, a dopiero potem dopisuje błędy typów.
- C2.** Stała `ARENA_CAPACITY` jest w `crates/bork_runtime/src/lib.rs`. Ten plik wykonuje się w procesie użytkownika. Tę samą pojemność opisuje `src/arena.rs`. To model w kompilatorze. Analiza własności `go` nie woła.

C3. Przy błędzie składni pole raportu w wyniku sprawdzenia jest puste. Wiersz poleceń drukuje drzewo tylko wtedy, gdy raport jest obecny.

C4. Brak listy zgaduje nazwy. Pusta lista to jawne move `()` i oznacza, że nic nie jest przenoszone.

C5. Komentarz w `src/hir/mod.rs` mówi, że regiony żyją w raporcie z analizy własności. Generator kodu bierze je we wspólnym spacerze, zsynchronizowanym z reprezentacją pośrednią.

C6. Współdzielenie zwraca `classify_use`, w gałęzi wiązania stałego, po sprawdzeniu, że typ nie jest kopiowalny i że region nie jest ten sam.

C7. Wywraca się `value_as_int`, przez `into_int_value`, wołane z `coerce_value_to_ty` przy emisji wywołania po spacerze. Kontrola przed generowaniem kodu ogląda kształt reprezentacji pośredniej: czy jest `Some`, funkcja na końcu wywołania, niedozwolony operator. Nie pyta, czy argument jest strukturą LLVM. Wywołanie z argumentem napisowym wygląda jak zwykłe wywołanie, więc kontrola milczy.

C8. Warstwy to gramatyka, drzewo składni, sprawdzanie typów, analiza własności, jeśli operator rusza nazwy, analiza ucieczki, jeśli rusza bajty, wspólny spacer, jeśli otwiera region, kontrola przed generowaniem kodu, potem emisja i test w `tests/build.rs`. Obowiązkowa przeciw awarii kompilatora jest kontrola, dopóki emisja wyrażeń nie ma gałęzi dla operatora. Inaczej dopasowanie operatora wpadnie w komunikat o braku wsparcia albo, co gorsza, w ścieżkę liczby całkowitej albo zmiennoprzecinkowej i w złe rzutowanie wartości LLVM.

Podsumowanie

- Ćwiczenia A kończą się na tekście komunikatu. Ćwiczenia B kończą się na kodzie procesu.
- Ćwiczenia C wskazują pliki, a nie ogólniki.
- Zakres `0..n` jest otwarty z prawej strony. Suma `0..5` to 10. Suma `0..4` to 6.
- Kontrola przed generowaniem kodu nie zastępuje rzutowania argumentu wywołania. Stąd osobna odpowiedź C7.

Dodatek A. Zestawienie składni

Ten dodatek zbiera składnię, którą przyjmuje czytanie programu w tej rewizji kompilatora. Gwiazdka przy wierszu znaczy, że sprawdzenie program przyjmuje, a bork build albo odrzuca go komunikatem, albo kończy się awarią kompilatora. Szczegóły są w dodatku C. Zdania wokół zestawienia mówią, jak czytać skrót. Sam skrót jest po to, żeby nie szukać reguły w całym rozdziale.

Program i parametry

Funkcja ma nazwę, listę parametrów w nawiasach i ciało w nawiasach klamrowych. Typ wyniku, jeśli jest, stoi po dwukropku przed ciałem. Brak typu wyniku znaczy unit.

```
fun nazwa(parametry) { instrukcje }  
fun nazwa(parametry): Typ { instrukcje }
```

Parametr zapisuje się jako nazwa: Typ, val nazwa: Typ albo var nazwa: Typ. Goły parametr, bez val i bez var, jest stały. Instrukcje rozdziela przejście do nowego wiersza, a nie średnik. Komentarz zaczyna się od // i trwa do końca wiersza.

Typy

Typy proste to i8, i16, i32, i64, u8, u16, u32, u64, f32, f64, bool i unit. Aliasy Int, Long, Byte, Float i Double znaczą odpowiednio i32, i64, u8, f32 i f64. Napis to String. Tablica o znanej długości to [T; N]. Wartość, która może być pusta, to T?. Typ funkcji to (A, B) -> R. Może też sam być pusty, na przykład ((A) -> R)?.

Kopiuwalne są tylko typy proste, które nie mogą być puste. Napis, tablica, typ funkcji i wartość pusta kopiuwalne nie są.

Instrukcje

```
val nazwa = wyrażenie  
val nazwa: Typ = wyrażenie  
var nazwa = wyrażenie  
var nazwa: Typ = wyrażenie  
nazwa = wyrażenie  
nazwa[indeks] = wyrażenie  
{ instrukcje }
```

```

for (nazwa in lo..hi) { instrukcje }
while (warunek) { instrukcje }
break
continue
return
return wyrażenie
move { instrukcje }
move () { instrukcje }
move (a, b) { instrukcje }
wyrażenie

```

Pętla for idzie po zakresie lo..hi typu i32 i nie wykonuje ciała dla wartości hi. Pętla while wymaga warunku typu bool. break i continue są legalne tylko wewnątrz pętli.

Wyrażenia

Najmocniej wiążą się wywołanie, indeks, wycinek, odczyt pola i przyrostkowe !!. Potem negacja !. Potem mnożenie i dzielenie. Potem dodawanie i odejmowanie. Potem porównania. Potem zakres ... Potem koniunkcja &&. Potem alternatywa ||. Najslabiej wiąże się ?: . Operator ?: jest prawostronnie łączny. Pozostałe operatory dwuargumentowe są lewostronnie łączne.

```

nazwa
liczba
liczba.liczba
"napis"          \n \r \t \\ \"
true false
[e, e, e]
nazwa[i]
nazwa[lo..hi]    lo i hi są literałami i32
nazwa.pole
nazwa?.pole
wyrażenie!!
wyrażenie ?: wyrażenie
Some(wyrażenie) None
move nazwa
promote nazwa
f(a, b)
f(a) { x -> instrukcje }          *
f(a) move (s) { x -> instrukcje } *

```

Jednoargumentowego minusa nie ma. Liczbę ujemną zapisuje się odejmowaniem, na przykład 0 - 1.

Warunek if (warunek) { } else { } jest wyrażeniem. Obie gałęzie są blokami. Bez else, gdy nikt nie oczekuje typu, wynikiem jest unit.

Własność nazw w skrócie

Gdy typ jest kopiowalny, wystarcza goła nazwa. Gdy stała z regionu zewnętrznego nie jest kopiowalna, też wystarcza goła nazwa: to współdzielenie. Gdy zmienna z regionu zewnętrznego nie jest kopiowalna, trzeba napisać `move nazwa`. To samo dotyczy przekazania stałej niekopiowalnej do parametru zmiennego. Literał napisowy do parametru zmiennego nie wymaga `move`, ale generowanie kodu dla napisu jako argumentu funkcji użytkownika dziś kończy się awarią kompilatora. Zapis do zmiennej zewnętrznej ma postać `outer = wyrażenie`, `outer = move inner` albo `outer = promote held`. Zwrot napisu to `return nazwa` na głębokości funkcji albo `return literału`. Zwrot wyniku `concat` oraz zwrot przeniesienia z regionu wewnętrznego są odrzucane.

Funkcje wbudowane i polecenia

`print` i `println` przyjmują `i32`, `i64` albo `String` i zwracają `unit`, mimo że ich nominalna sygnatura mówi o `i32`. `concat` bierze dwa napisy i zwraca napis.

```
bork plik.bork
bork --dump-arenas plik.bork
bork build plik.bork
bork build -o wyjście plik.bork
cargo test --workspace
cargo test --workspace --features codegen
```

Kod zero oznacza sukces. Kod jeden oznacza zły program. Kod dwa oznacza złe wywołanie albo brak narzędzia. Kod 101 oznacza awarię procesu `bork`. Kod 134 w powłoce oznacza przerwanie procesu użytkownika przez `abort`.

Fazy komunikatów to `parse`, `ownership`, `type` i `codegen`.

Dodatek B. Słownik pojęć

Każde hasło jest zdaniem albo krótkim akapitem. Angielski termin z kodu podaję raz, obok polskiego, i dalej trzymam polski, chyba że w zdaniu chodzi o nazwę w źródle.

Analiza ucieczki. Przejście w pliku `src/escape.rs`, które liczy głębokość bajtów i odrzuca użycie, które przeżyłoby zwolnienie bufora regionu. Komunikat ma fazę `ownership`. W kodzie funkcja licząca głębokość nazywa się `place`.

Analiza własności. Przejście w katalogu `src/sema`, które buduje drzewo regionów i decyduje, czy nazwę wolno skopiować, współdzielić albo trzeba przenieść. Po angielsku `semantic analysis`. Nie przydziela buforów.

Arena. Bufor jednego regionu, o pojemności 4096 bajtów, z przesuwającym się wskaźnikiem. W źródłach ta sama nazwa oznacza jeszcze węzeł drzewa regionów, `ArenaNode`. To nie jest ten sam obiekt. Rozdział 2 i rozdział 14 rozdzielają te znaczenia.

Drzewo składni. Nietypowany wynik parsera, w kodzie `AST`, od angielskiego `abstract syntax tree`. Definicja jest w `src/ast.rs`.

Głębokość. Liczba regionów między funkcją a miejscem, w którym leżą bajty. Zero to region funkcji. Analiza ucieczki porównuje głębokość bajtów z głębokością odbiorcy.

Kontrola przed generowaniem kodu. Zejście po reprezentacji pośredniej w `src/codegen/gate.rs`, zanim powstanie moduł LLVM. Odrzuca konstrukcje, których emisja nie tłumaczy, komunikatem fazy `codegen`. Nie łapie każdego miejsca, w którym emisja kończy się awarią.

Kopiowanie. Przekroczenie granicy regionu przez wartość typu prostego, która nie może być pusta. Źródło zostaje żywe. W wydruku drzewa znacznik to `Copy`.

Miejsce przeznaczenia. Odbiorca bajtów wyniku: cel przypisania, powrót albo wynik `concat`. Steruje tym, w której arenie powstanie bufor wyniku. W kodzie bywa nazywane `sink`.

Promocja. Zapis `promote` nazwa po prawej stronie przypisania do zmiennej z regionu zewnętrznego. Kopiuje bajty do areny tej zmiennej i unieważnia źródło.

Przeniesienie. Zapis `move`, po którym źródłowa nazwa jest martwa. Bajty w starej arenie zostają niedostępne aż do wyczyszczenia bufora. W wydruku drzewa znacznik to `Moved`.

Pula buforów. Lista wolnych aren w bibliotece wykonawczej. Zwrot bufora oddaje go na listę. Wyczyszczenie wskaźnika bufora nie oddaje go na listę. Pobranie bierze bufor z listy albo alokuje nowy.

Reprezentacja pośrednia. Drzewo programu po sprawdzeniu typów. Po angielsku high-level intermediate representation, w kodzie HIR. Niesie typ i rodzaj użycia nazwy. Nie niesie numeru regionu.

Region. Fragment programu i czas życia pamięci z nim związany: ciało funkcji, blok, gałąź warunku, ciało pętli albo funkcja dopisana na końcu wywołania, o ile zdjęcie opakowań nie skleiło go z otoczeniem.

Rodzaj użycia. Adnotacja na nazwie w reprezentacji pośredniej: użycie lokalne, współdzielenie, kopia, przeniesienie albo promocja. W kodzie UseKind. Nie zastępuje analizy własności.

Sprawdzenie. Funkcja `frontend::check`: czytanie składni, sprawdzanie typów, analiza własności, a przy braku błędów także oznaczenie buforów, wyniesienie alokacji i analiza ucieczki.

Typ nieznany. Typ w reprezentacji pośredniej, który nie ma odpowiednika w składni. Tłumi część dalszych komunikatów o typie. Nie jest kopiowalny. W kodzie Unknown.

Wartość pusta. Typ z dopiskiem `?`. Sprawdzanie typów ją rozumie. Generowanie kodu jej nie tłumaczy.

Współdzielenie. Odczyt stałej, której nie wolno skopiować, z regionu zewnętrznego. Region wewnętrzny nie kopiuje bajtów i nie unieważnia nazwy. W wydruku drzewa znacznik to Shared.

Wyniesienie alokacji. Rozpoznanie dwóch sąsiednich instrukcji i ustawienie pola `alloc_in_binding`, żeby bajty powstały od razu w arenie zmiennej docelowej. Plik `src/hoist.rs`.

Zakres źródłowy. Para przesunąć bajtowych w pliku. W kodzie Span. Część komunikatów go nie ma. Wiersz poleceń pomija wtedy numer linii.

Zdjęcie opakowań. Usunięcie bloków, które zawierają tylko jeden wewnętrzny blok, żeby nie tworzyć regionu na same nawiasy. W kodzie `peel_blocks`. Licznik zdjętych nawiasów ląduje w `compacted_braces`.

codegen_push. Flaga na węźle regionu. Wygenerowany kod woła `bork_arena_push` tylko wtedy, gdy flaga jest ustawiona. Korzeń funkcji ma ją zawsze.

decl_tys. Wektor typów deklaracji w kolejności źródła, przekazany ze sprawdzania typów do analizy własności.

loop_move_ban. Zbiór nazw, których nie wolno przenieść w ciele pętli, bo były widoczne na wejściu do pętli.

unit. Typ funkcji bez zadeklarowanego wyniku. W składni pisze się małymi literami. Nie nazywa się Unit.

Dodatek C. Czego kompilator jeszcze nie potrafi

Ten dodatek wymienia miejsca, w których kod jest węższy niż to, co rozumie sprawdzenie programu, dokumentacja mówi co innego niż kod, albo kompilator kończy się awarią zamiast komunikatem. Nic z tej listy nie jest propozycją naprawy. Książka nie zmienia kompilatora.

Rzeczy świadomie niezaimplementowane

Z pliku `src/escape.rs`, z `docs/language.md` i z `docs/memory-model.md` wynika następująca lista. Nie ma areny wyniku po stronie wywołującego, więc zwrot przeniesienia, zwrot wyniku `concat` i zwrot bajtów z regionu wewnętrznego są odrzucane. Promocja na powrocie z funkcji też jest odrzucana. Wyniesienie alokacji działa tylko dla dwóch sąsiednich instrukcji. Nie ma usuwania zbędnego kopiowania pamięci, gdy bajty już leżą w buforze celu. Nie ma osobnej areny tymczasowej na czas zwykłego wywołania. Generowanie kodu nie tłumaczy funkcji dopisanej na końcu wywołania, słów `Some` i `None`, wykrzykników `!!`, operatora `?:` ani pól innych niż `length`. Funkcja `main` nie ma parametrów i nie zwraca typu innego niż `i32` i `unit`. Wywołanie pośrednie, w którym rzeczą wywoływaną nie jest nazwa, nie jest tłumaczone. Nie ma modułów, struktur, typów ogólnych, wyjątków, sprawdzania pożyczek ani odśmiecania.

Plik `TODO.md` wymienia prace nad samym kompilatorem, a nie nad semantyką języka. Są to rozcięcie gościa generowania kodu, kursor regionów zamiast makra, test zgodności kontroli przed generowaniem kodu ze sprawdzeniem, więcej testów par wejście-wyjście oraz uporządkowanie planów w `docs/superpowers/plans/`.

Gdzie dokument mówi co innego niż kod

Przykład `concat(left, right)` przy dwóch zmiennych napisowych w `docs/language.md` jest pokazany jako wzorzec. Sprawdzenie odrzuca go komunikatem, że `left` nie jest kopiowalne i trzeba je przenieść do bloku. Zdanie o kolejności operatorów w tym samym dokumencie pomija `&&`, `||` i `!`. Gramatyka te piętra ma.

Notatka projektowa z 23 września 2026 o reprezentacji pośredniej i LLVM mówi o LLVM 18 i o kolejności „najpierw własność, potem typy”. Kod używa LLVM 23, a sprawdzanie typów wykonuje się przed analizą własności. Specyfikacja serwera edytora mówi o komunikatach parsera. Kod woła pełne `frontend::check`. Specyfikacja

pierwszej wersji regionów mówi o braku pełnego sprawdzania typów. Sprawdzanie typów jest. Nagłówek w README mówi o diagnostyce składni, a akapit niżej i kod robią pełne sprawdzenie. Specyfikacja parsera mówi o wyniku Unit i o braku zakresów źródłowych. W kodzie typ nazywa się unit i zakresy są. Dokument języka mówi, że dzielenie minimalnej liczby całkowitej przez minus jeden przerywa program. Strażnik w guard_int_div ten przypadek obsługuje, ale literału ujemnego nie da się zapisać, bo nie ma jednoargumentowego minusa.

Awarie i ostre krawędzie, które zostały uruchomione

Kompilator był złożony z opcją codegen, na LLVM 23.1.2 i rustc 1.98.1.

Przekazanie napisu do funkcji użytkownika, czy literałem, czy przez move, czy przez stałą, przechodzi sprawdzenie i kontrolę przed generowaniem kodu, a potem kompilator kończy się awarią w src/codegen/llvm/expr.rs, w funkcji value_as_int, na wywołaniu into_int_value. Kod procesu kompilatora to 101.

Porównanie f64, na przykład warunek $x > 1.0$ przy x typu f64, kończy się awarią w tym samym miejscu, bo wartość jest liczbą zmiennoprzecinkową LLVM. Dodawanie f32 daje komunikat, a nie awarię. Treść mówi o wewnętrznej niezgodności harmonogramu i o tym, że dodawanie zmiennoprzecinkowe po spacerze nie jest jeszcze obsługiwane. Dwa operatory na liczbach zmiennoprzecinkowych kończą się na dwa różne sposoby.

Indeks poza zakresem i dzielenie przez zero kompilują się. Proces użytkownika kończy się sygnałem przerwania, w powłoce kodem 134, bez tekstu z Borka.

Przepełnienia bufora nie uruchamiałem dużym literałem w książce. Krótki przykład trudno przepchnąć ponad 4096 bajtów bez pętli, która alokuje, a pętla czyści bufor przy obiegu. Kod w bibliotece wykonawczej przerywa się tekstem arena overflow, z podaną liczbą bajtów i pojemnością 4096. Test jednostkowy biblioteki to zamyka. To nie jest komunikat kompilacji.

Zwrot wyniku concat i część błędów analizy ucieczki nie mają zakresu źródłowego. Wiersz poleceń pomija wtedy linię i kolumnę.

Błąd zgłoszony przez regułę gramatyki, na przykład zły cel przypisania albo za duża liczba, wskazuje koniec pliku.

Parametry funkcji dopisanej na końcu wywołania mają w analizie własności typ nieznany. Wydruk drzewa pokazuje współdzielenie dla parametrów, które sprawdzanie typów uważa za kopiowalne liczby. Sprawdzenie przechodzi. To myli przy czytaniu --dump-arenas. Nie jest błędem użytkownika.

Komunikat analizy ucieczki o gałęzi warunku mówi o napisie. Warunek w kodzie obejmuje każdy typ trzymany w buforze regionu, także tablicę.

To, że u32 albo alias Byte nie może być wynikiem main zwracającego i32, nie jest ograniczeniem szerokich liczb całkowitych w ogóle. Funkcja pomocnicza na i64 i porównanie w main działają. Na sprawdzonym programie z odejmowaniem kod wyjścia wyniósł 7. Ograniczenie main dotyczy typu wyniku main, a nie lokalnej wartości i64.

Są dwie stałe 4096: w src/arena.rs i w bibliotece wykonawczej. Analiza własności nie woła żadnej. Łatwo poprawić zły plik.

Opakowanie o wewnętrznej niezgodności harmonogramu potrafi ukryć zwykły komunikat o braku wsparcia w tekście, który brzmi jak błąd wewnętrzny kompilatora. Czasem niezgodność naprawdę dotyczy dzieci drzewa regionów. Czasem jest tylko przedrostkiem.

Specyfikacje w docs/superpowers opisują świat sprzed tablic, sprzed pętli while i sprzed LLVM 23. Plik TODO.md każe je przenieść, gdy wchłoną się w dokument albo w zamknięte poprawki. Nie są mapą bieżącego kodu.

Co jest pokryte testami

Żeby lista braków nie przesłoniła reszty: parser, sprawdzanie typów liczb i tablic, reguła kopiowania, współdzielenia i przeniesienia, wydruk drzewa regionów, pętle for i while z break i continue, wypisywanie liczb i napisów w main, concat i promocja w main, wycinki o granicach będących literałami oraz konsolidacja z biblioteką wykonawczą są pokryte testami w tests/build.rs albo pełnym sprawdzeniem. Zostały powtórzone przy pisaniu tej książki. Braki są na brzegach: wartości puste w generowaniu kodu, liczby zmiennoprzecinkowe, napis jako argument wywołania i powrót świeżego napisu.

Kolejność zaufania

Gdy komentarz, specyfikacja i kod się różnią, kolejność przyjęta w książce jest taka. Najpierw test wykonawczy w tests/build.rs. Potem frontend::check na przykładzie. Potem kod fazy. Potem docs/language.md. Na końcu docs/superpowers. Zdanie w README o braku sprawdzania pożyczek i braku odśmiecania zgadza się z modelem. Przykład concat na dwóch zmiennych napisowych się nie zgadza.